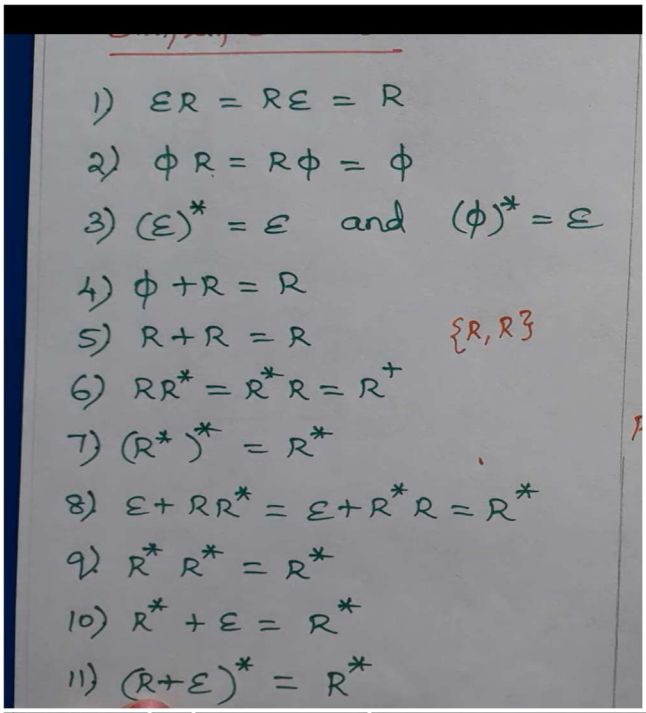
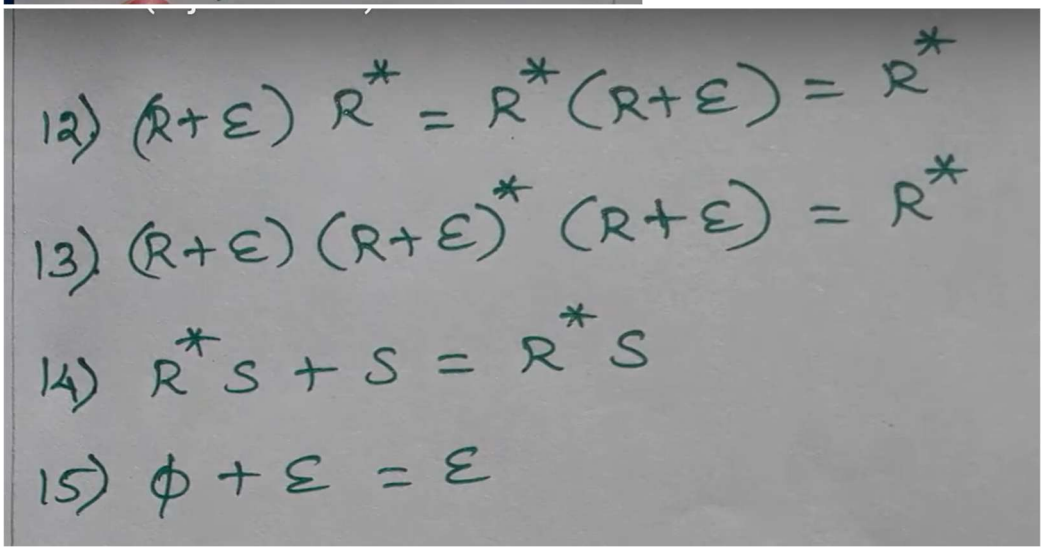
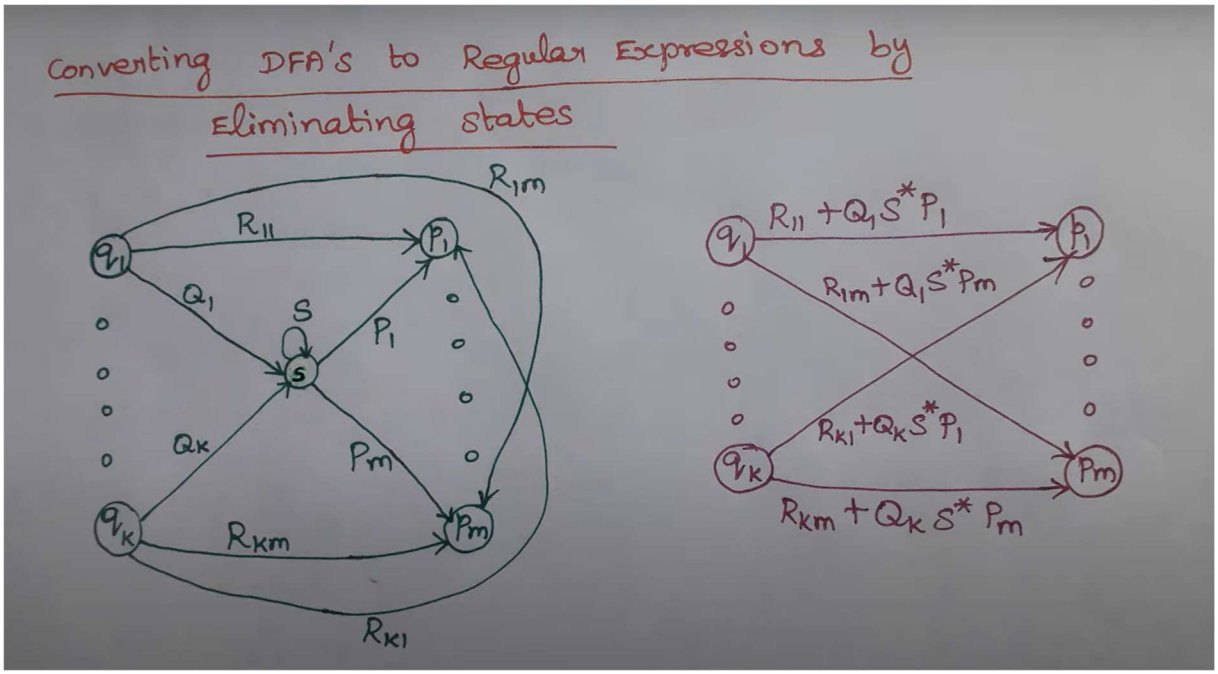
Theoretische Informatik

Es geht hier erstmal um Reguläre Ausdrücke und wie man einen Regulären Ausdruck zu einem Automaten konvertiert und umgekehrt(viel wichtiger ist es, wie man einen Automaten zu einem regulären Ausdruck konvertiert)





Convert a DFA to RE by State Elimination Method



The state that we want to eliminate is the state S.

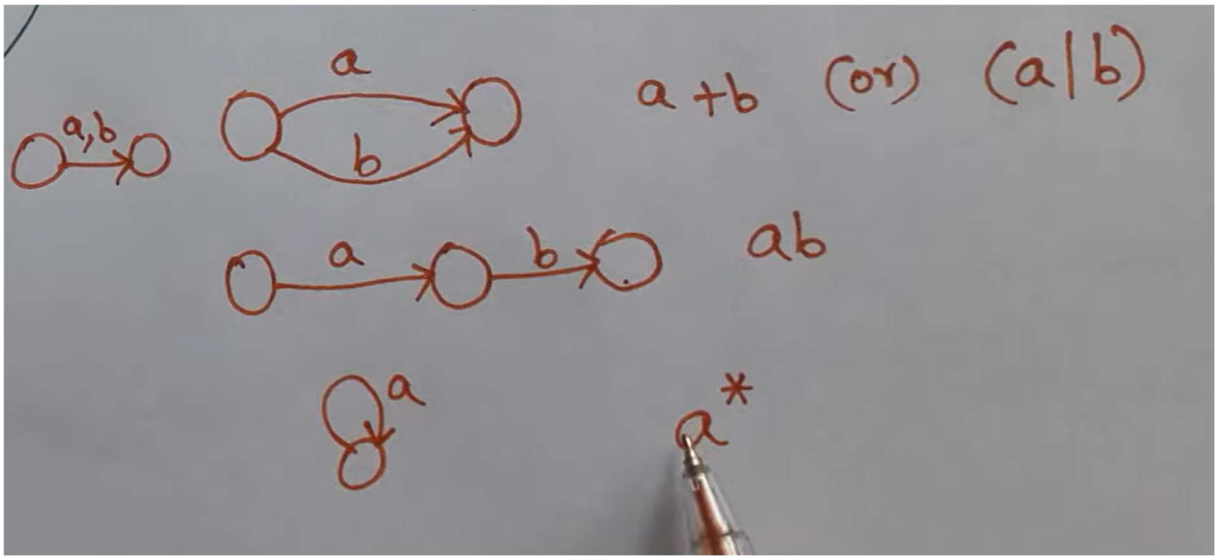
Example from the state q_1 to the state p_m, there is a direct transition between the both states R_{1m} and there is a transition from q_1 to the state p_m through the state S, that we want to the state that we want to eliminate

Q_1 is a transition from the predecessor state to the successor state

S is self-transition

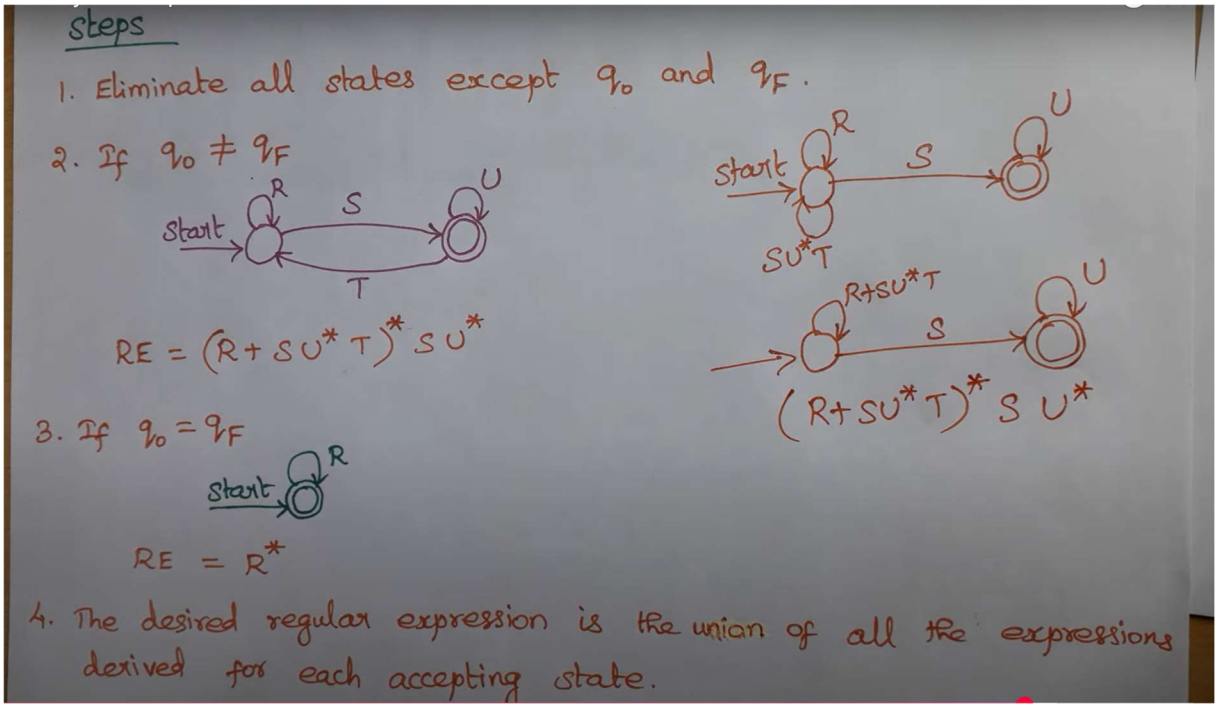
P_m is a transition from the eliminated state to the successor state

Therefore, we have $R_{1m} \mid Q_1 S^* P_m$



If you have more than one transition from one state to another state, that means you have two possibilities to go from one state to another state , the regular expression is the union of the two regular expressions , here it is (a|b)

Self- transition means a* and concatenation means you should follow only this path to get from one state to the other state.



This is the Algorithm of how to convert the DFA to a Regular expression

You should eliminate all states except the final and start state, then you are left with two states , they could have this form shown above.

R is self- transition from the start state to itself

S is a transition from the start state to the final state

T is transition from the final state to the start state

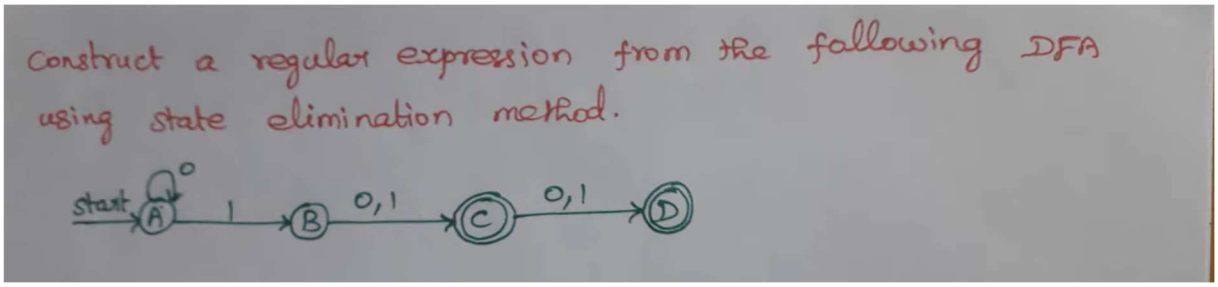
U is a self-transition from the final state to itself

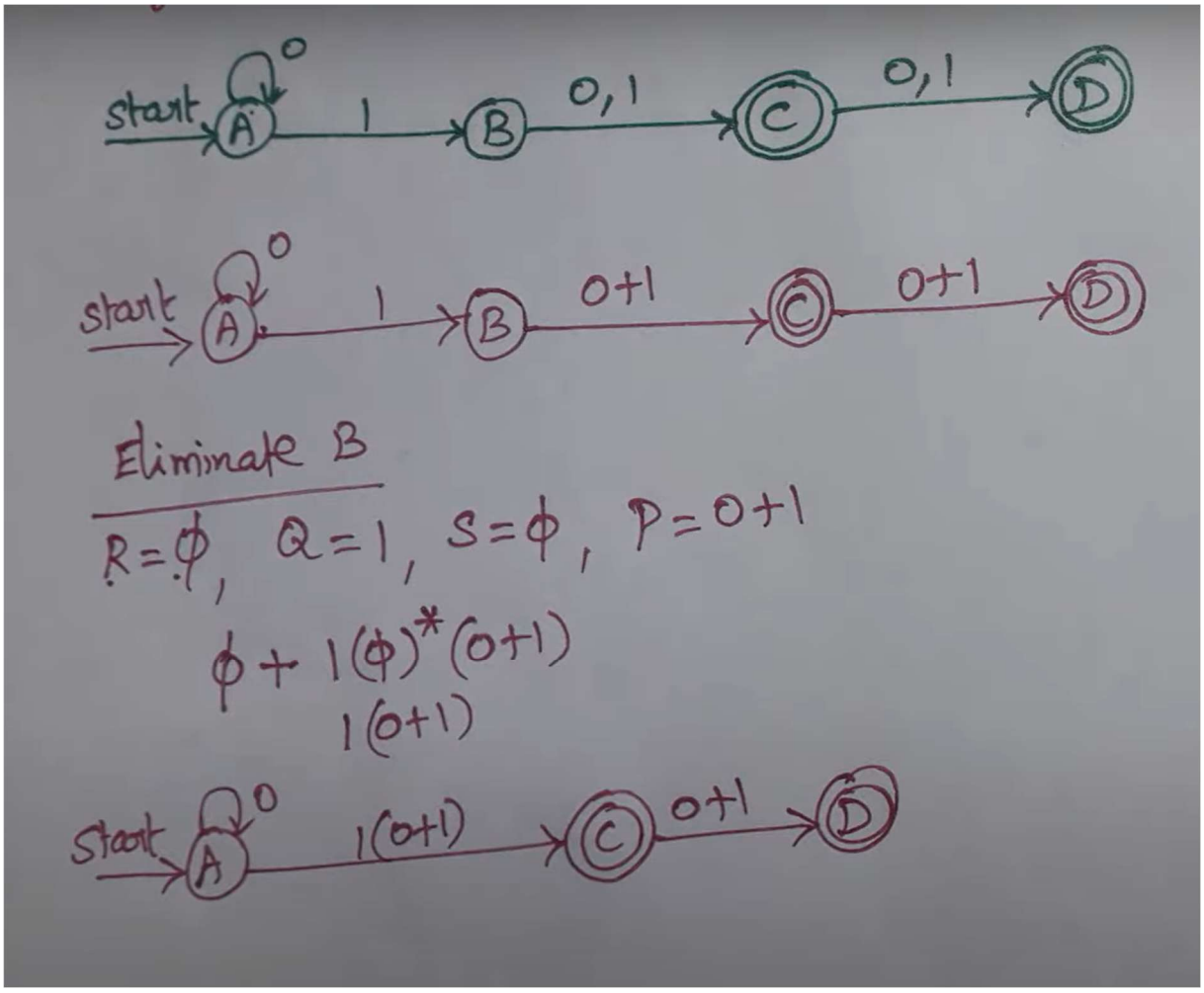
Am Ende entsteht diesen regulären Ausdruck **(R + S U*T)* S U***

There is another case, if the start state is = final state , so the final obtained regular expression is R * .

If you have more than one final state, then the final regular expression is the union of all regular expressions obtained from all states.

Example


Construct a regular expression from the following DFA using state elimination method.
start
A
B
C
D
0
1
0,1
0,1


start
A
B
C
D
0
1
0,1
0,1
start
A
B
C
D
0
1
0+1
0+1
Eliminate B
R=ϕ, Q=1, S=ϕ, P=0+1
ϕ + 1(ϕ)*(0+1)
1(0+1)
Start
A
C
D
0
1(0+1)
0+1

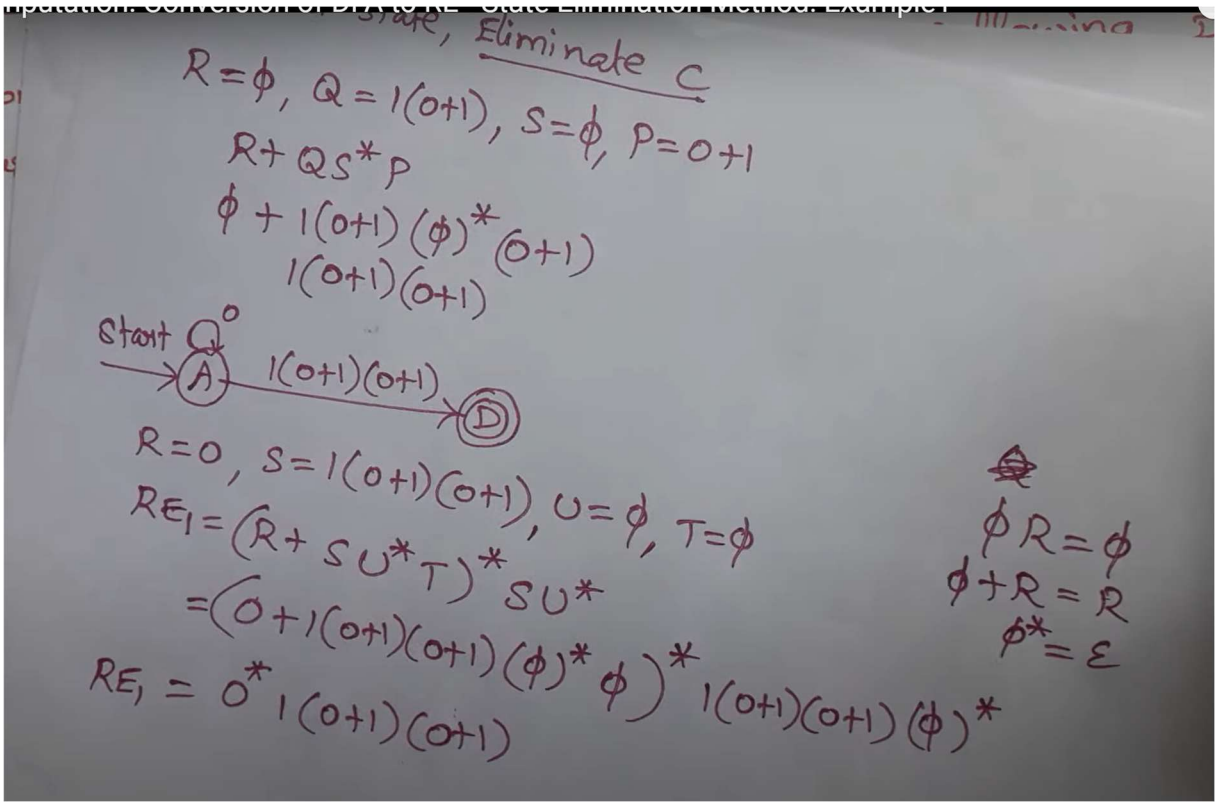
D → Final state, Eliminate C

$R=\phi,\ Q=1(0+1),\ S=\phi,\ P=0+1$

$R+QS^*P$

$\phi+1(0+1)(\phi)^*(0+1)$

$1(0+1)(0+1)$



Now find the Regular expression 2:

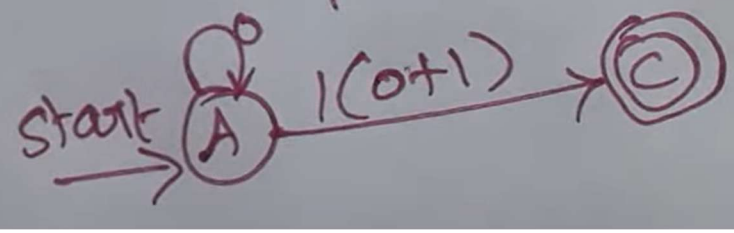
(ii) C ⇒ Final state, Eliminate D.

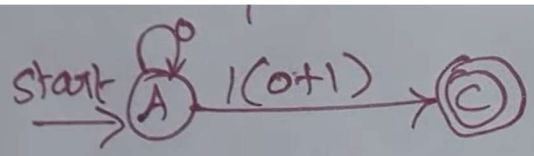
$R = \phi$, $Q = 0+1$, $S = \phi$, $P = \phi$

$R + QS^* P$

$\phi + (0+1)(\phi)^* \phi$

ϕ





$R = 0$, $S = 1(0+1)$, $U = \phi$, $T = \phi$

$RE_2 = (R + SU^* T)^* SU^*$

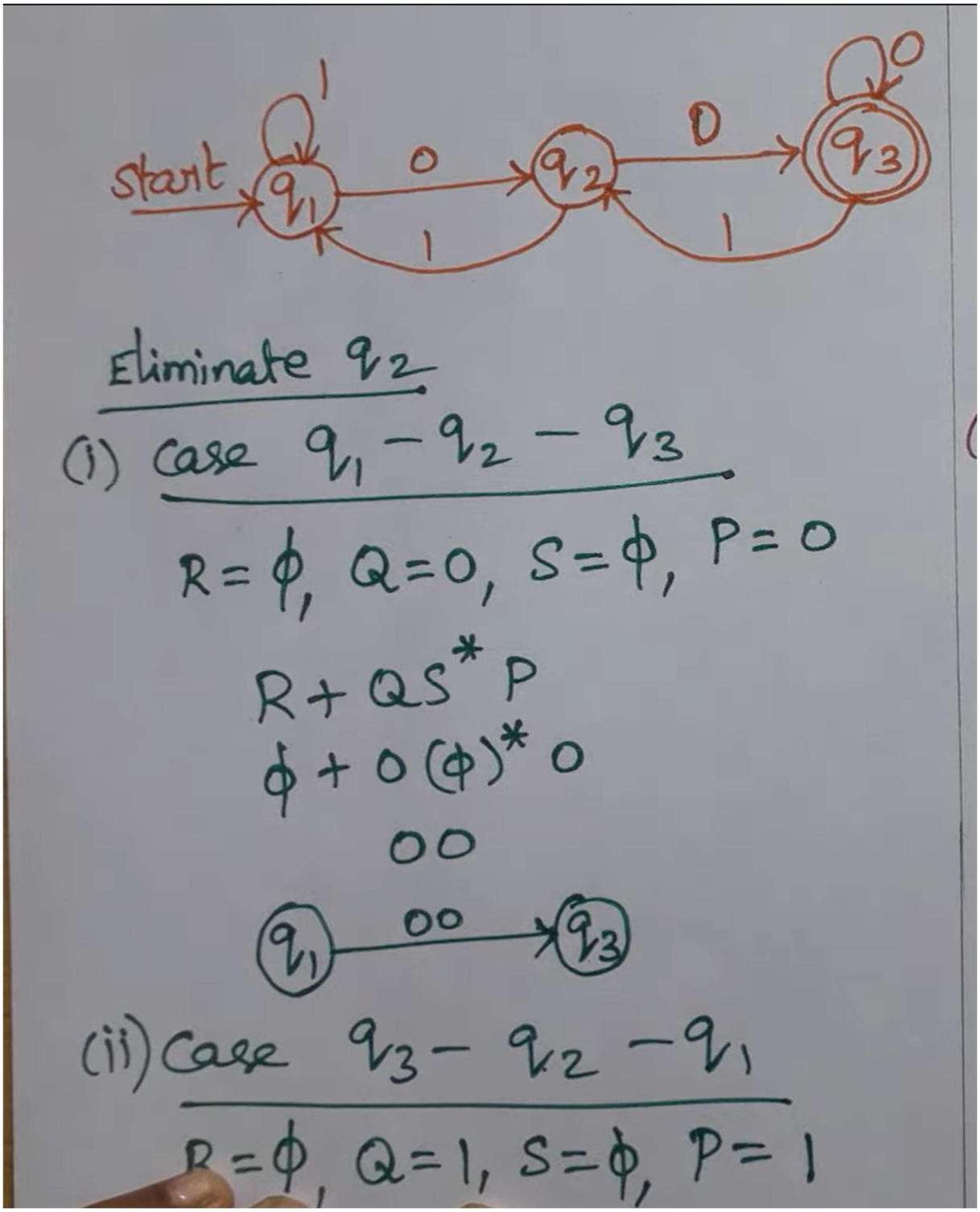
$= (0 + 1(0+1)(\phi)^* \phi)^* 1(0+1)(\phi)^*$

$RE_2 = 0^* 1(0+1)$

$RE = RE_1 + RE_2$

$RE = (0^* 1(0+1)(0+1)) + 0^* 1(0+1)$

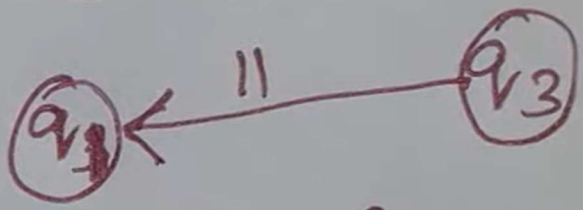
Example 2:


start
q_1
q_2
q_3
1
0
0
1
1
0
Eliminate q_2
(i) Case $q_1 - q_2 - q_3$
$R = \phi,\ Q = 0,\ S = \phi,\ P = 0$
$R + QS^* P$
$\phi + 0(\phi)^* 0$
00
q_1
00
q_3
(ii) Case $q_3 - q_2 - q_1$
$R = \phi,\ Q = 1,\ S = \phi,\ P = 1$

mination Method- Example2

$R + QS^* P$

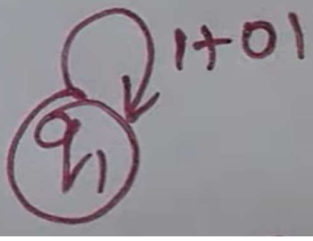
$\phi + 1(\phi)^* 1 = 11$



(iii) Path $q_1 - q_2 - q_1$

$R = 1, Q = 0, S = \phi, P = 1$

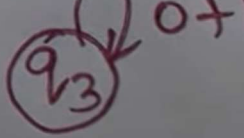
$1 + 0(\phi)^* 1 = 1 + 01$

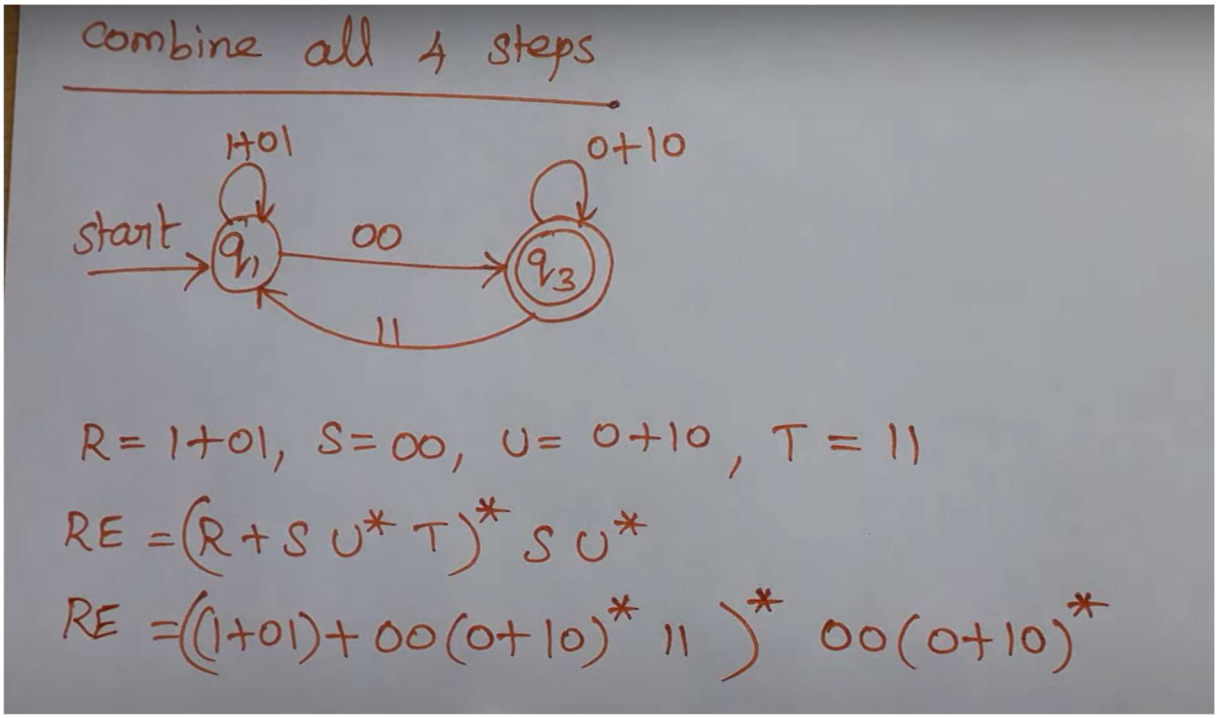


iv) Path $q_3 - q_2 - q_3$

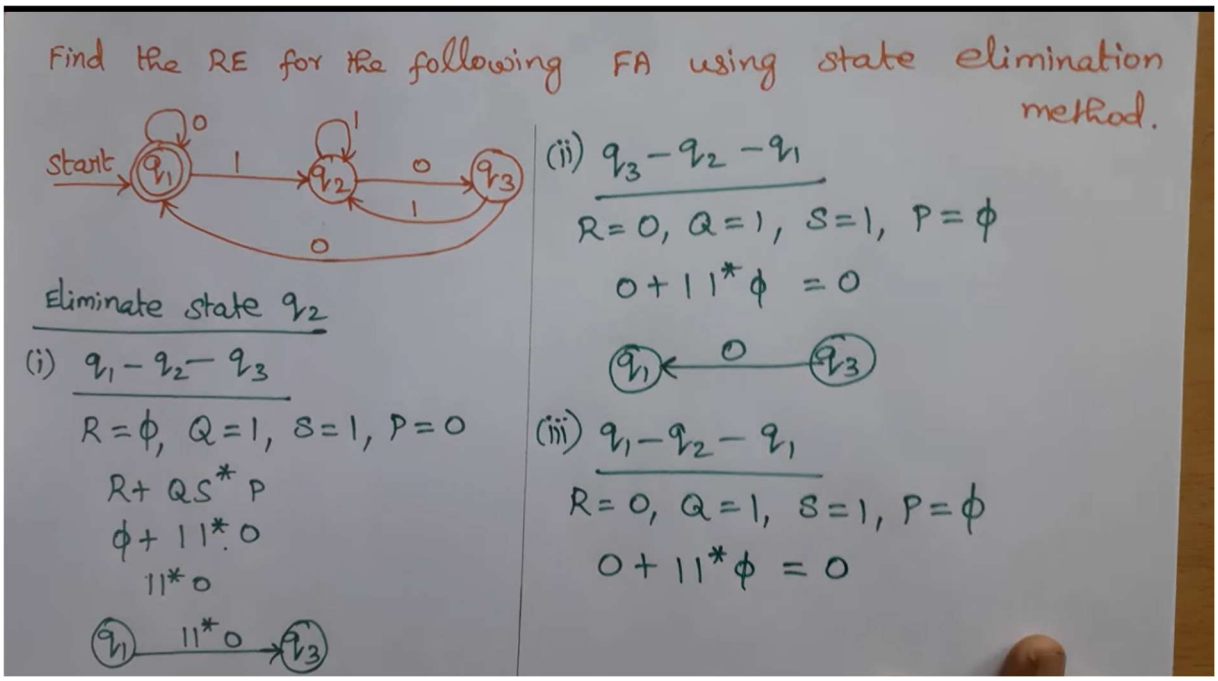
$R = 0, Q = 1, S = \phi, P = 0$

$0 + 1(\phi)^* 0 = 0 + 10$




Combine all 4 steps
1+01
0+10
start
00
11
R = 1+01, S = 00, U = 0+10, T = 11
RE = (R + S U* T)* S U*
RE = ((1+01) + 00(0+10)* 11)* 00(0+10)*

Example 3:

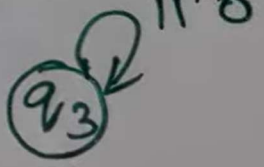

Find the RE for the following FA using state elimination method.
start
0
1
0
1
0
Eliminate state q2
(i) q1 − q2 − q3
R = ϕ, Q = 1, S = 1, P = 0
R + QS* P
ϕ + 11* 0
11* 0
11* 0
(ii) q3 − q2 − q1
R = 0, Q = 1, S = 1, P = ϕ
0 + 11* ϕ = 0
0
(iii) q1 − q2 − q1
R = 0, Q = 1, S = 1, P = ϕ
0 + 11* ϕ = 0

q_1

iv) $q_3 - q_2 - q_3$

$R = \phi,\ Q = 1,\ S = 1,\ P = 0$

$\phi + 11^*0 = 11^*$



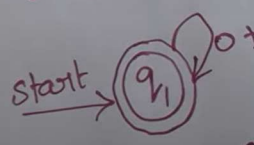
Eliminate q_3.

$q_1 - q_3 - q_1$

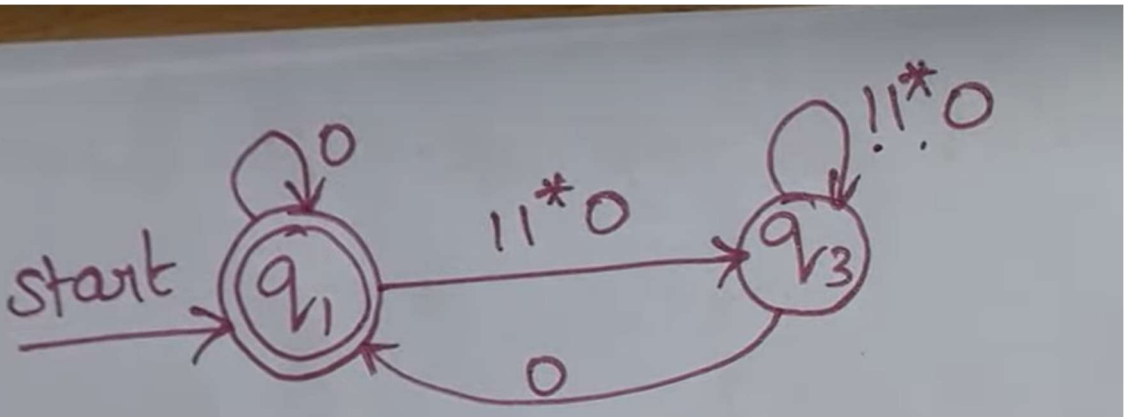
$R = 0,\ Q = 11^*0,\ S = 11^*0,\ P = 0$

$R + QS^*P$

$0 + (11^*0)(11^*0)^*0$



$RE = (0 + (11^*0)(11^*0)^*0)^*$



Eliminate q_3.

$q_1 - q_3 - q_1$

Identities of Regular Expression

1) $\emptyset + R = R$
2) $\emptyset R + R\emptyset = \emptyset$
3) $\epsilon R = R\epsilon = R$
4) $\epsilon^* = \epsilon$ and $\emptyset^* = \epsilon$
5) $R + R = R$
6) $R^*R^* = R^*$
7) $RR^* = R^*R$
8) $(R^*)^* = R^*$
9) $\epsilon + RR^* = \epsilon + R^*R = R^*$
10) $(PQ)^*P = P(QP)^*$
11) $(P + Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$
12) $(P + Q)R = PR + QR$ and

$R(P + Q) = RP + QR$

ARDEN'S THEOREM

If P and Q are two Regular Expressions over Σ, and if P does not contain ϵ, then the following equation in R given by $R = Q + RP$ has a unique solution i.e. $R = QP^*$

$R = Q + RP \longrightarrow (1)$

$= Q + QP^*P$

$= Q(\epsilon + P^*P)$

$= QP^*$ Proved

$R = QP^*$

$[\epsilon + R^*R = R^*]$

$R = Q + RP$

$= Q + [Q + RP]P$

$= Q + QP + RP^2$

$= Q + QP + [Q + RP]P^2$

$= Q + QP + QP^2 + RP^3$

$\vdots$

$= Q + QP + QP^2 + \ldots QP^n + RP^{n+1}$

$[R = QP^*]$

$= Q + QP + QP^2 + \ldots QP^n + QP^*P^{n+1}$

$= Q[\epsilon + P + P^2 + \ldots P^n + P^*P^{n+1}]$

$R = QP^*$

An Example Proof using Identities of Regular Expressions

Prove that (1+00*1) + (1+00*1) (0+10*1)* (0+10*1) is equal to 0*1 (0+10*1)*

$$LHS = (1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)$$

$$= (1+00^*1)\left[\epsilon + (0+10^*1)^*(0+10^*1)\right] \qquad \epsilon + R^*R = R^*$$

$$= (1+00^*1)(0+10^*1)^* \qquad \epsilon \cdot R = R$$

$$= (\epsilon \cdot 1+00^*1)(0+10^*1)^*$$

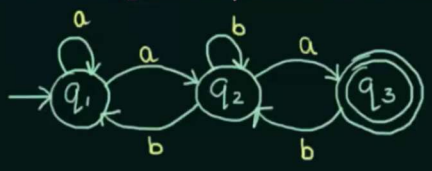
$$= (\epsilon+00^*)1(0+10^*1)^*$$

$$= 0^*1(0+10^*1)^* = RHS$$

From Automat to Regular Expression

Designing Regular Expression - Examples (Part-2)

Find the Regular Expression for the following NFA



$$q_3 = q_2a \longrightarrow (1)$$

$$q_2 = q_1a + q_2b + q_3b \longrightarrow (2)$$

$$q_1 = \epsilon + q_1a + q_2b \longrightarrow (3)$$

(1) $\Rightarrow q_3 = q_2a$

$$= (q_1a + q_2b + q_3b)a$$

$$= q_1aa + q_2ba + q_3ba \longrightarrow (4)$$

(2) $\Rightarrow q_2 = q_1a + q_2b + q_3b$ Putting value of q_3 from (1)

$$= q_1a + q_2b + (q_2a)b$$

$$= q_1a + q_2b + q_2ab$$

$$\underset{R}{q_2} = \underset{Q}{q_1a} + \underset{R}{q_2}\underset{P}{(b+ab)}$$

$R = Q + RP$ Arden's Theorem

$R = QP^*$

$$q_2 = (q_1a)(b+ab)^* \longrightarrow (5)$$

③ → $q_1 = \epsilon + q_1a + q_2b$

Putting value of q_2 from ⑤

$q_1 = \epsilon + q_1a + ((q_1a)(b+ab)^*)b$

$R = Q + RP$

$R = QP^*$

$q_1 = \epsilon + q_1(a + a(b+ab)^*)b$

$R \quad Q \quad R \quad P$

$\epsilon \cdot R = R$

$q_1 = \epsilon((a+a(b+ab)^*)b)^*$

$q_1 = (a+a(b+ab)^*b)^*$ → ⑥

Final state q_3

$q_3 = q_2a$

$= q_1a(b+ab)^*a$ Putting value of q_2 from ⑤

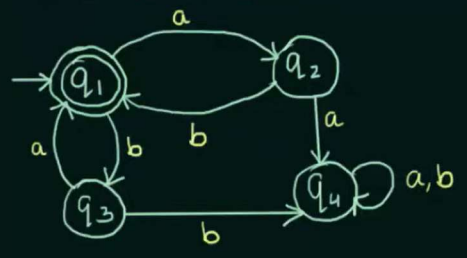
$q_3 = (a+a(b+ab)^*b)^*\ a(b+ab)^*a$ Putting value of q_1 from ⑥

= Required Regular Expression for the given NFA

Example:

Designing Regular Expression - Examples (Part-3)

Find the Regular Expression for the following DFA



$q_1 = \epsilon + q_2b + q_3a$ → (I)

$q_2 = q_1a$ → (II)

$q_3 = q_1b$ → (III)

$q_4 = q_2a + q_3b + q_4a + q_4b$ → (IV)

(I) $\rightarrow$ $q_1 = \epsilon + q_2 b + q_3 a$

Putting values of q_2 and q_3 from (II) and (III)

$q_1 = \epsilon + q_1 ab + q_1 ba$

$q_1 = \epsilon + q_1(ab + ba)$

R Q R P

$R = Q + RP$

$R = QP^*$ Arden's Theorem

$q_1 = \epsilon . (ab + ba)^*$

$\epsilon R = R$

$q_1 = (ab + ba)^*$

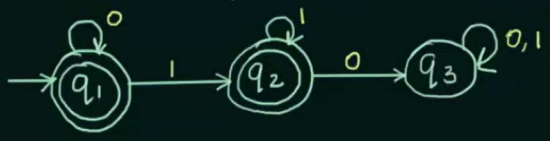
$\hookrightarrow$ Regular Expression .

Example:

Designing Regular Expression - Examples (Part-4)

(When there are Multiple Final States)

Find the Regular Expression for the following DFA



$q_1 = \epsilon + q_1 0 \rightarrow$ (I)

$q_2 = q_1 1 + q_2 1 \rightarrow$ (II)

$q_3 = q_2 0 + q_3 0 + q_3 1 \rightarrow$ (III)

Final State q_1

(I) $\rightarrow$ $q_1 = \epsilon + q_1 0$

R Q R P

$R = Q + RP$

$R = QP^*$ Arden's Theorem

$q_1 = \epsilon . 0^*$

$\epsilon R = R$

$q_1 = 0^* \rightarrow$ (4)

Final State q_2

(II) $\rightarrow$ $q_2 = q_1 1 + q_2 1$

$q_2 = 0^* 1 + q_2 1$ Putting value of q_1 from (4)

R Q R P

$R = Q + RP$

$R = QP^*$

$q_2 = 0^* 1 (1)^*$

R = union of both Final States

$= 0^* + 0^*11^*$

$= 0^*(\epsilon + 11^*)$ $\quad \epsilon + RR^* = R^*$

$= 0^*1^*$

↳ Regular Expression

Conversion of Regular Expression to Finite Automata

(a+b)



(a.b)



a*

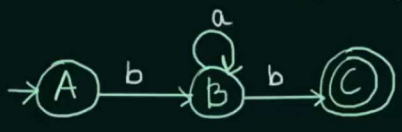


Conversion of Regular Expression to Finite Automata - Examples (Part-1)

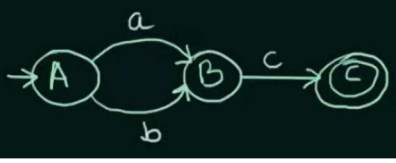
Convert the following Regular Expressions to their equivalent Finite Automata:

1) b a* b
2) (a+b) c
3) a (bc)*

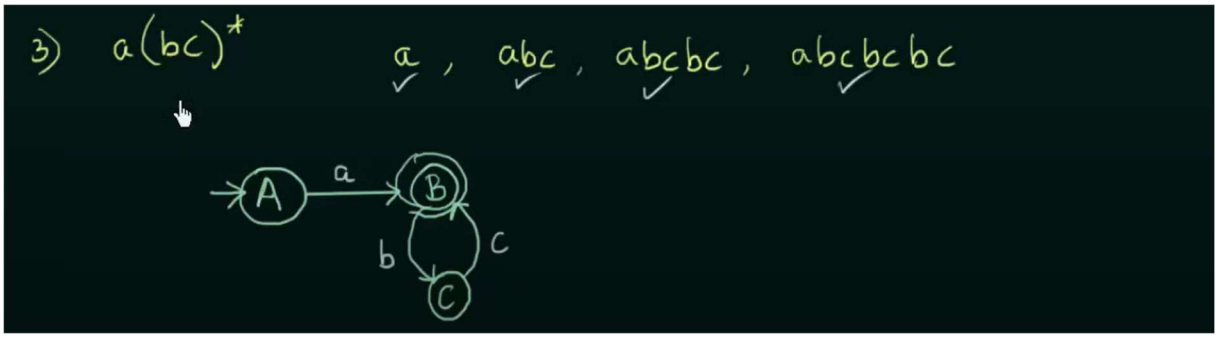
1) ba^*b $\quad bb, bab, baab, \ldots$



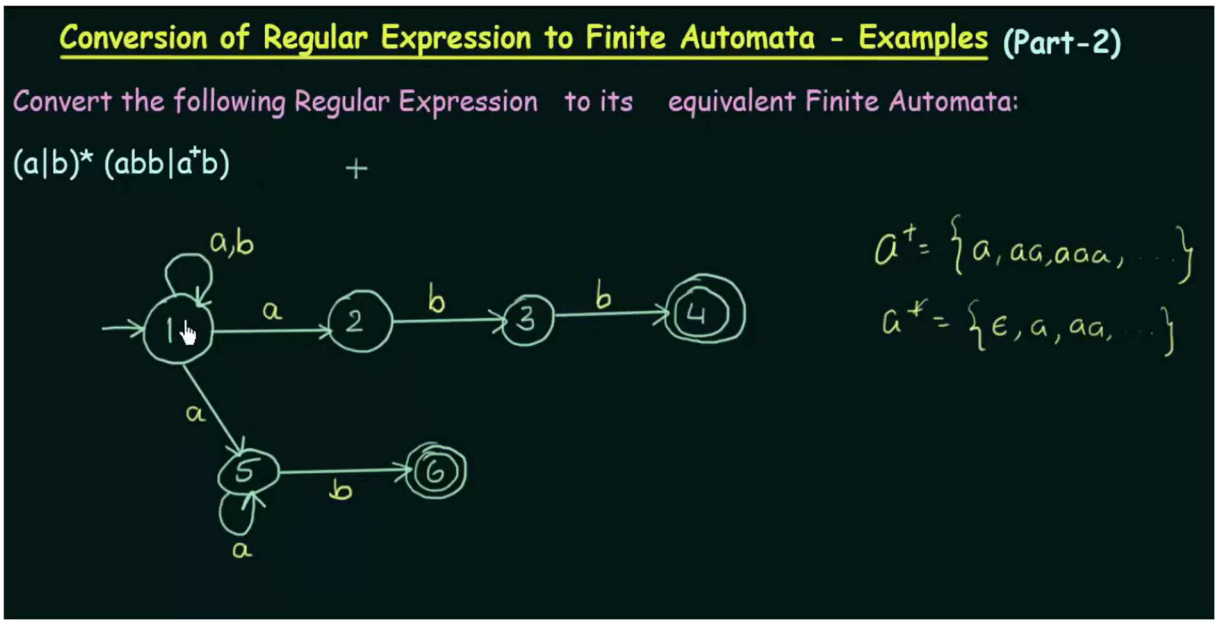
2) $(a+b)c$



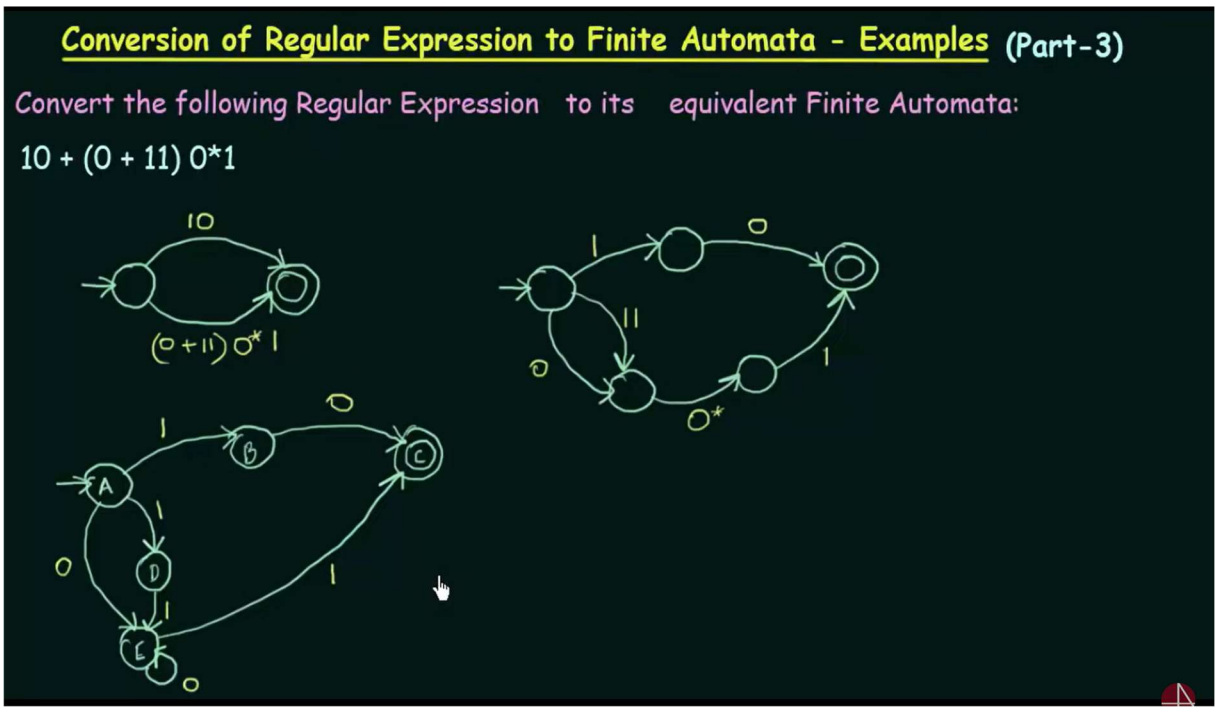
ac ✓
bc ✓



Example:



Example:



Equivalence of two finite Automaton:

Equivalence of two Finite Automata

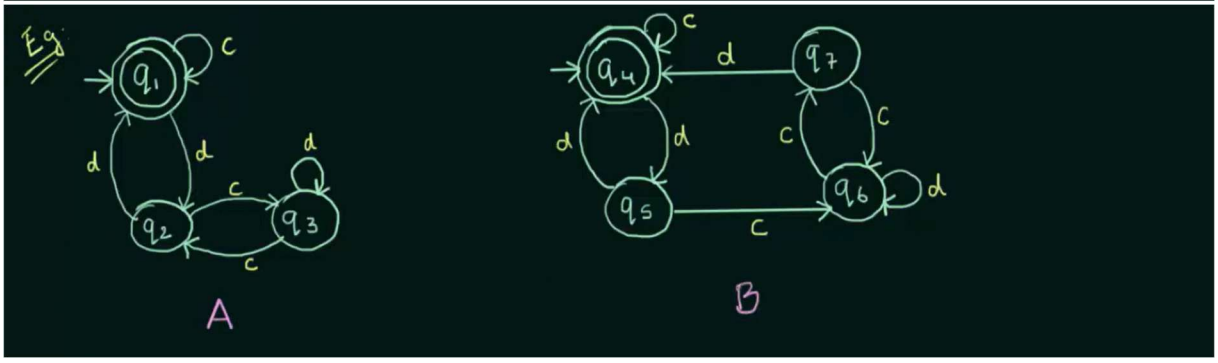
Steps to identify equivalence

1) For any pair of states $\{q_i , q_j\}$ the transition for input $a \in \Sigma$ is defined by $\{q_a , q_b\}$ where $\delta\{q_i , a\} = q_a$ and $\delta\{q_j , a\} = q_b$
The two automata are not equivalent if for a pair $\{q_a , q_b\}$ one is INTERMEDIATE State and the other is FINAL State.

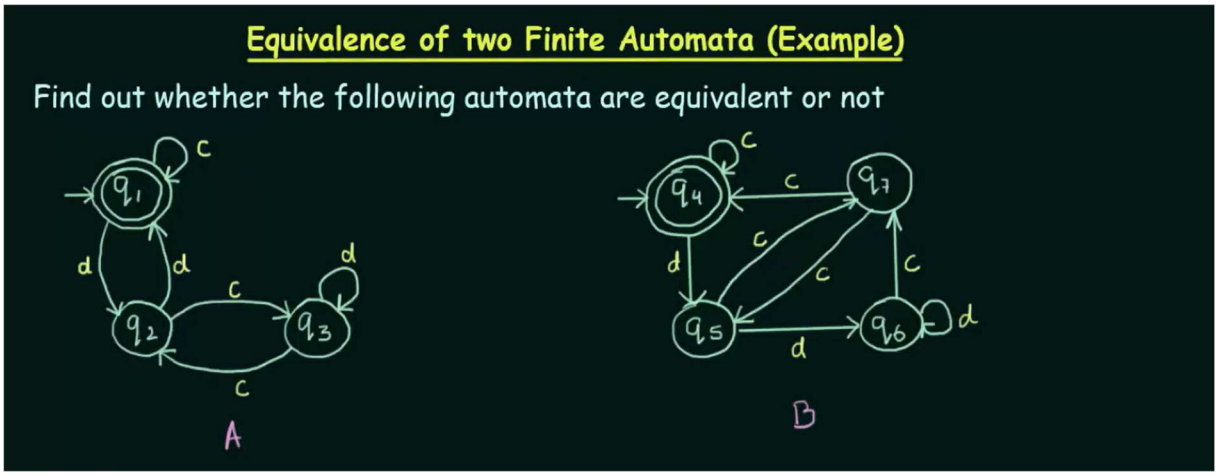
2) If Initial State is Final State of one automaton, then in second automaton also Initial State must be Final State for them to be equivalent.

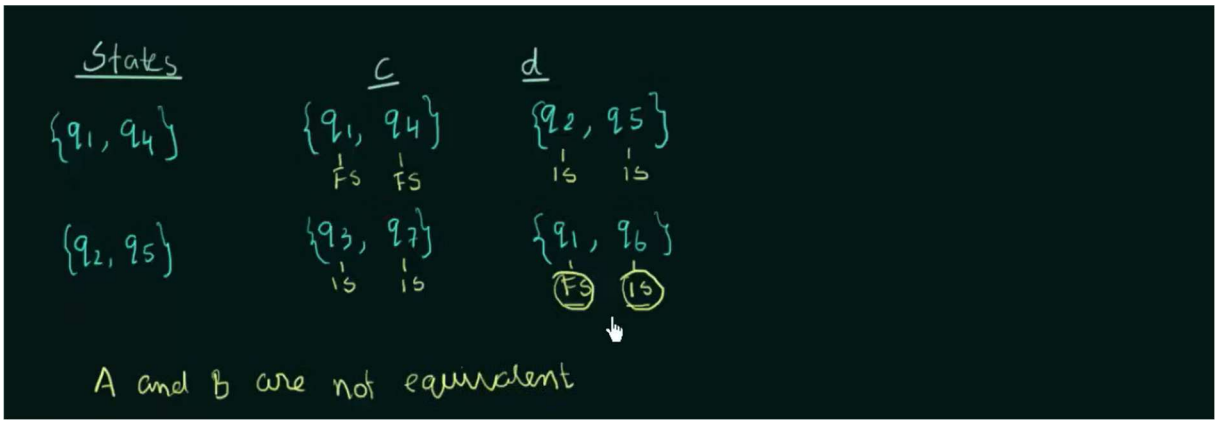
States	c	d
(q_1, q_4)	(q_1, q_4) FS FS	(q_2, q_5) IS IS
(q_2, q_5)	(q_3, q_6) IS IS	(q_1, q_4) FS FS
(q_3, q_6)	(q_2, q_7) IS IS	(q_3, q_6) IS IS
(q_2, q_7)	(q_3, q_6) IS IS	(q_1, q_4) FS FS

A and B are equivalent

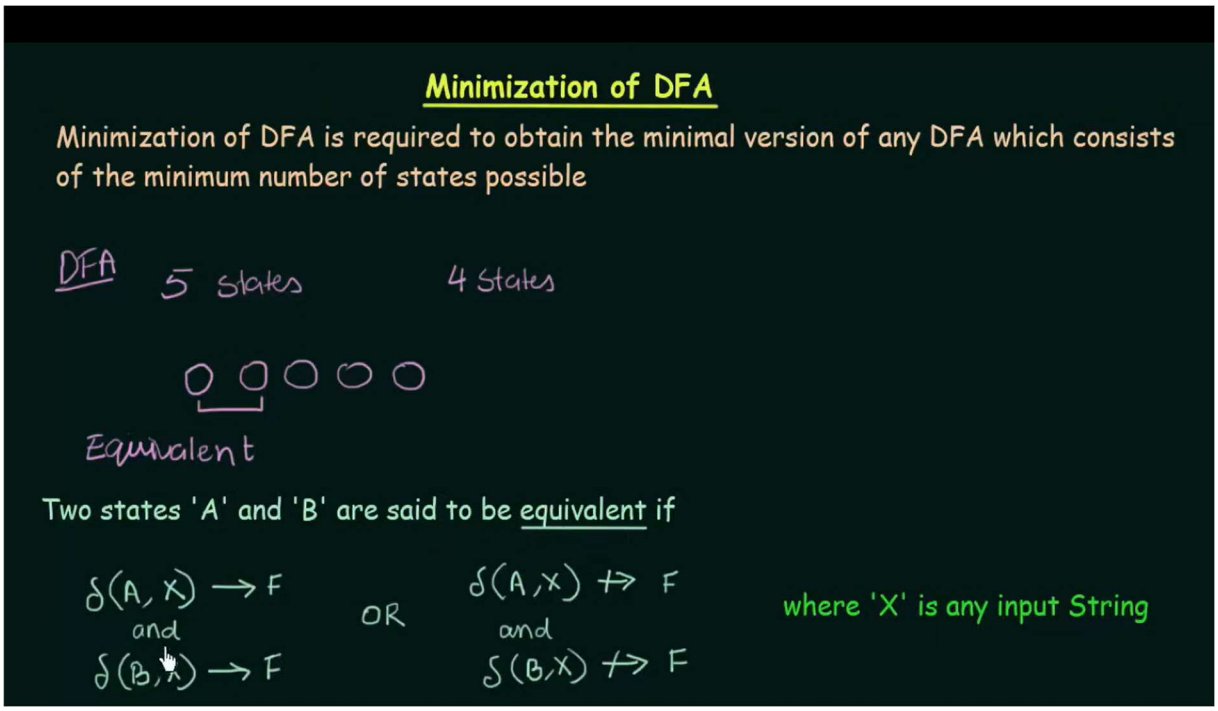


Another Example:



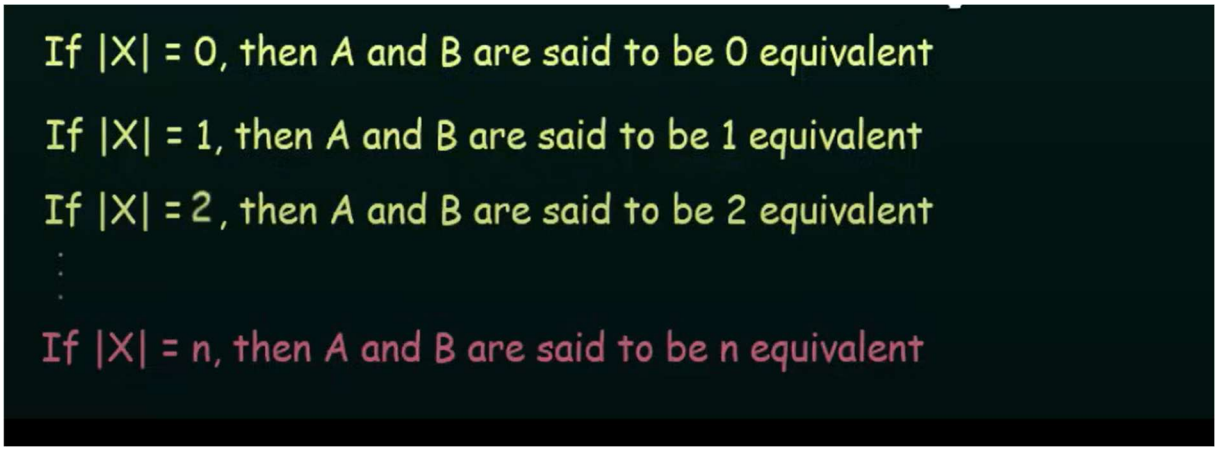


Minimization of the DFA

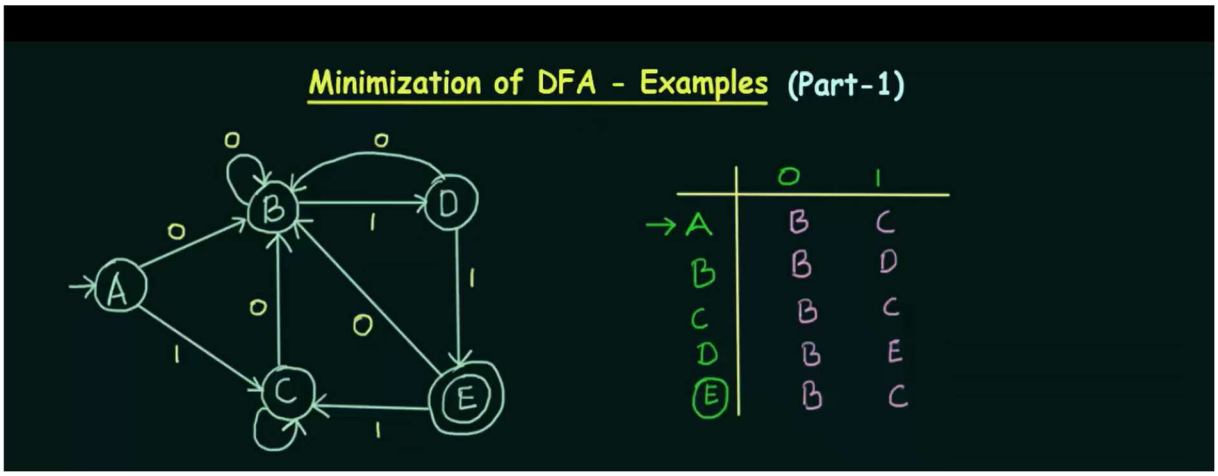


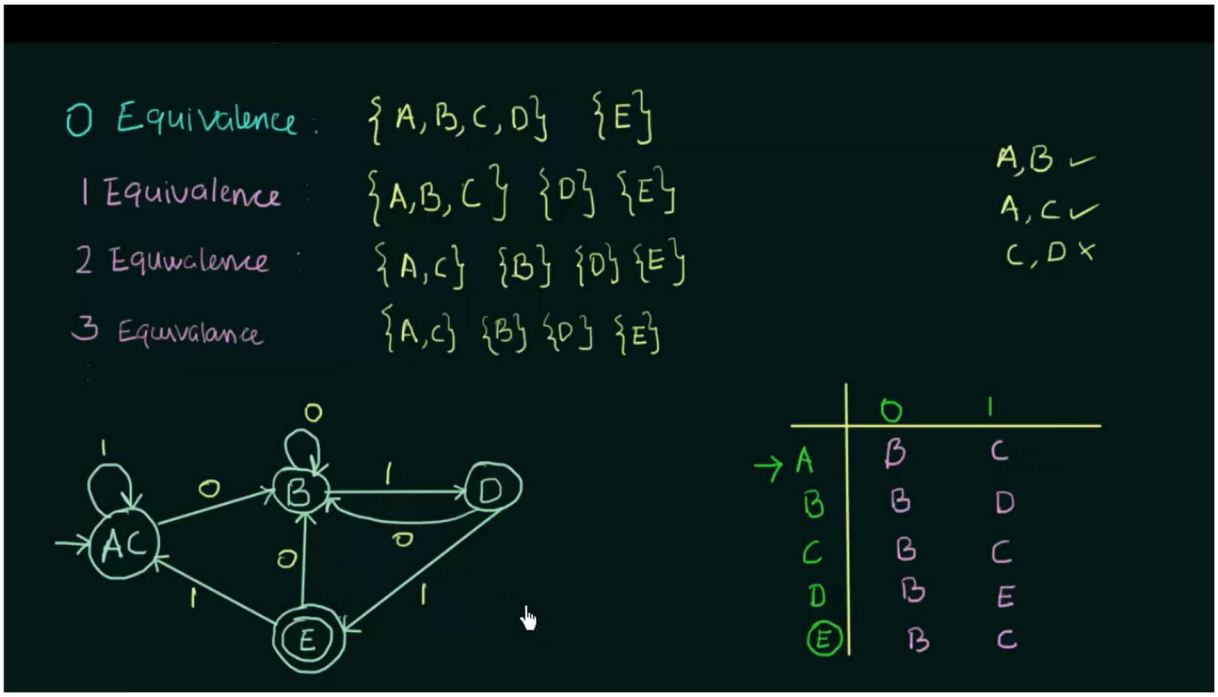
If the state A on getting input x does go to a final state , and the state B on getting input x does go to a final state ➔ A, and B are equivalent

A and B are equivalent, if the state A on getting input x does not go to a Final state and B on getting input x does not go to a Final state ➔ A and B are equivalent.

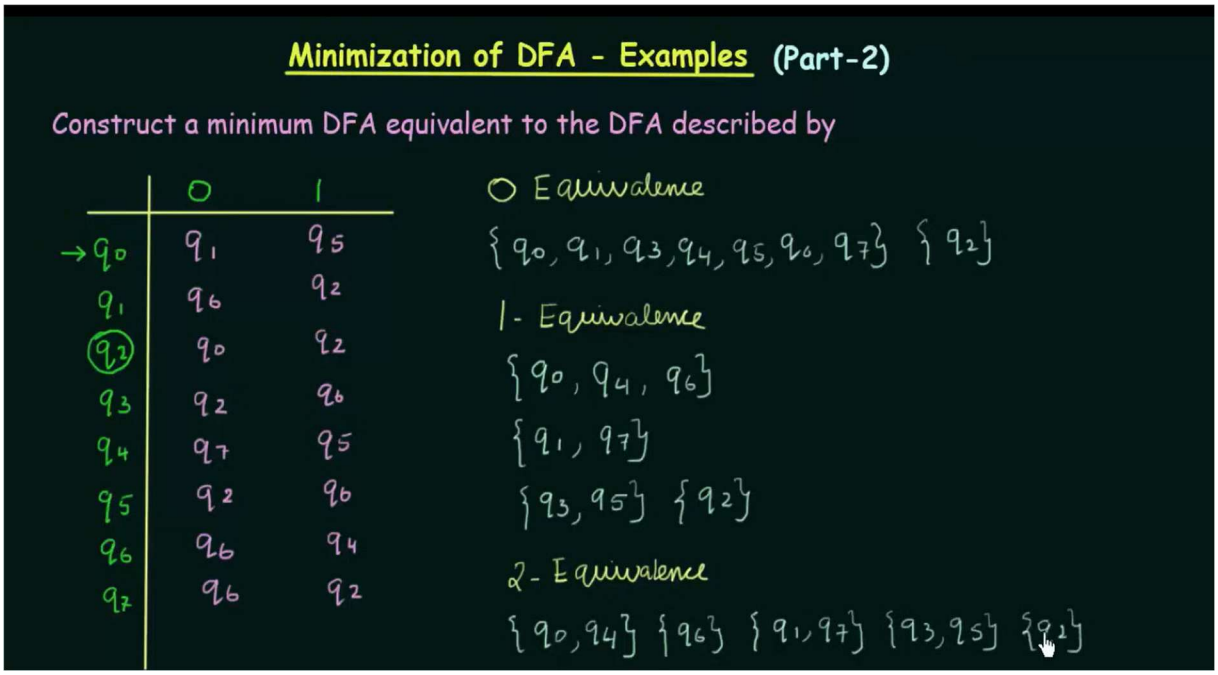


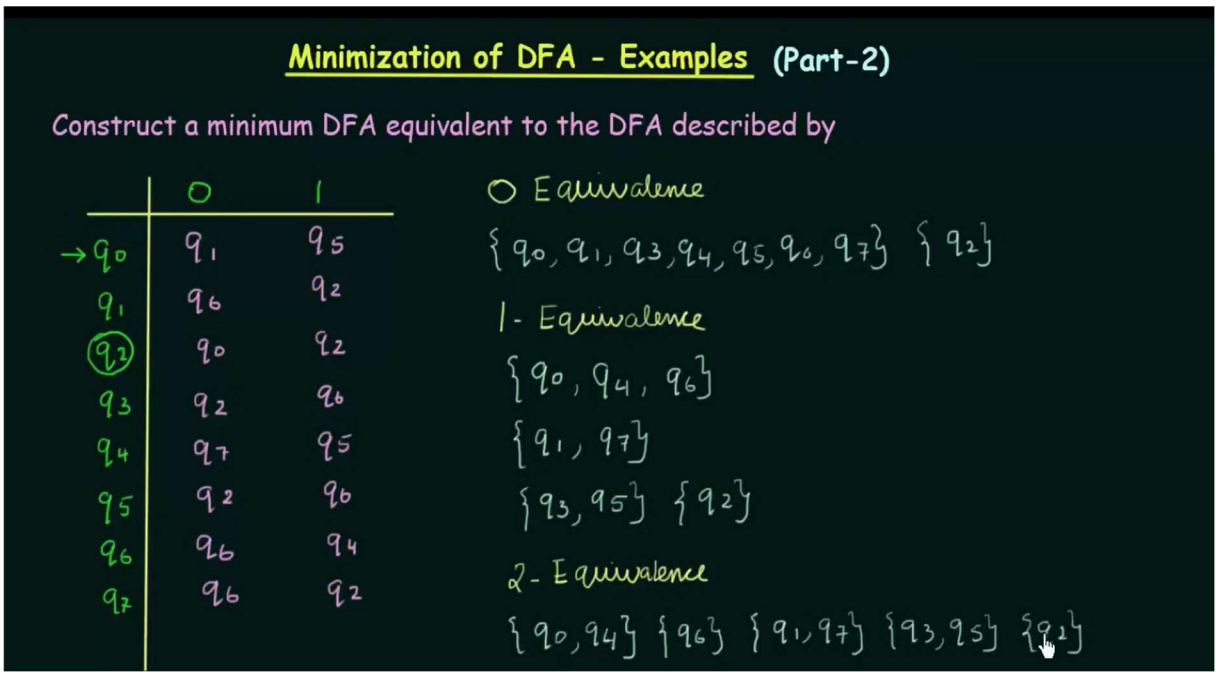
Example of Minimization:



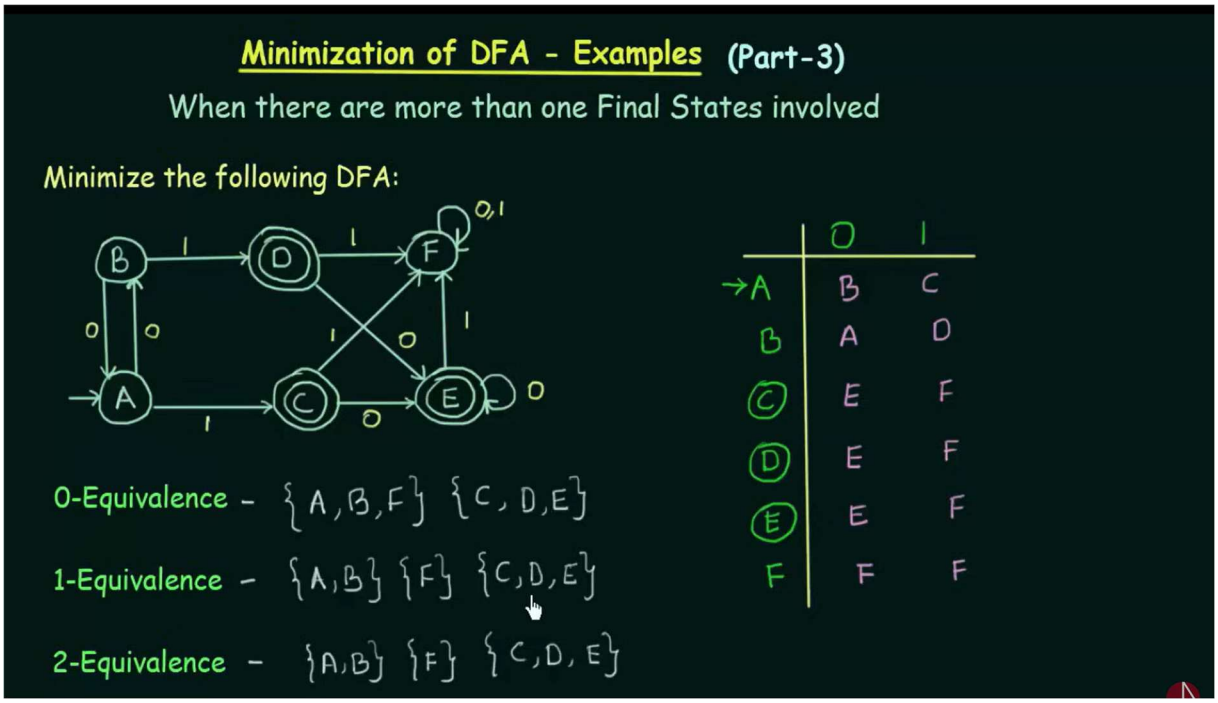


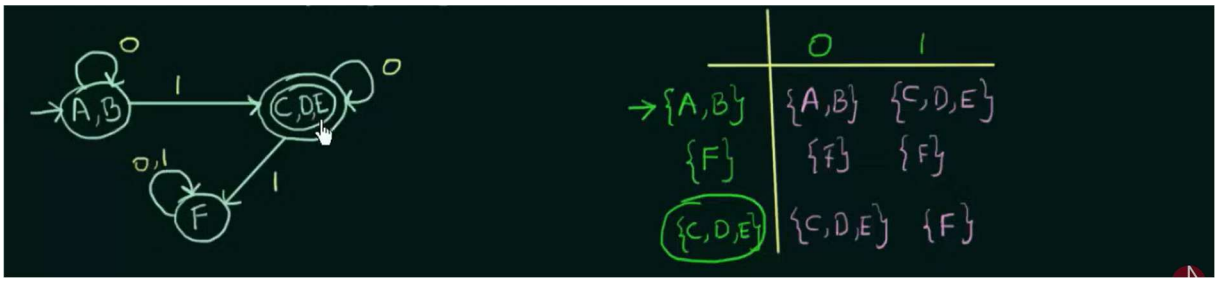
Another Example:


Minimization of DFA - Examples (Part-2)
Construct a minimum DFA equivalent to the DFA described by
0 1
→q0 q1 q5
q1 q6 q2
(q2) q0 q2
q3 q2 q6
q4 q7 q5
q5 q2 q6
q6 q6 q4
q7 q6 q2
0 Equivalence
{q0, q1, q3, q4, q5, q6, q7} {q2}
1- Equivalence
{q0, q4, q6}
{q1, q7}
{q3, q5} {q2}
2- Equivalence
{q0, q4} {q6} {q1, q7} {q3, q5} {q2}

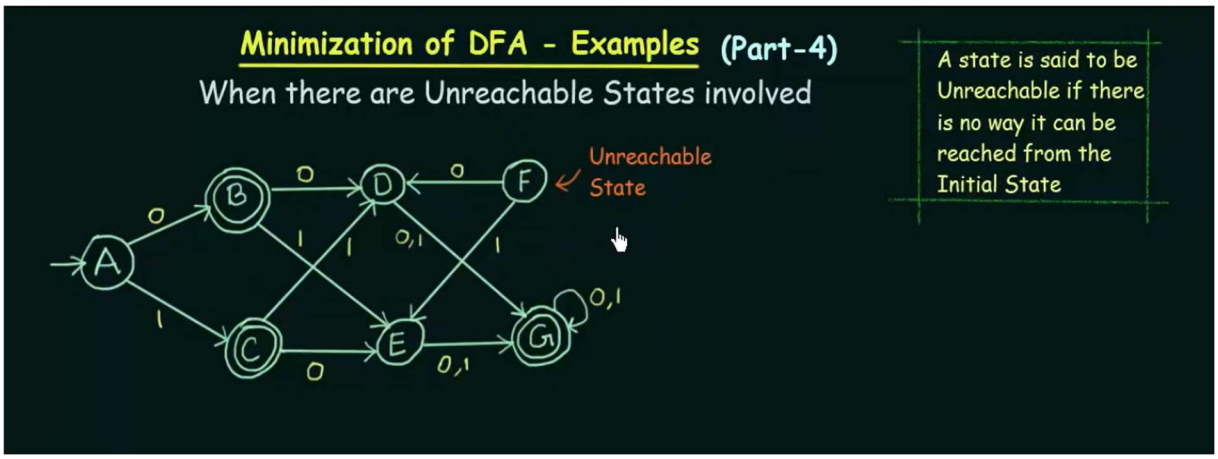

Minimization of DFA - Examples (Part-2)
Construct a minimum DFA equivalent to the DFA described by
0 1
→q0 q1 q5
q1 q6 q2
(q2) q0 q2
q3 q2 q6
q4 q7 q5
q5 q2 q6
q6 q6 q4
q7 q6 q2
0 Equivalence
{q0, q1, q3, q4, q5, q6, q7} {q2}
1- Equivalence
{q0, q4, q6}
{q1, q7}
{q3, q5} {q2}
2- Equivalence
{q0, q4} {q6} {q1, q7} {q3, q5} {q2}

Example 3:



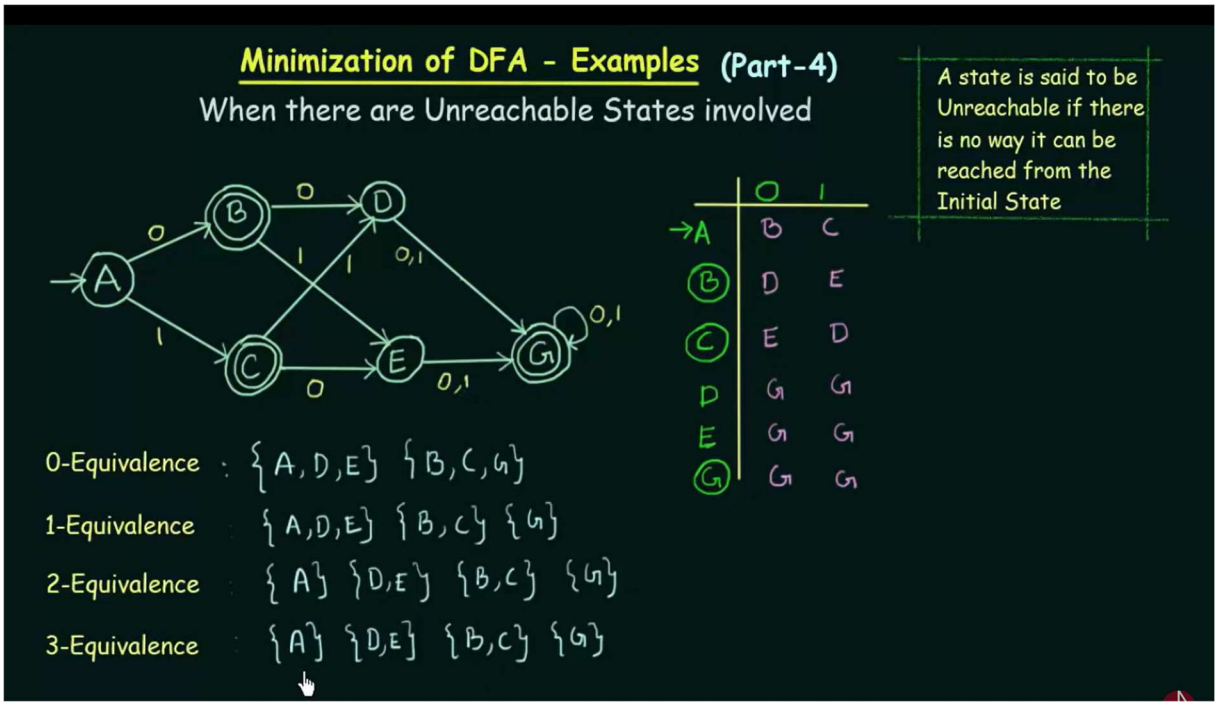


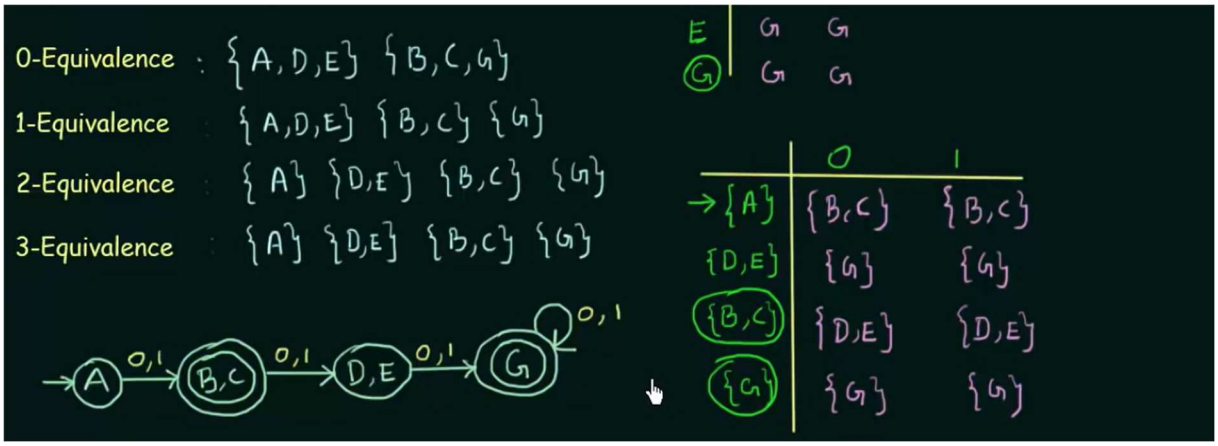
Example 4 with unreachable state:



If you have an unreachable state, first remove the unreachable state and proceed as you normally do with the 0 Equivalence 1 Equivalence etc.

You can directly see that the unreachable state, that only have transitions that are going out of the state and no transitions that are coming into the state

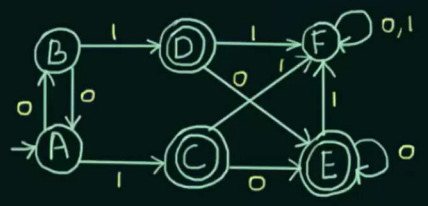




Note this method to minimize the DFA is known as the portioning method. There is also another method for converting (Table Filling method or known as Myhill- Nerode-Theorem).

Minimization of DFA - Table Filling Method

(Myhill-Nerode Theorem)



	A	B	C	D	E	F
A						
B						
C						
D						
E						
F						

Steps:

1) Draw a table for all pairs of states (P,Q)
2) Mark all pairs where $P \in F$ and $Q \notin F$
3) If there are any Unmarked pairs (P,Q) such that $[\delta(P,x), \delta(Q,x)]$ is marked, then mark [P,Q] where 'x' is an input symbol
   REPEAT THIS UNTIL NO MORE MARKINGS CAN BE MADE
4) Combine all the Unmarked Pairs and make them a single state in the minimized DFA

---

then mark [P,Q] where 'x' is an input symbol
REPEAT THIS UNTIL NO MORE MARKINGS CAN BE MADE
4) Combine all the Unmarked Pairs and make them a single state in the minimized DFA

	A	B	C	D	E	F
A						
B						
C	✓	✓				
D	✓	✓				
E	✓	✓				
F	✓	✓	✓	✓	✓	

(D,C) - $\delta(D,0) = E$, $\delta(D,1) = F$; $\delta(C,0) = E$, $\delta(C,1) = F$

(E,C) - $\delta(E,0) = E$, $\delta(E,1) = F$; $\delta(C,0) = E$, $\delta(C,1) = F$

(E,D) - $\delta(E,0) = E$, $\delta(E,1) = F$; $\delta(D,0) = E$, $\delta(D,1) = F$

(F,A) - $\delta(F,0) = F$, $\delta(F,1) = F$; $\delta(A,0) = B$, $\delta(A,1) = C$

(F,B) = $\delta(F,0) = F$; $\delta(B,0) = A$

(B,A) - $\delta(B,0) = A$, $\delta(B,1) = D$; $\delta(A,0) = B$, $\delta(A,1) = C$

(A,B) (D,C) (E,C) (E,D)

REPEAT THIS UNTIL NO MORE MARKINGS CAN BE MADE
4) Combine all the Unmarked Pairs and make them a single state in the minimized DFA

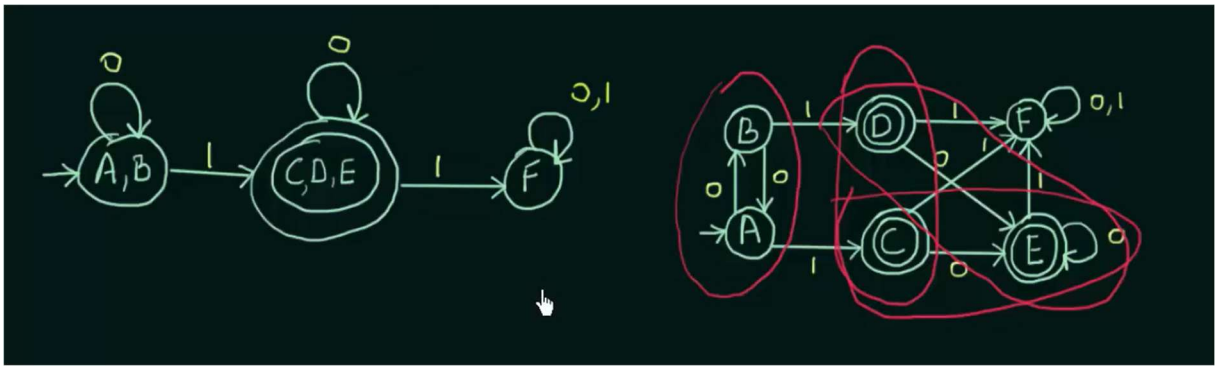
	A	B	C	D	E	F
A						
B						
C	✓	✓				
D	✓	✓				
E	✓	✓				
F			✓	✓	✓	

(D,C) - $\delta(D,0) = E$, $\delta(D,1) = F$; $\delta(C,0) = E$, $\delta(C,1) = F$

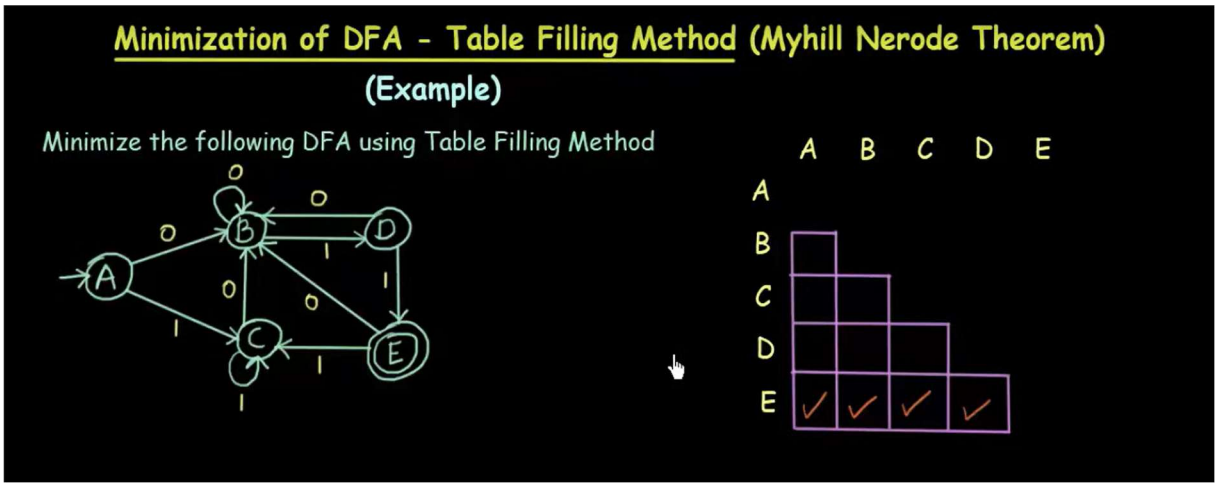
(E,C) - $\delta(E,0) = E$, $\delta(E,1) = F$; $\delta(C,0) = E$, $\delta(C,1) = F$

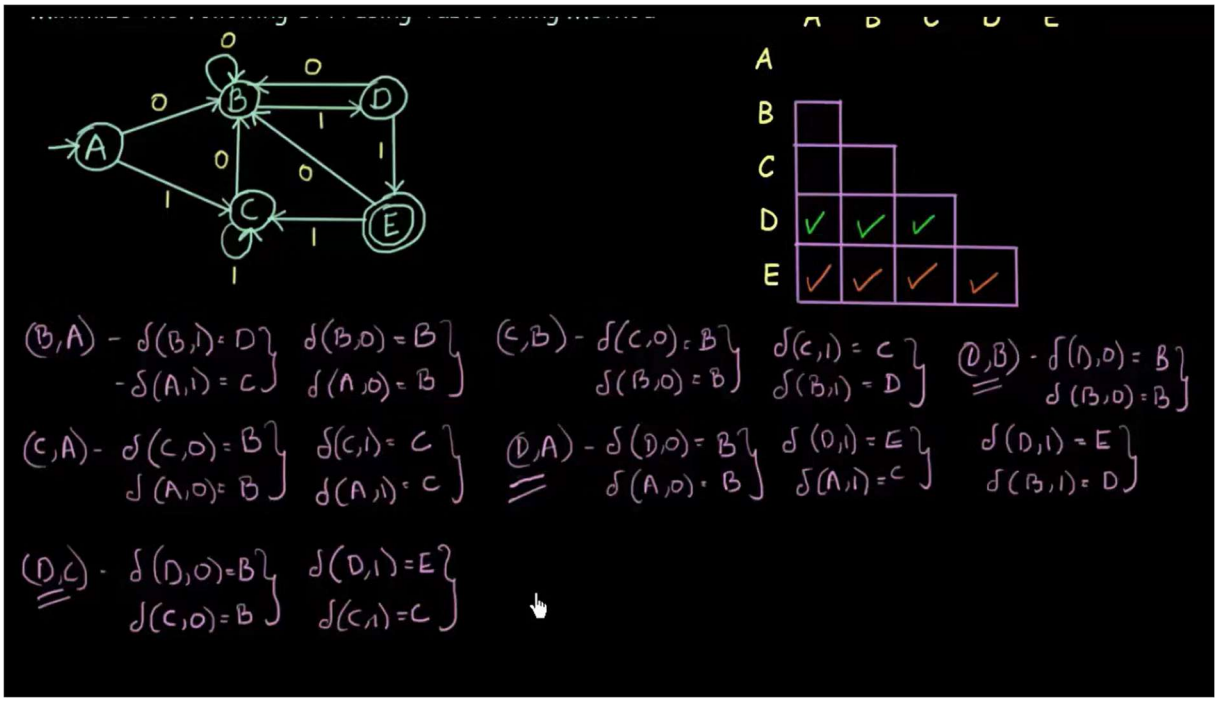
(E,D) - $\delta(E,0) = E$, $\delta(E,1) = F$; $\delta(D,0) = E$, $\delta(D,1) = F$

(B,A) - $\delta(B,0) = A$, $\delta(B,1) = D$; $\delta(A,0) = B$, $\delta(A,1) = C$



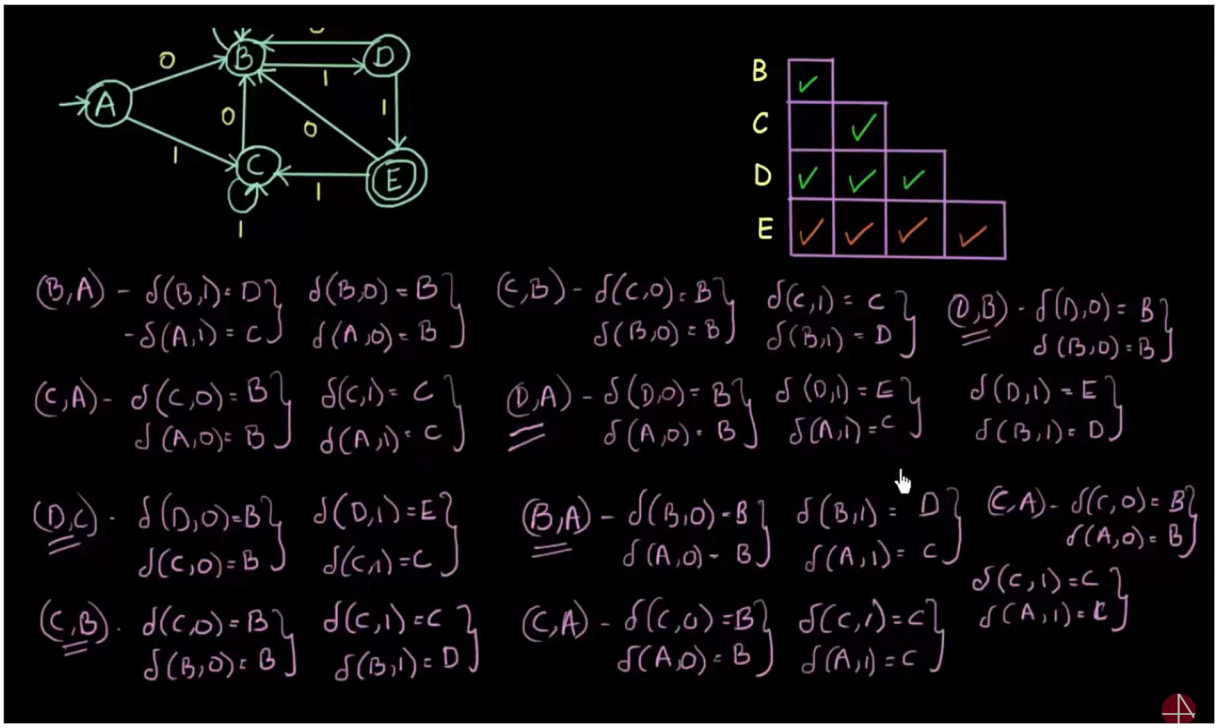
Example 2:





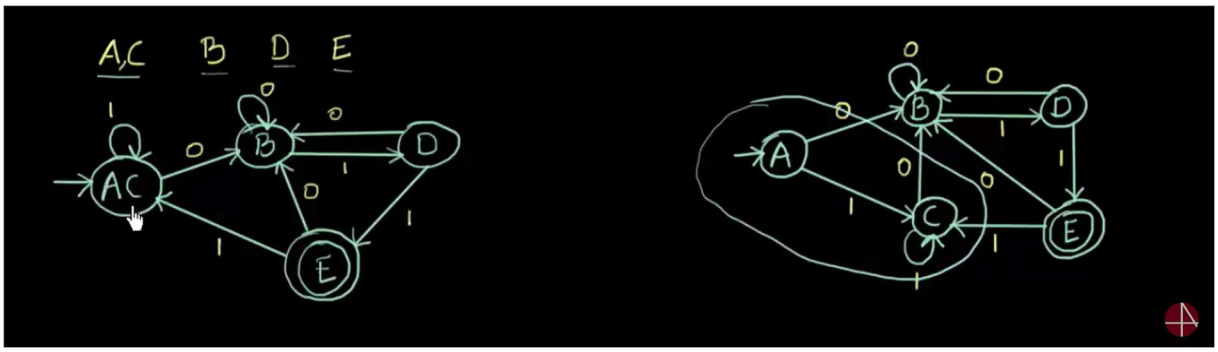
Now we have completed the 1st iteration of the Algorithm, we should repeat the same procedure until no more marking for the unmarked pairs can be done

Check again for the unmarked pairs (A, B) , (A,C) , (B,C)



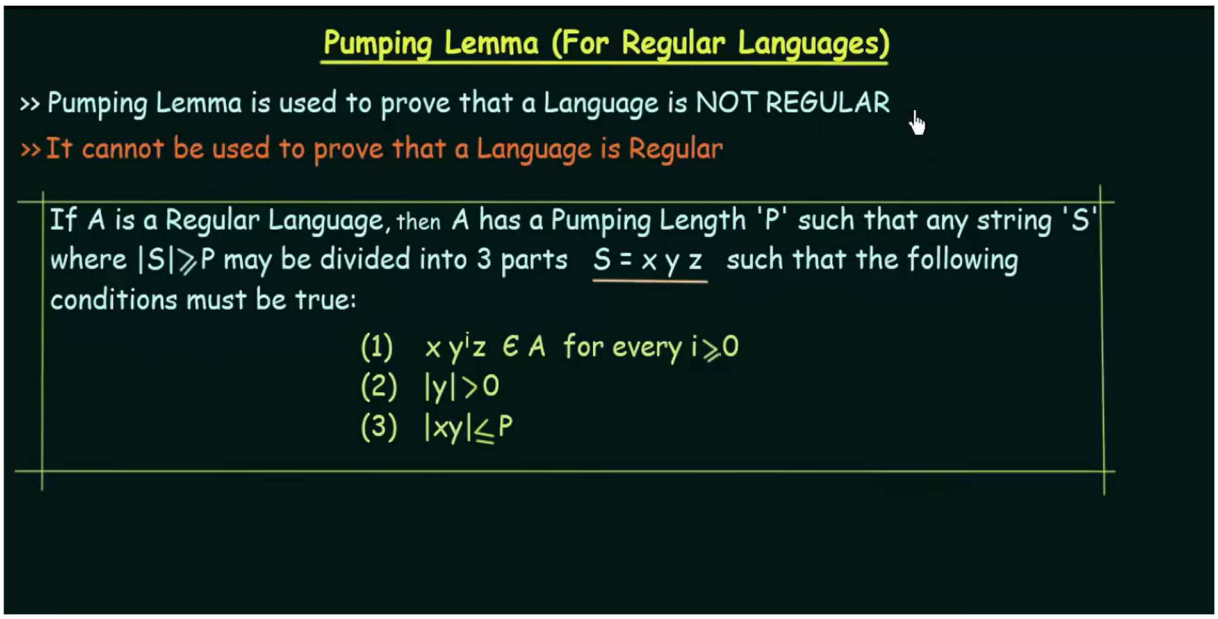
This is the result of all pairs that we can mark, the only pair that you can't mark is (C,A)

Because you have only one unmarked pair, which is (A,C) or (C,A) then in the minimized DFA, you should combine both states A, C together.



This is the final result.

Pumping Lemma for Regular Languages

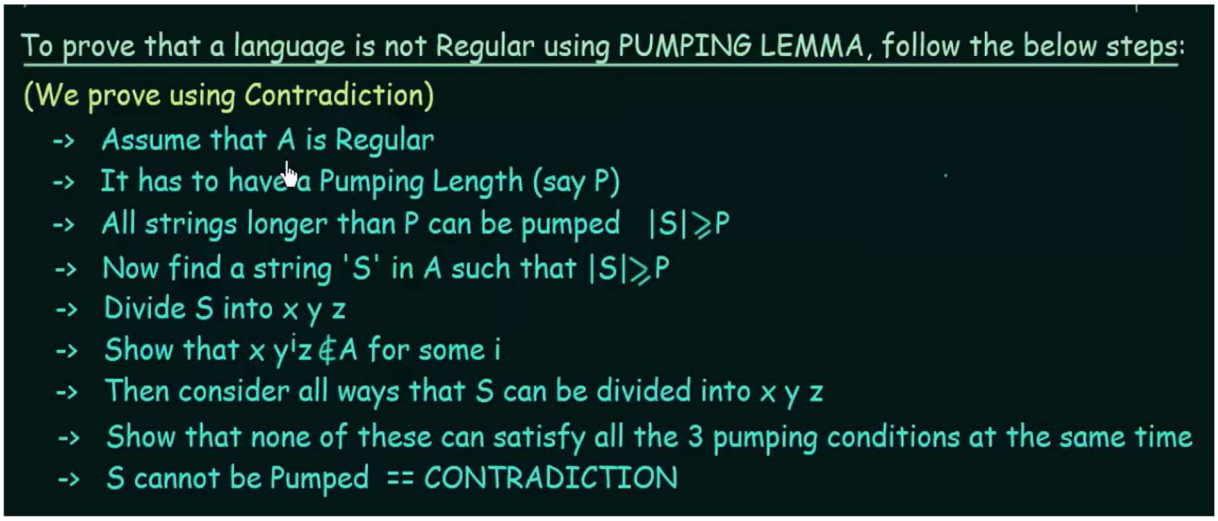


A regular Language is a language that we can design using a finite state machine, we know that the properties of finite state machine and we know there limitations and due to their limitations, there are some languages that can't be designed using finite state machine ➔ those languages are known as non-regular languages.

For proving that a language isn't regular ➔ we use the pumping lemma.

If A is a regular language , then A must have a pumping length P, any String S, where |S| >= P , S can be divided into 3 Parts S = x y z , we divide the string S into 3 Pats x ,y and z, the conditions are:

1) If you increase the middle part i number of times, then the string that you get should be in the regular language A
2) The length of y , |y| > 0
3) The length of xy <= to the pumping length P, |xy| <= P.

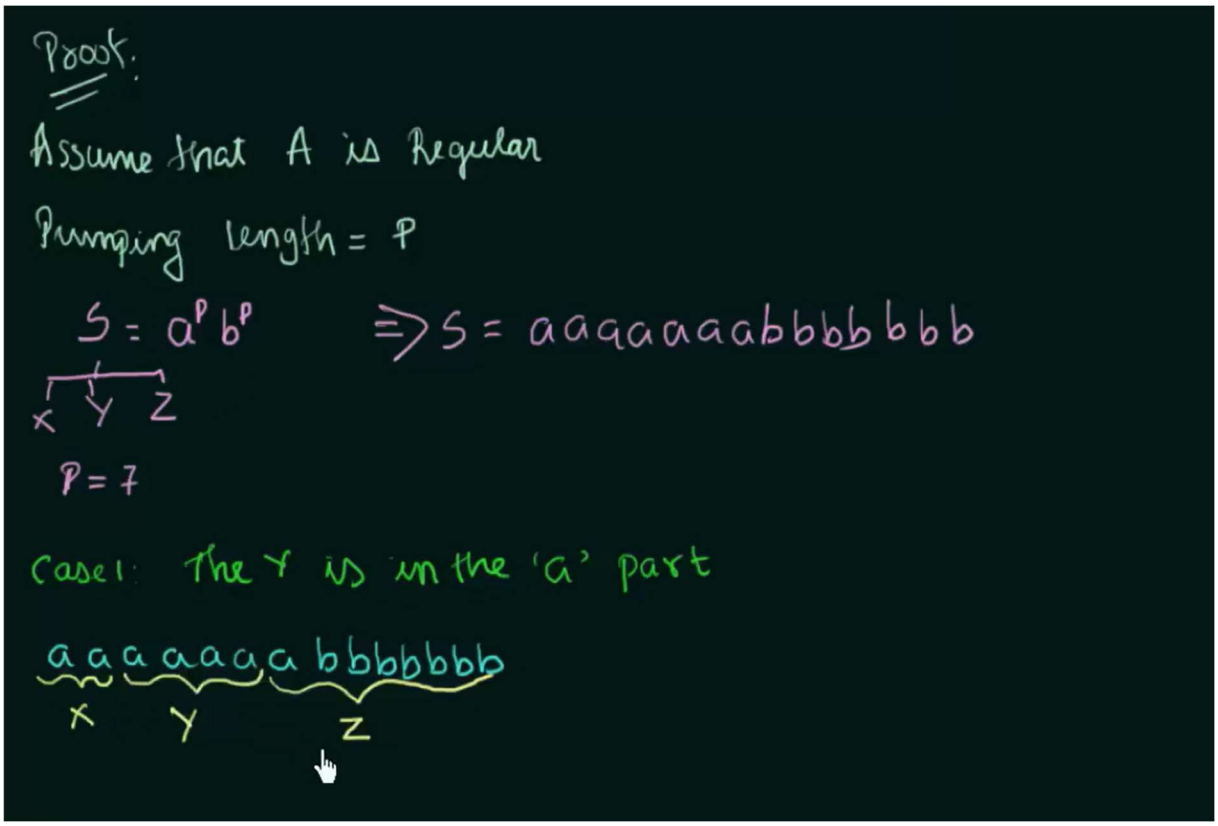


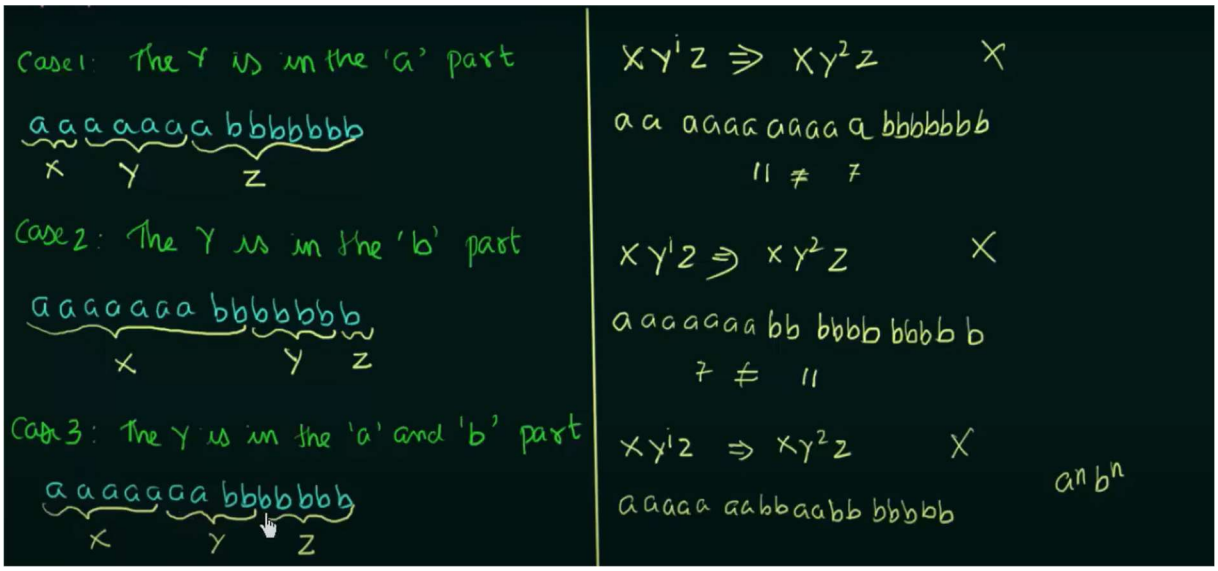
Example:

Pumping Lemma (For Regular Languages) - EXAMPLE (Part-1)

Using Pumping Lemma prove that the language $A = \{a^n b^n \mid n \geq 0\}$ is Not Regular

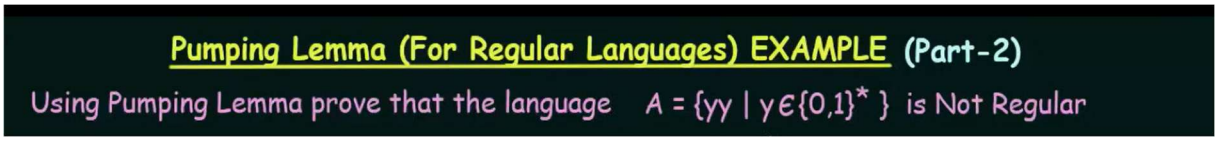
This language is not regular, because in order to make so many a's as b's we should count how many a's do we have to make b's. That is not possible with the finite state machine, a finite state machine has very limited memory. It can't count or store. Because this language can't be designed using finite state machine, therefore this language isn't regular.





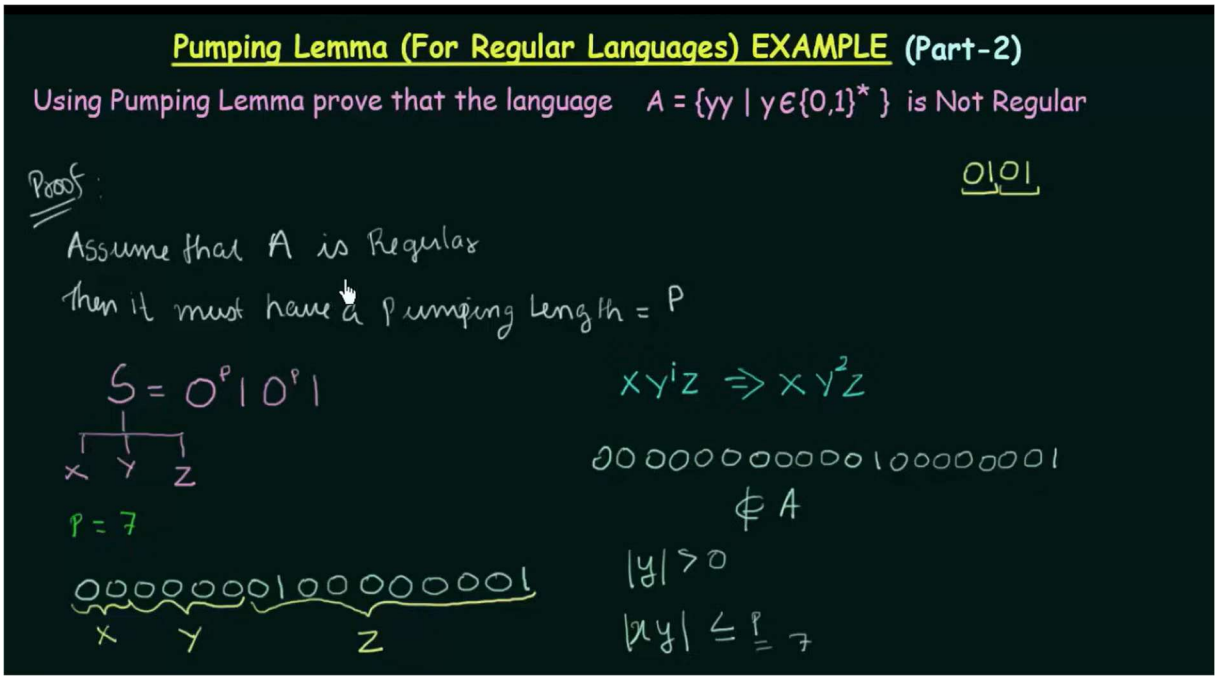
Even the 3rd condition |xy| isn't in all cases satisfied, in the 1st case it is true but in the 2nd and the 3rd case |xy| < P , not all the 3 conditions can be satisfied all at the same time, therefore S can't be pumped and this is our contradiction➔ A isn't a regular language.

Example 2:



This language isn't regular language because in order to make a string , that consists of two strings where the 1st substring should be equal to the 2nd substring , the substring is any combination of the set {0,1} of any length, in order to do that we need to store the 1st substring in a memory to repeat it .

For example if y= 01 ➔ 0101 Is an Element of the language A, or if y= 0011 ,then 0011 0011 element of the language A . This language can't be designed using a finite state machine ➔ this language isn't regular.



The 1st condition is not true here ,the 2nd condition is true because the length of y ,|y| >= 0 and the length of |xy| <= P , is in this case satisfied but if you choose the y in such a manner that has more length than it is, then this condition isn't satisfied.

Regular Grammar

Noam Chomsky gave a Mathematical model of Grammar which is effective for writing computer languages

The four types of Grammar according to Noam Chomsky are:

Grammar Type	Grammar Accepted	Language Accepted	Automaton
TYPE-0	Unrestricted Grammar	Recursively Enumerable Language	Turing Machine
TYPE-1	Context Sensitive Grammar	Context Sensitive Language	Linear Bounded Automaton
TYPE-2	Context Free Grammar	Context Free Language	Pushdown Automata
TYPE-3	Regular Grammar	Regular Language	Finite State Automaton

Grammar:

A Grammar 'G' can be formally described using 4 tuples as $G = (V,T,S,P)$ where,

- V = Set of Variables or Non-Terminal Symbols
- T = Set of Terminal Symbols
- S = Start Symbol
- P = Production rules for Terminals and Non-Terminals

  A production rules has the form $\alpha \longrightarrow \beta$ where α and β are strings on V U T and atleast one symbol of α belongs to V.

Example: $G = (\{S,A,B\}, \{a,b\}, S, \{S \rightarrow AB,\ A \rightarrow a,\ B \rightarrow b\})$

$V = \{S, A, B\}$

$T = \{a, b\}$

$S = S$

$P = S \rightarrow AB,\ A \rightarrow a,\ B \rightarrow b$

Eg: $S \rightarrow AB$

$\rightarrow aB$

$\rightarrow ab$

Regular Grammar:

Regular Grammar can be divided into two types:

Right Linear Grammar	Left Linear Grammar
A grammar is said to be Right Linear if all productions are of the form $A \rightarrow xB$ $A \rightarrow x$ where $A,B \in V$ and $x \in T$	A grammar is said to be Left Linear if all productions are of the form $A \rightarrow Bx$ $A \rightarrow x$ where $A,B \in V$ and $x \in T$

Eg: $S \rightarrow abS \mid b$ - Right linear

$S \rightarrow Sbb \mid b$ - Left linear

Derivations from a Grammar

The set of all strings that can be derived from a Grammar is said to be the LANGUAGE generated from that Grammar

Example 1: Consider the Grammar G1 = ({S,A}, {a,b}, S, {S→ aAb, aA→ aaAb, A→ ϵ })

$S \rightarrow aAb$ [by $S \rightarrow aAb$]
$\rightarrow aaAbb$ [by $aA \rightarrow aaAb$]
$\rightarrow a\,aaAbbb$ [by $aA \rightarrow aaAb$]
$\rightarrow a\,aabbb$ [by $A \rightarrow \epsilon$]

Example 2: G2 = ({S,A,B}, {a,b}, S, {S→AB, A→a, B→b})

$S \rightarrow AB$
$\rightarrow ab$

$L(G2) = \{ab\}$

Example 3: G3 = ({S,A,B}, {a,b}, S, {S→AB, A→aA|a, B→bB|b})

$S \rightarrow AB \rightarrow ab$

$S \rightarrow AB \rightarrow aAbB \rightarrow aabb$

$S \rightarrow AB \rightarrow aAb \rightarrow aab$

$S \rightarrow AB \rightarrow abB \rightarrow abb$

$L(G3) = \{ab, a^2b^2, a^2b, ab^2, \ldots\}$

$= \{a^m b^n \mid m \geq 0 \text{ and } n \geq 0\}$

Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{V, \Sigma, S, P\}$ where

- V = Set of Variables or Non-Terminal Symbols
- Σ = Set of Terminal Symbols
- S = Start Symbol
- P = Production Rule

Context Free Grammar has Production Rule of the form

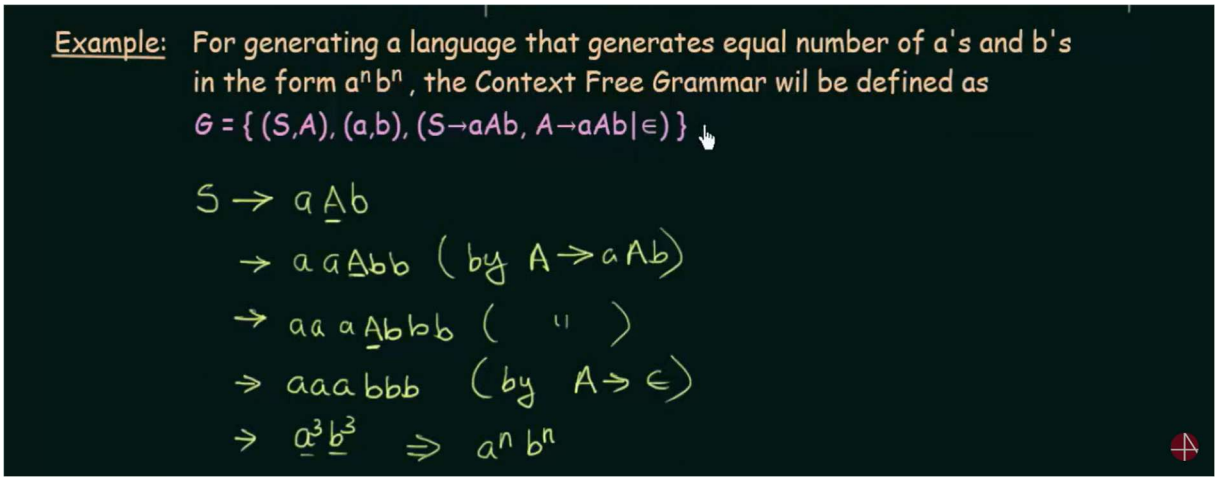
$A \rightarrow \alpha$

where, $\alpha = \{V \cup \Sigma\}^*$ and $A \in V$

The difference between Regular Grammar and the context free grammar is in the production rule

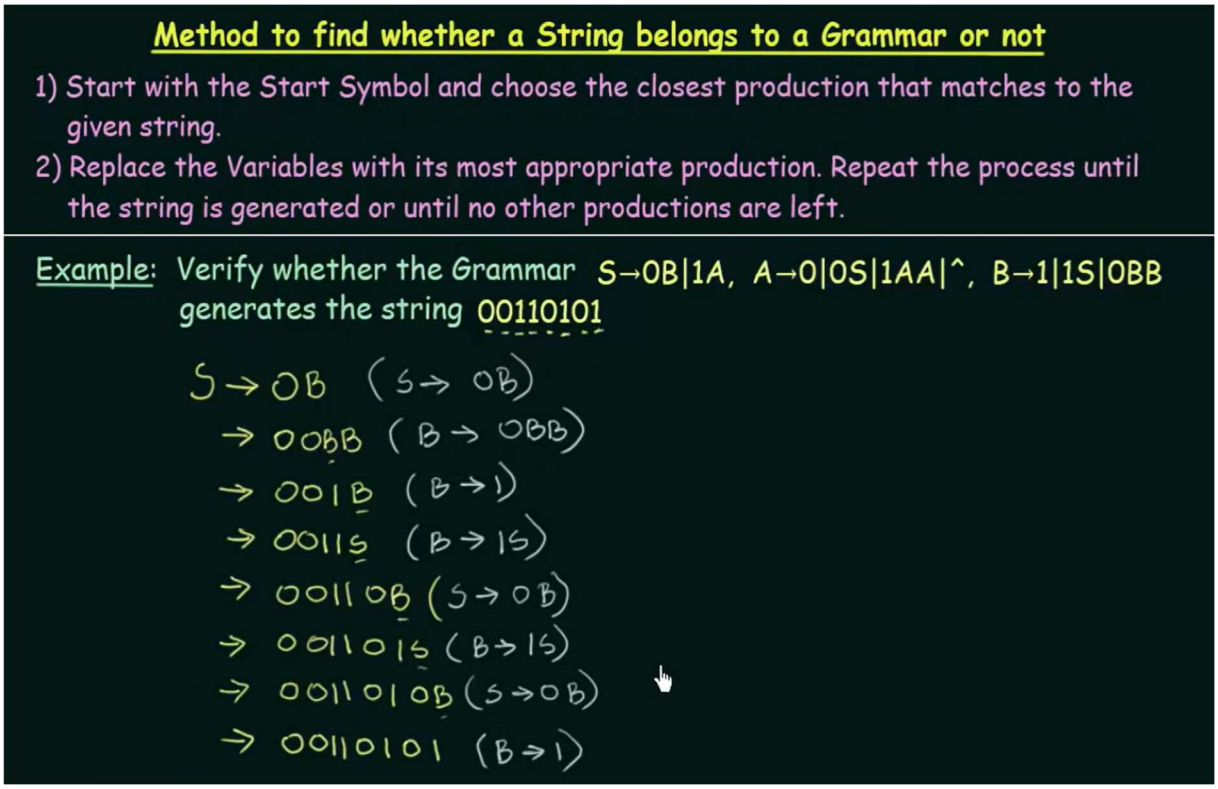
Alpha can be a non-terminal symbol or a terminal symbol or epsilon or any combination of non-terminal symbol and terminal symbol.

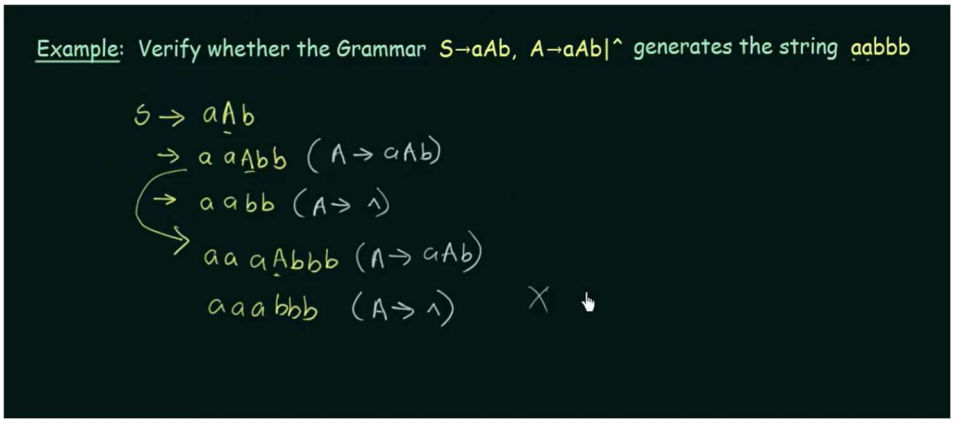
A is a non-terminal symbol.

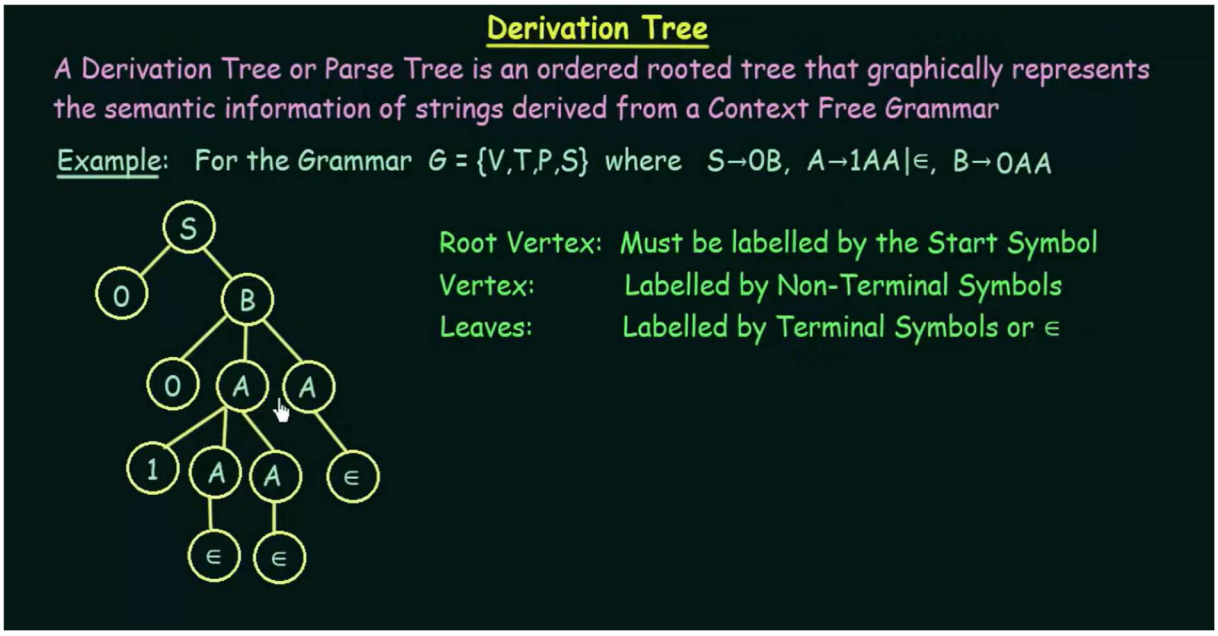


This language isn't a regular language and because the context free language is a higher language than the regular language, the language $a^n b^n$ can designed using the context free Grammar and we will see later , that we can design for the context free Grammar the Pushdown Automaton.

⇨ See the solved problems later

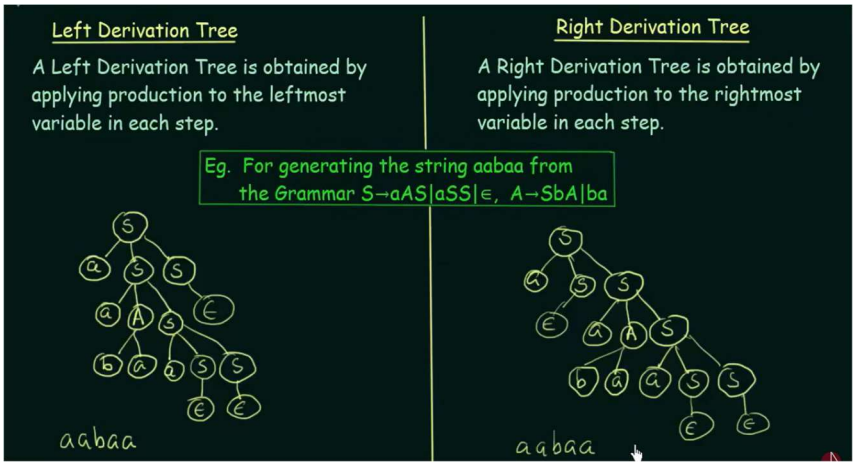


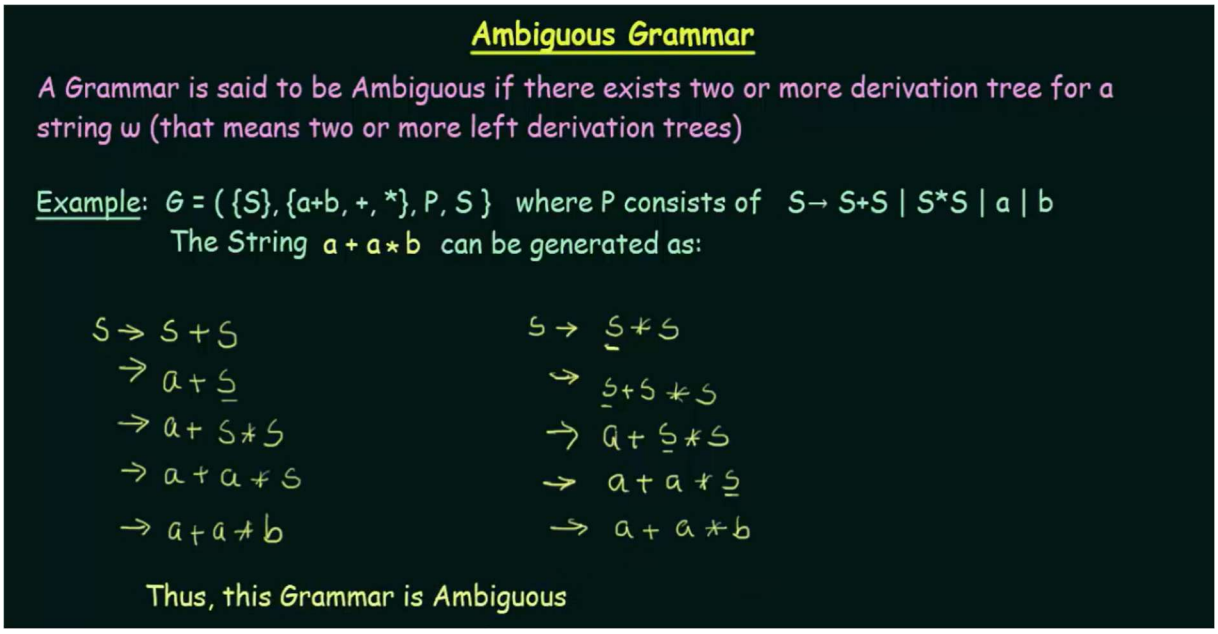




A derivation tree is an ordered rooted tree that shows the graphical representation of how strings can be generated from the context free grammar.

The root vertex is a vertex that is labeled using the start symbol, A leave is a vertex that doesn't have any children (we can say ending vertex) the vertex that is labeled by epsilon is also a leave , a vertex is labeled by a non-terminal symbol.





We used here left derivation tree and we see also that the string a + a * b is generated by two left derivation trees, therefore the grammar is ambiguous.

Simplification of context free Grammar and pumping lemma for context free language



REDUCTION OF CFG

CFG are reduced in two phases

Phase 1: Derivation of an equivalent grammar G' , from the CFG, G, such that each variable derives some terminal string

Derivation Procedure:

Step 1: Include all Symbols W_1 ,that derives some terminal and initialize i = 1

Step 2: Include symbols W_{i+1} , that derives W_i

Step 3: Increment i and repeat Step 2, until $W_{i+1} = W_i$

Step 4: Include all production rules that have W_i in it

Phase 2: Derivation of an equivalent grammar G'' , from the CFG, G', such that each symbol appears in a sentential form

Derivation Procudure:

Step 1: Include the Start Symbol in Y_1 and initialize i =1

Step 2: Include all symbols Y_{i+1} , that can be derived from Y_i and include all production rules that have been applied

Step 3: Increment i and repeat Step 2, until $Y_{i+1} = Y_i$

Example: Find a reduced grammar equivalent to the grammar G, having production rules

P: $S \rightarrow AC|B,\ A \rightarrow a,\ C \rightarrow c|BC,\ E \rightarrow aA|e$

You can see that the non terminal symbols are S, A, C, B,E

And the terminal symbols are a,c,e.

Phase 1

$T = \{a, c, e\}$

$W_1 = \{A, C, E\}$

$W_2 = \{A, C, E, S\}$

$W_3 = \{A, C, E, S\}$

$G' = \{(A, C, E, S), \{a, c, e\}, P, (S)\}$

P: $S \rightarrow AC,\ A \rightarrow a,\ C \rightarrow c,\ E \rightarrow aA|e$

First we write the terminal symbols {a,c,e} and then we write the non-terminal symbols {A,E,C} .

Phase 1: In w1 we write all the symbols that derive the terminal symbols.

In w2 write all symbols that can derive the symbols present in the set w1

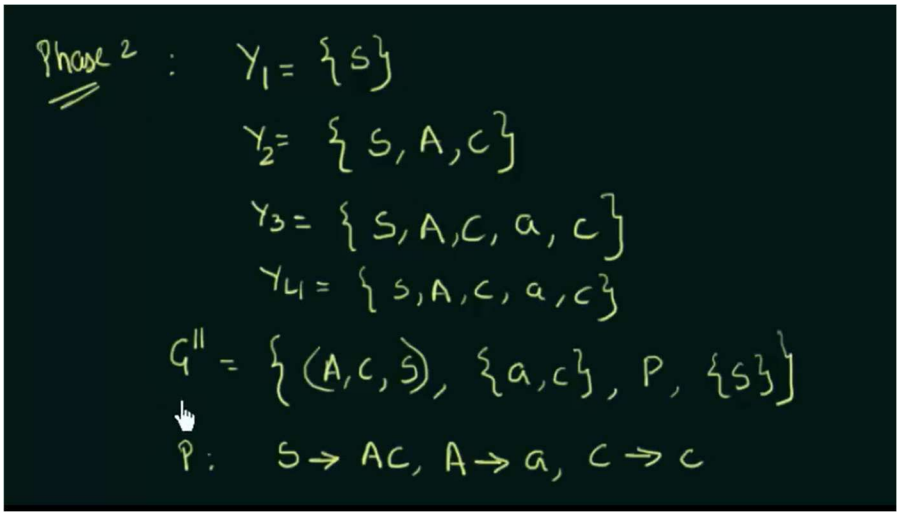
In W2 will contain for sure A, C, E and also S , because S derives AC | B.

In w3 we will write all symbols that can derive A,C,E,S

We found that in 2 consecutive steps that we have got the same result, therefore we can stop.

The new Grammar is G'

The production rule in the new Grammar G', because we didn't have B as non-terminal symbol in the new Grammer , we shouldn't use B in the production rule.



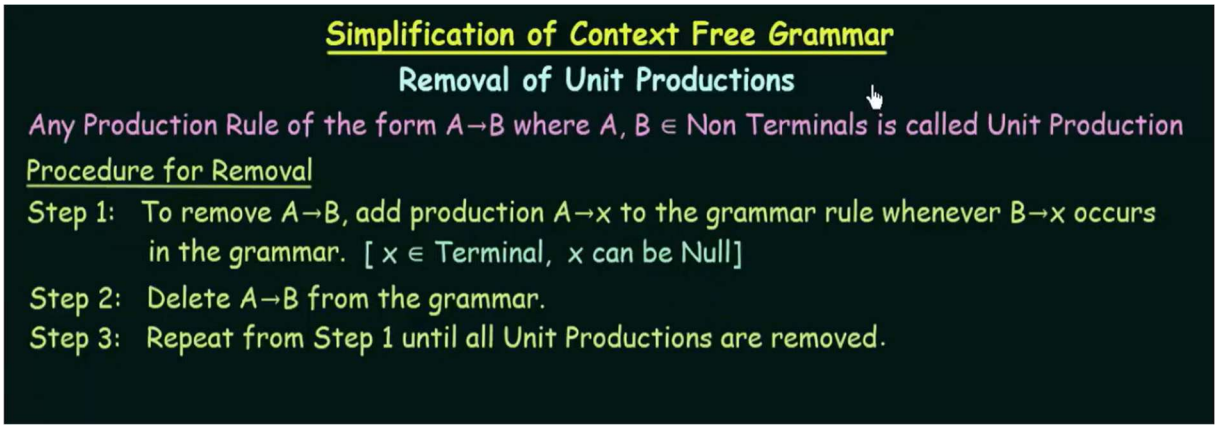
Phase 2: in y1 will just include the start symbol

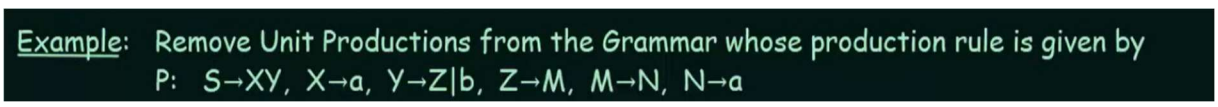
In y2 you include all symbols that can be derived from the start symbol.

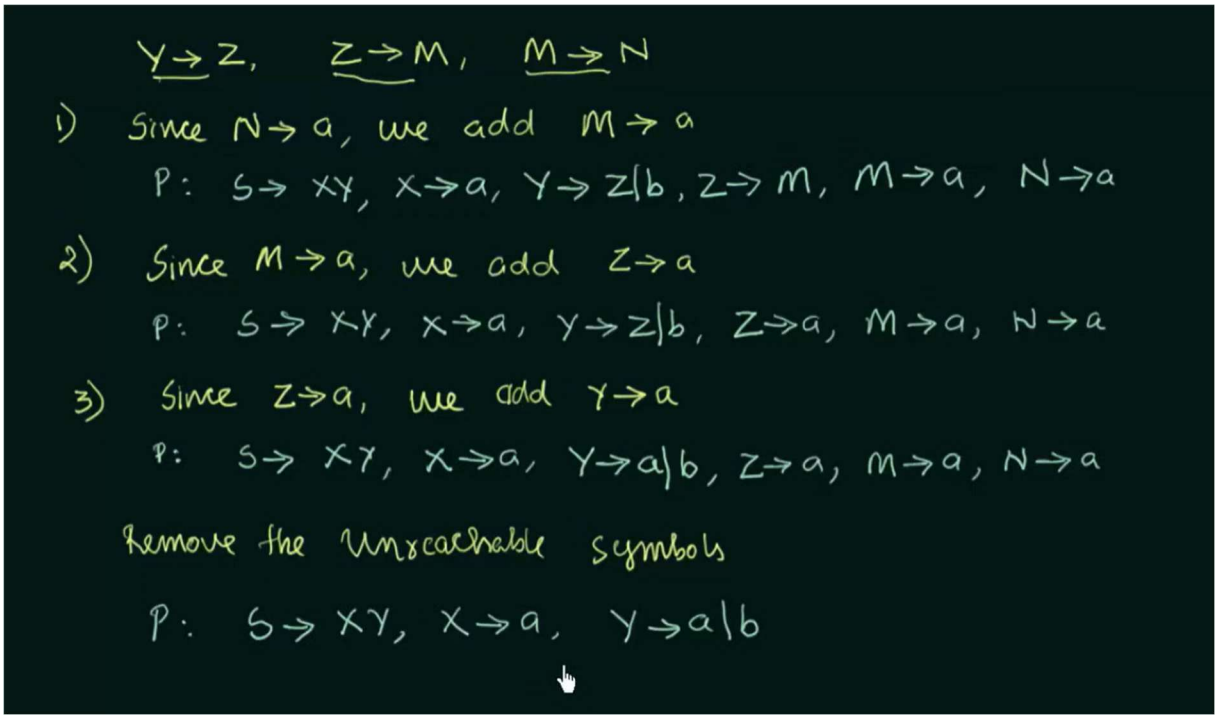
In y3 we include all symbols that can be derived from the y2 set. In y3 and y4 the same procedure.

In then define the new Grammar G''.

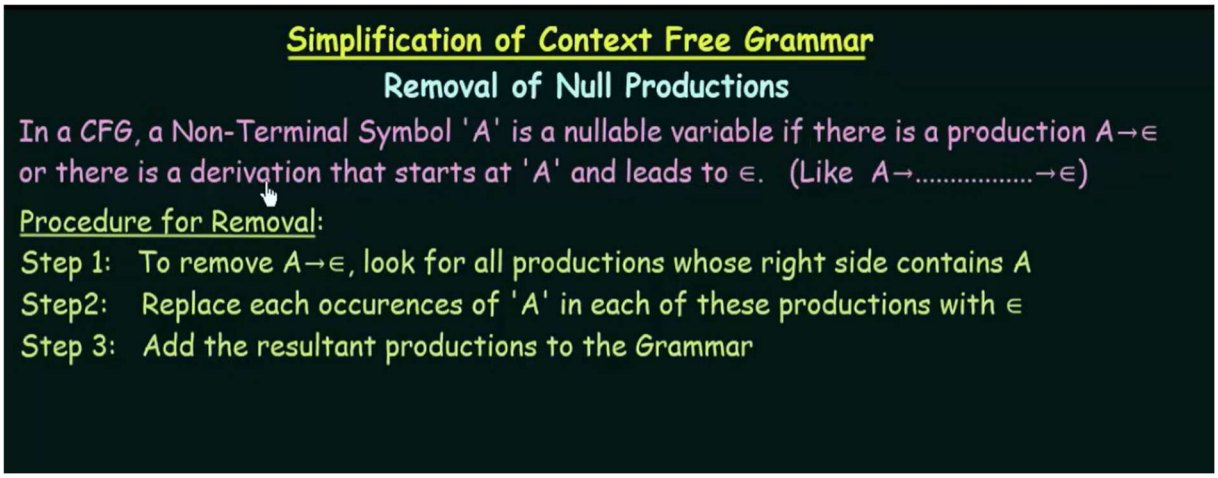
G'' is a reduced form of the Grammer G, and is the result of the simplification .

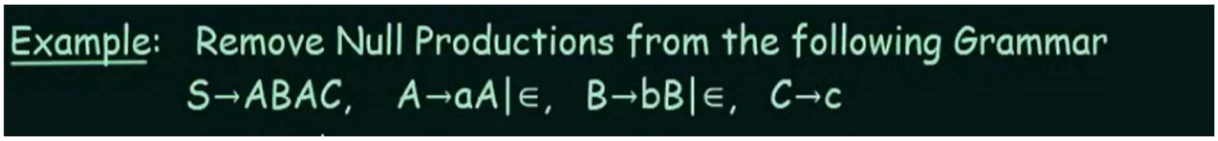






Removal of Null Productions





$S \to ABAC,\ A \to aA \mid \in,\ B \to bB \mid \in,\ C \to c$

$A \to \in$, $B \to \in$

1) To eliminate $A \to \in$

$S \to ABAC$

$S \to ABC \mid BAC \mid BC$

$A \to aA$

$A \to a$

New production: $S \to ABAC \mid ABC \mid BAC \mid BC$

$A \to aA \mid a$, $B \to bB \mid \in$, $C \to c$

2) To eliminate $B \to \in$

$S \to AAC \mid AC \mid C$, $B \to b$

New production: $S \to ABAC \mid ABC \mid BAC \mid BC \mid AAC \mid AC \mid c$

$A \to aA \mid a$

$B \to bB \mid b$

$C \to C$

Pumping Lemma for context free Languages

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

Context Free Language

In formal language theory, a Context Free Language is a language generated by some Context Free Grammar.

The set of all CFL is identical to the set of languages accepted by Pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{ V, \Sigma, S, P \}$ where

- V = Set of Variables or Non-Terminal Symbols
- Σ = Set of Terminal Symbols
- S = Start Symbol
- P = Production Rule

Context Free Grammar has Production Rule of the form

$A \to a$

where, $a = \{V \cup \Sigma\}^*$ and $A \in V$

Pumping Lemma (For Context Free Languages)

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free

If A is a Context Free Language, then, A has a Pumping Length 'P' such that any string 'S', where $|S| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

(1) $u\,v^i x\,y^i z$ is in A for every $i \geq 0$

(2) $|v\,y| > 0$

(3) $|v\,x\,y| \leq P$

To prove that a Language is Not Context Free using Pumping Lemma (for CFL) follow the steps given below: (We prove using CONTRADICTION)

-> Assume that A is Context Free
-> It has to have a Pumping Length (say P)
-> All strings longer than P can be pumped $|S| \geq P$
-> Now find a string 'S' in A such that $|S| \geq P$
-> Divide S into uvxyz
-> Show that $u\,v^i x\,y^i z \notin A$ for some i
-> Then consider the ways that S can be divided into uvxyz
-> Show that none of these can satisfy all the 3 pumping conditions at the same time
-> S cannot be pumped == CONTRADICTION

Example:

Pumping Lemma (for Context Free Languages) - Example (Part-1)

Show that $L = \{ a^N b^N c^N \mid N \geq 0 \}$ is Not Context Free

Pumping Lemma (for Context Free Languages) - Example (Part-1)

Show that $L = \{ a^N b^N c^N \mid N \geq 0 \}$ is Not Context Free

-> Assume that L is Context Free
-> L must have a pumping length (say P)
-> Now we take a string S such that $S = a^P b^P c^P$
-> We divide S into parts u v x y z

Eg. P = 4 So, $S = a^4 b^4 c^4$

Case I : v and y each contain only one type of symbol

a a a a b b b b c c c c

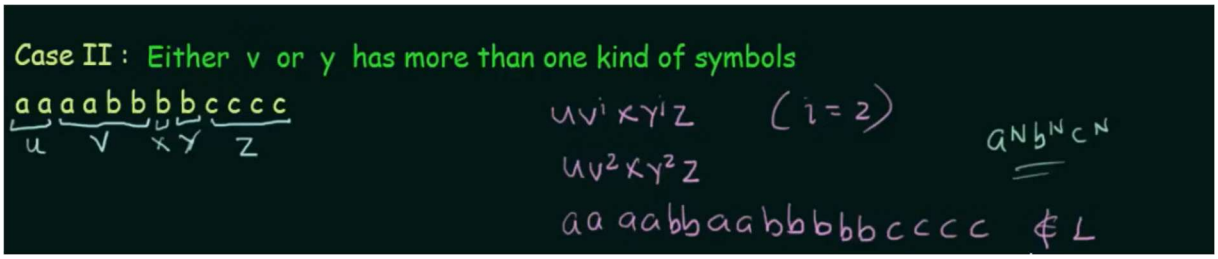
u v x y z

uv^ixy^iz $(i = 2)$

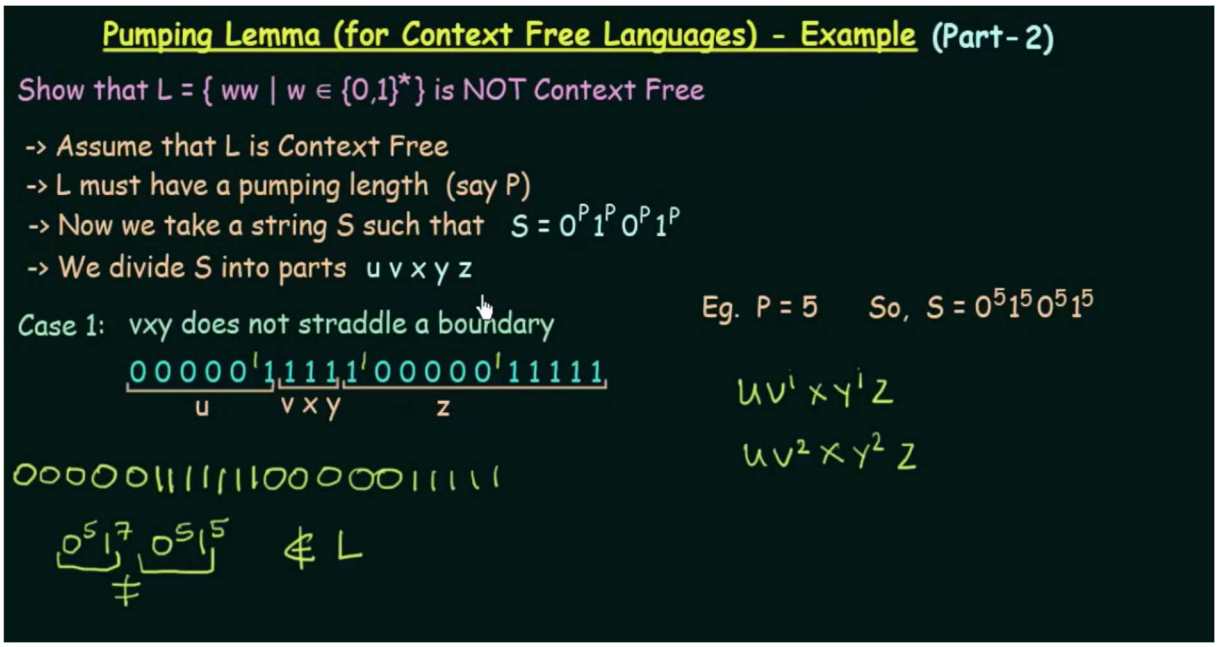
uv^2xy^2z

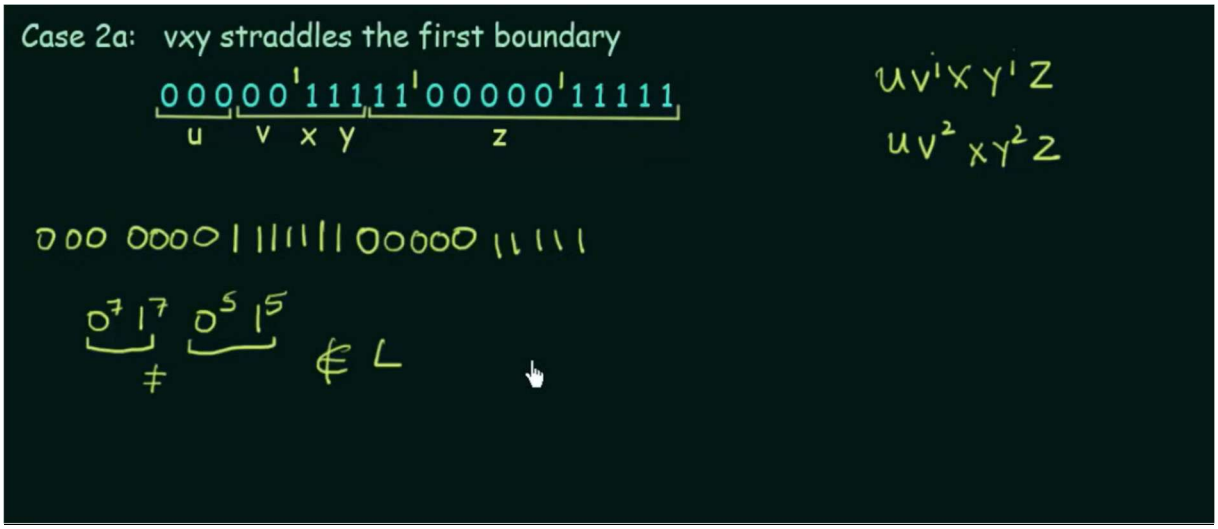
a a a a a a b b b b c c c c c

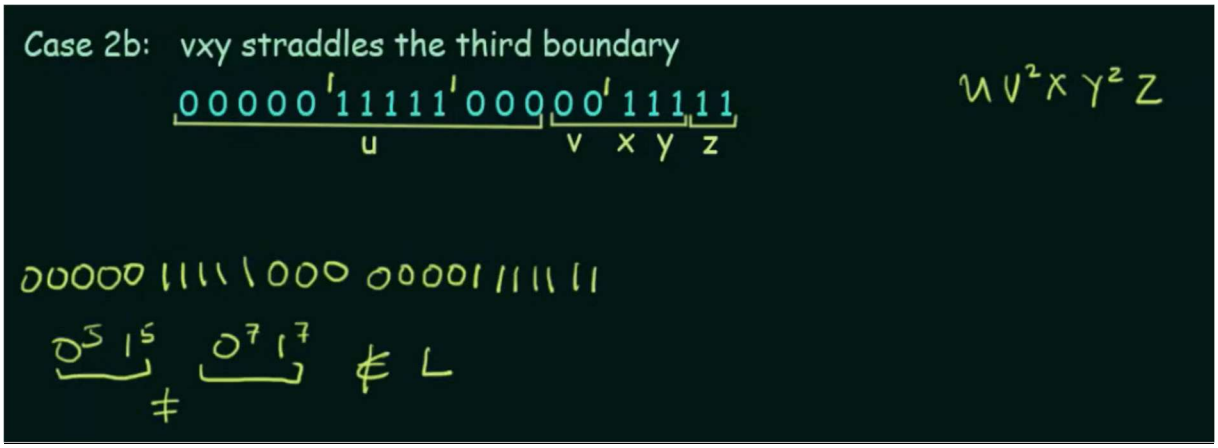
$a^6 b^4 c^5 \notin L$

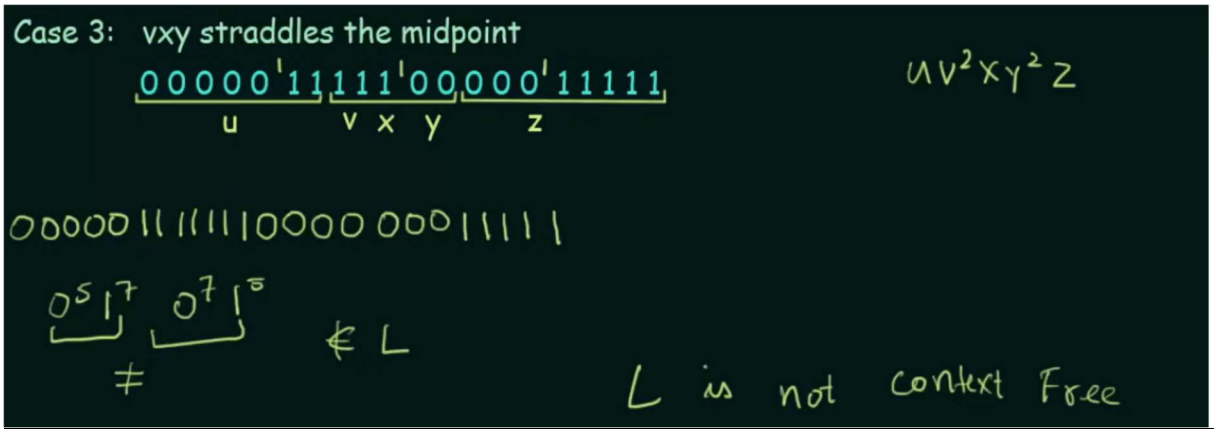

Case II : Either v or y has more than one kind of symbols
aaaabbbbcccc
u v x y z
uv^i xy^i z (i = 2)
uv^2 xy^2 z
aa aabbaabbbbbbcccc ∉ L
a^N b^N c^N

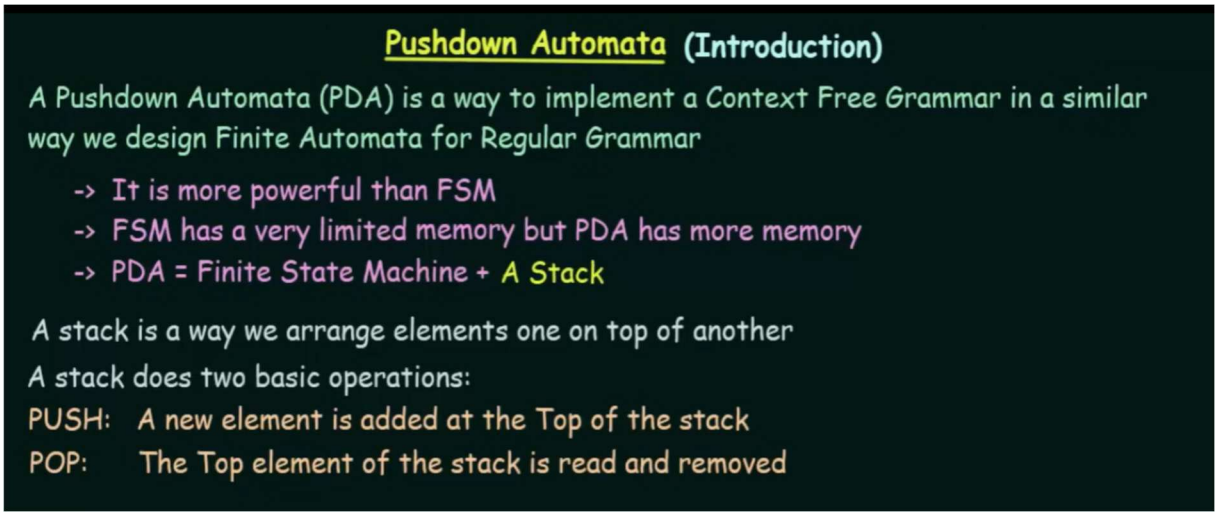
Example 2:


Pumping Lemma (for Context Free Languages) - Example (Part-2)
Show that L = { ww | w ∈ {0,1}* } is NOT Context Free
-> Assume that L is Context Free
-> L must have a pumping length (say P)
-> Now we take a string S such that S = 0^P 1^P 0^P 1^P
-> We divide S into parts u v x y z
Case 1: vxy does not straddle a boundary
Eg. P = 5 So, S = 0^5 1^5 0^5 1^5
00000 1111 1 00000 11111
u v x y z
uv^1 xy^1 z
uv^2 xy^2 z
000001111111000001111
0^5 1^7 0^5 1^5 ∉ L
≠


Case 2a: vxy straddles the first boundary
00000 11111 00000 11111
u v x y z
uv^i xy^i z
uv^2 xy^2 z
000 0000 1111111 00000 11111
0^7 1^7 0^5 1^5 ∉ L
≠







We can define regular grammar and the language accepted by this grammar is the regular language and regular grammar can be designed using finite state machine.

We can also define context free grammar and the language accepted by this grammar is the context free language, the Automata that can accept this grammar and this language is the pushdown Automata.

We saw the limitations of the finite state machine ➔ they have limited memory, there were some languages, that can't be accepted using this language.

PDA is more powerful than FSM and those languages which can't be designed using FSM, can be designed using PDA.

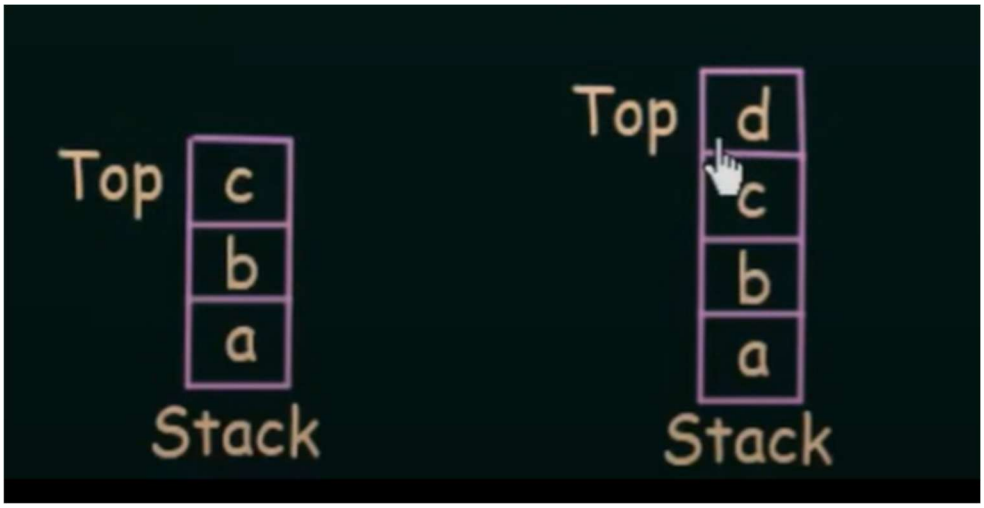
PDA = FSM + Stack

This Stack has infinite memory, this memory Is the thing that facilitates the higher power of the pushdown automata.

A stack is a way to arrange elements where we place the elements on top of another, a stack has 2 operations: push and pop.

Push operation: A new element is added at the Top of the Stack

Pop: the top element is read and removed

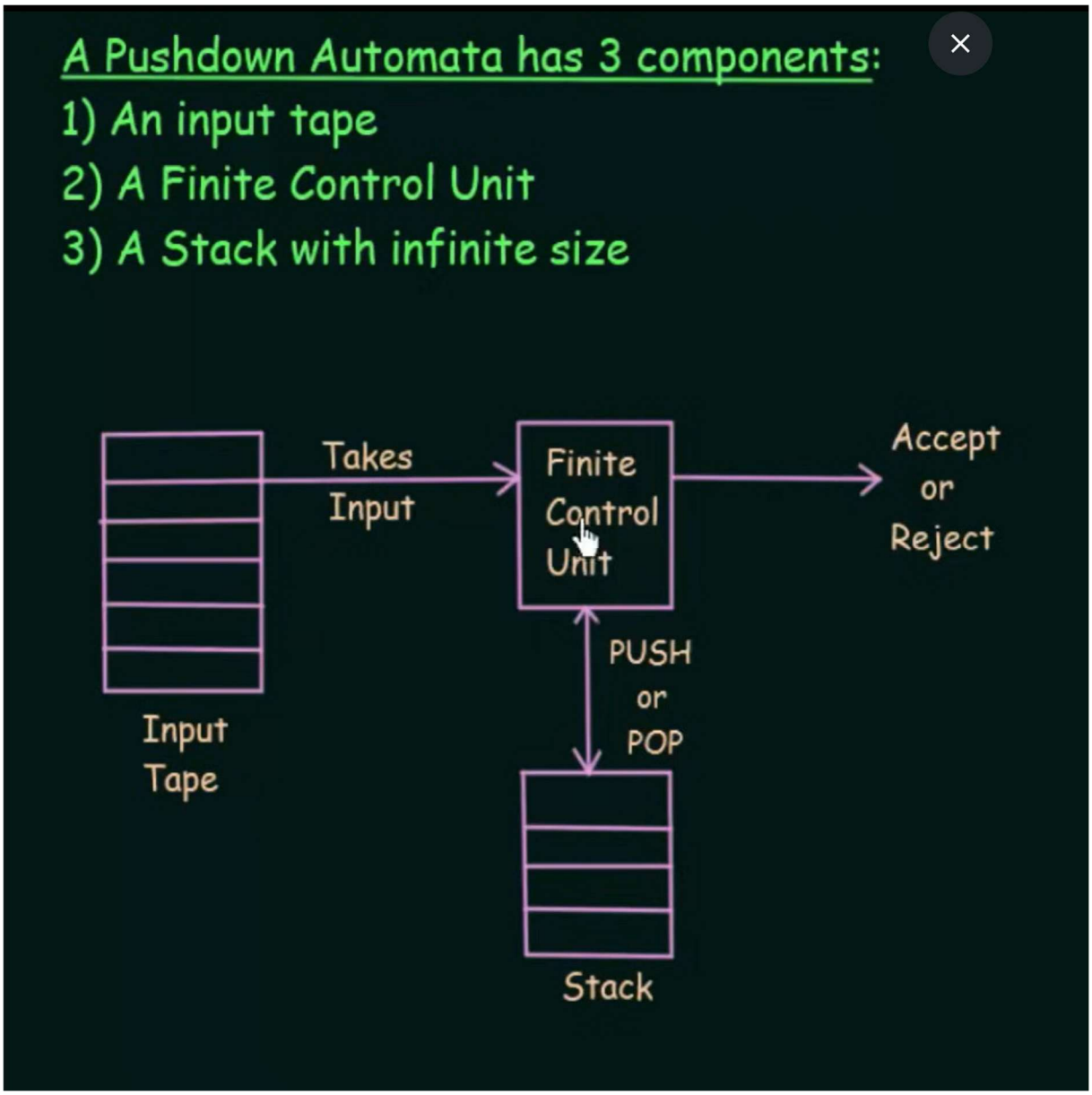




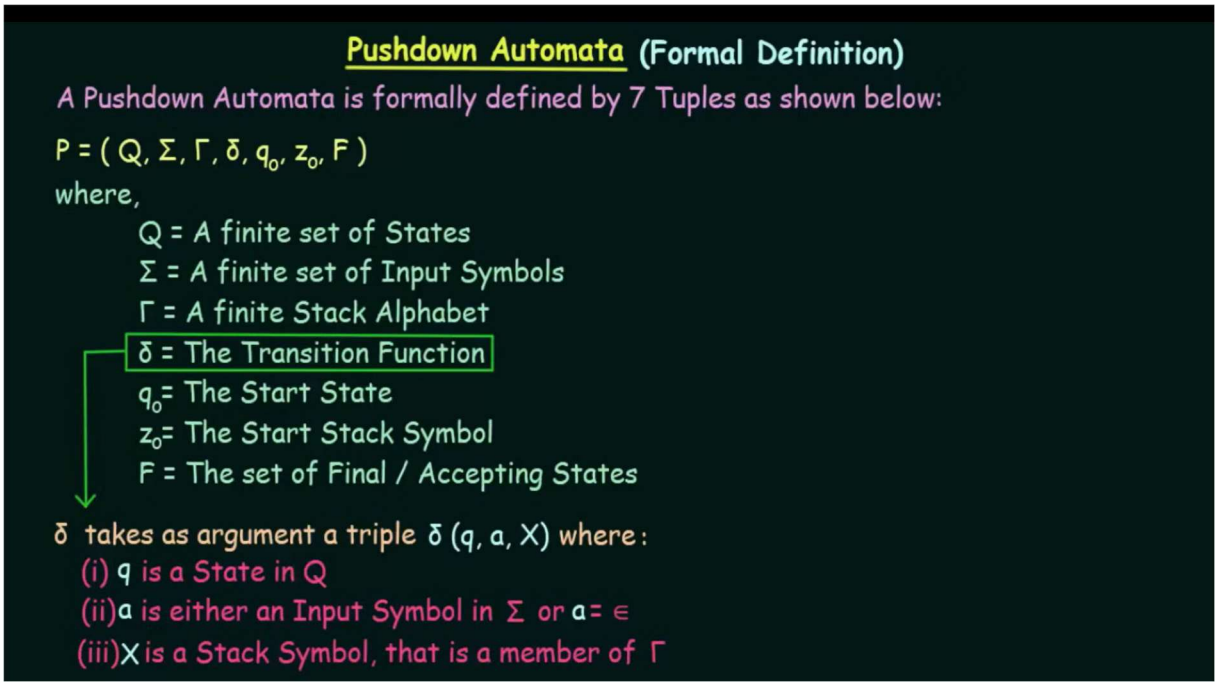
You can imagine a stack is a pile of books , if you have three elements a, b and c, they will put on top of another , the topmost element is denoted by TOP , If you arrange a pile of books you will put books on top of another , if you insert a new book to the stack you will insert it at the top of the topmost book, analog if you insert an element in the stack you will insert it at the topmost element of the stack , the TOP will point to the inserted element in the stack.

POP: if you want to remove a book from the pile , you will start by the topmost element and you should remove it and the TOP will point to the topmost element after removing the element from the stack.

If you perform POP on the stack on the left hand side that contains the a,b,c, then the topmost element will be removed

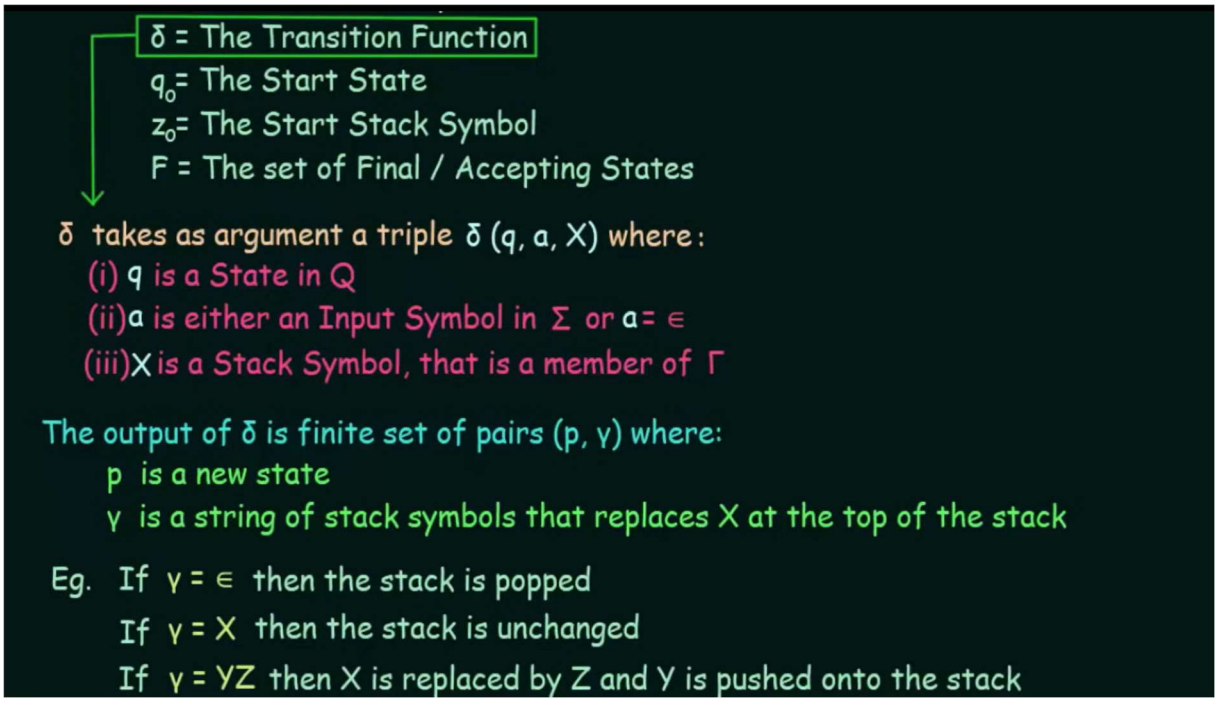


An input tape is like the input string that we have, the input string is given to the finite control unit, the finite control unit communicates with the stack and at the end the input string will be accepted or rejected.



The difference between the 2 definitions of the FSM and PDA, is the 2 tuples z_0 and symbol Γ.

The stack makes the PDA more powerful than the FSM, the transition function δ takes 3 arguments q,a,X.

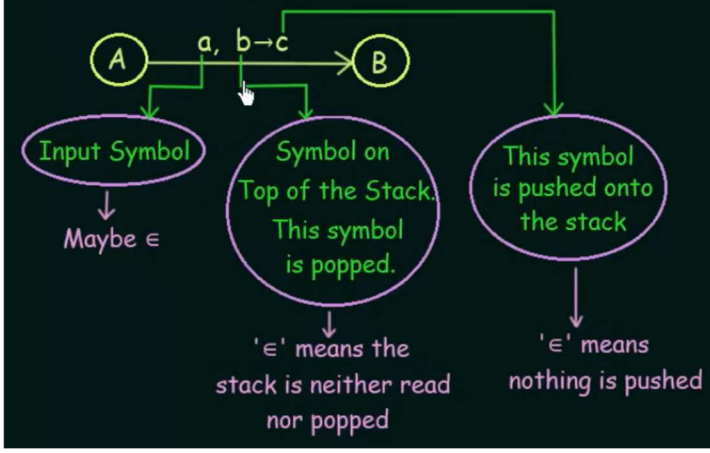


Pushdown Automata (Graphical Notation)

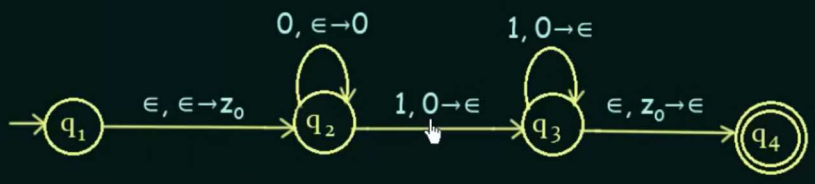
Finite State Machine



Pushdown Automata

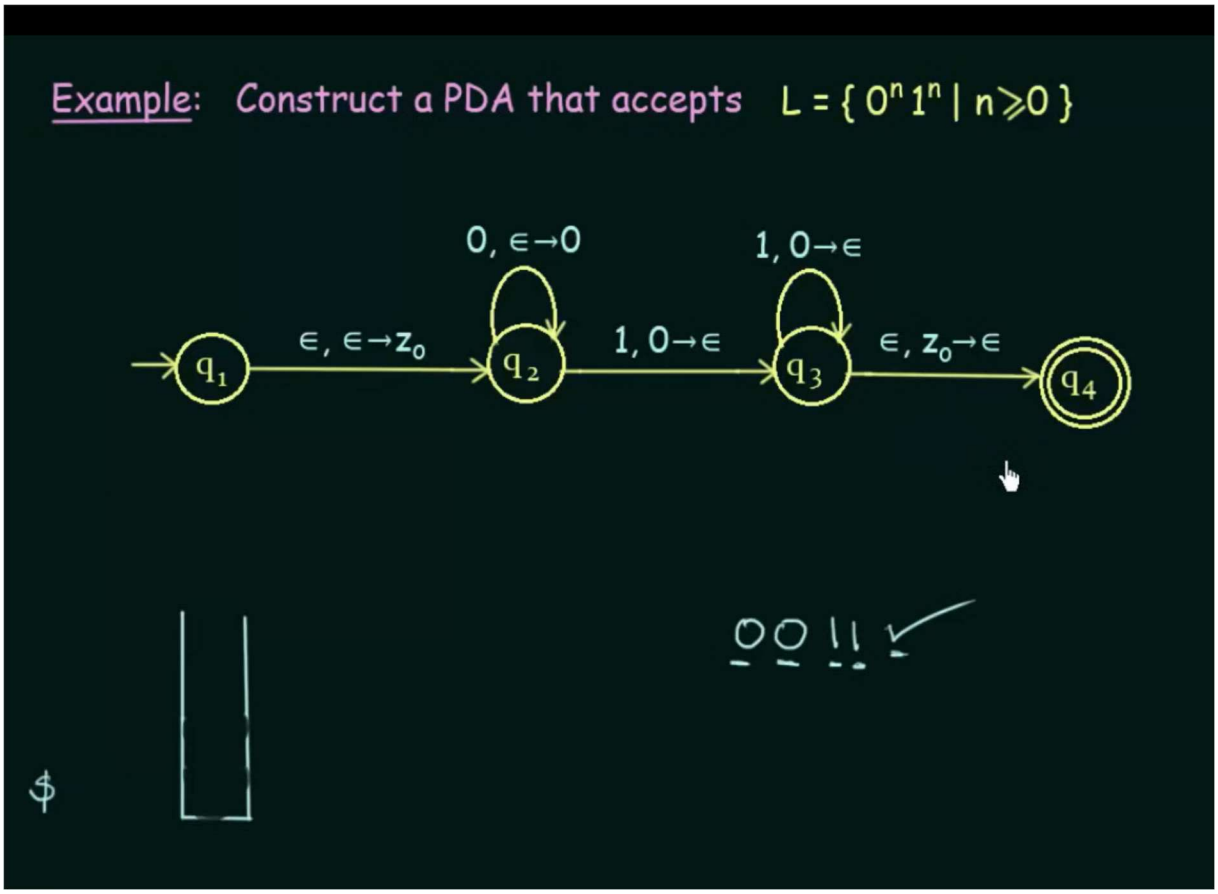


Example: Construct a PDA that accepts $L = \{ 0^n 1^n \mid n \geq 0 \}$

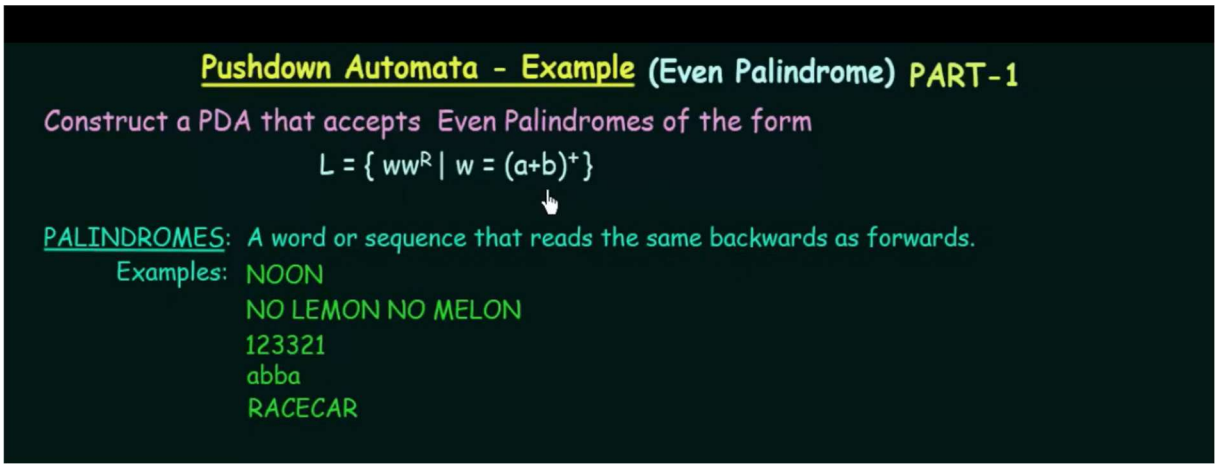




0 0 1 1



A string is accepted if I read the whole string and reach the final state or when the stack is empty, in this case both of them are true.



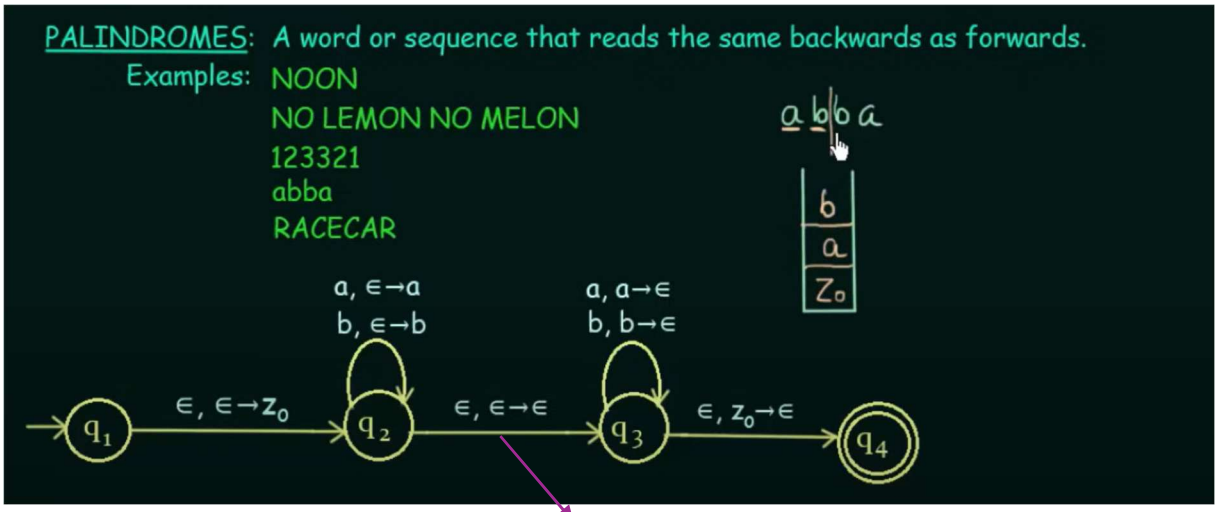
$L = ww^R$, es bedeutet dass erst ein Wort kommt danach kommt das selbst Wort aber in umgekehrter Reihenfolge, daher w^R , d.h. ein Word Reverse

Hier wollen wir Palindroms machen, die aus a,b bestehen, also $(a \mid b)^+$

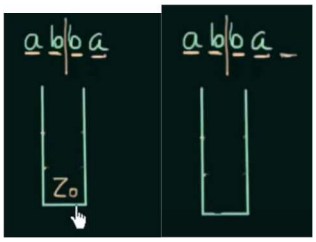
Alle Kombinationen von a und b, empty string isn't allowed und hier muss es gerade sein .

NOON, NO LEMON NO MELON , 123321, abba ➔ sind even palindrome

RACECAR ➔ odd palindrome

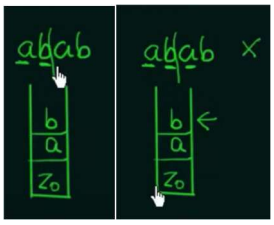


At this Point, you have reached the middle point of the given String.



Because I have read the whole input string and I have reached the final state and the stack is empty, therefore this string is accepted.

If the given string is s = abab, this isn't an even palindrome.

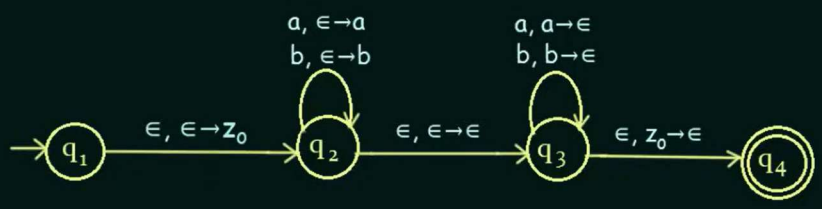


But how do we know, that I have reached the midpoint of the stack and when should I take the transition ε, ε → ε .

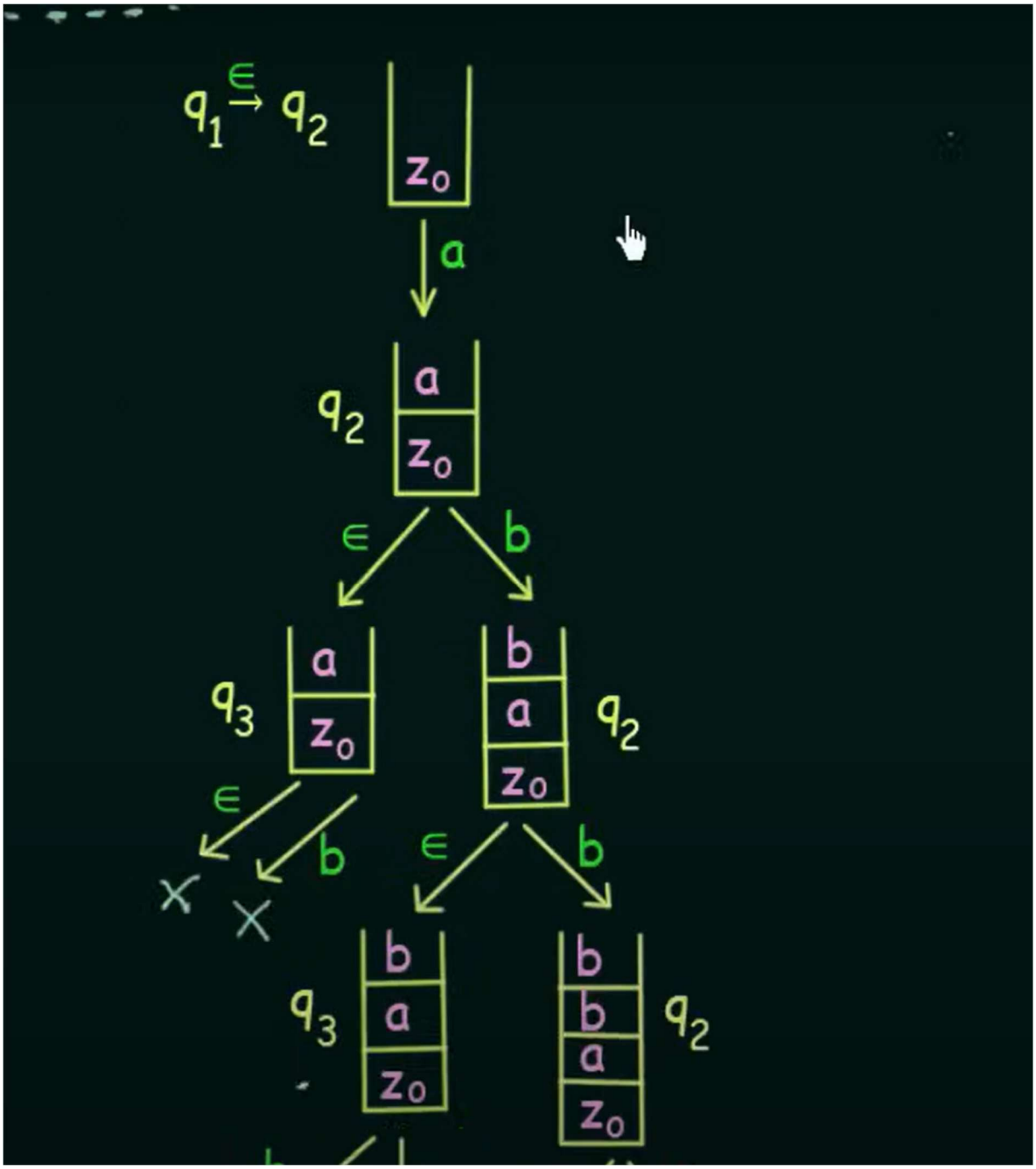
Pushdown Automata - Example (Even Palindrome) PART- 2

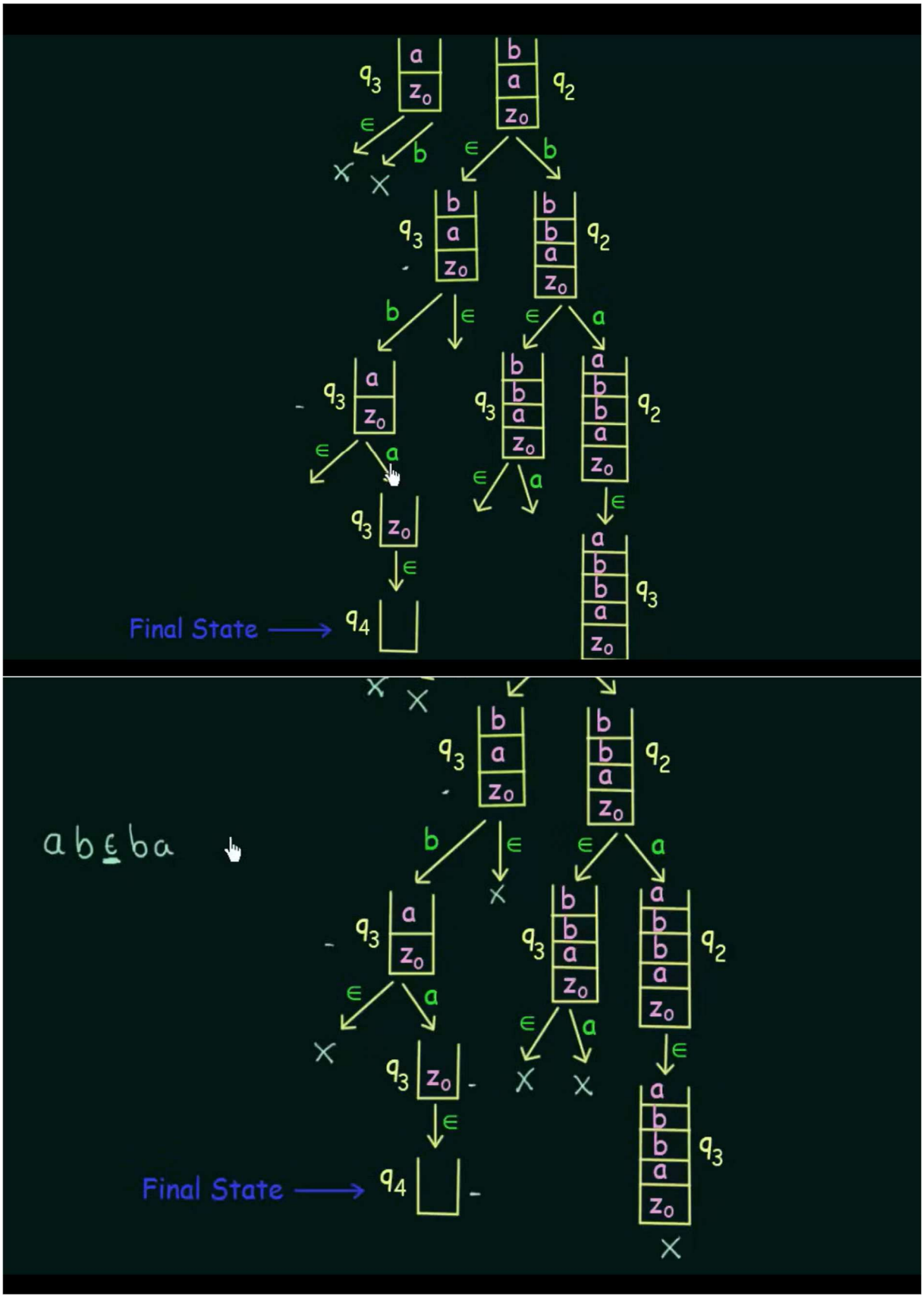
Construct a PDA that accepts Even Palindromes of the form

$$L = \{ ww^R \mid w = (a+b)^+ \}$$



Example: ∈ a ∈ b ∈ b ∈ a ∈


$q_1 \xrightarrow{\in} q_2$
z_0
a
q_2
a
z_0
∈
b
q_3
a
z_0
b
a
z_0
q_2
∈
b
X
X
∈
b
q_3
b
a
z_0
b
b
a
z_0
q_2



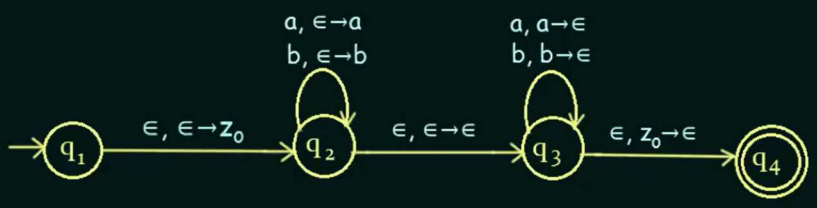
The path, that we have to follow is abεba. There is only one path, that we can follow to get to the final state and where the stack get empty at the end.

What will happen if I pass not an even palindrome ?

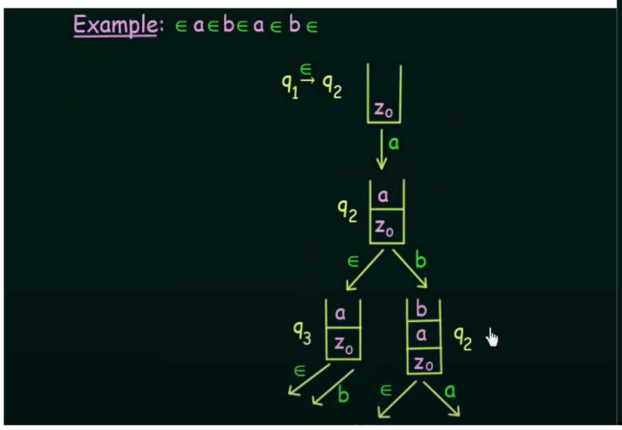
Pushdown Automata - Example (Even Palindrome) PART- 3

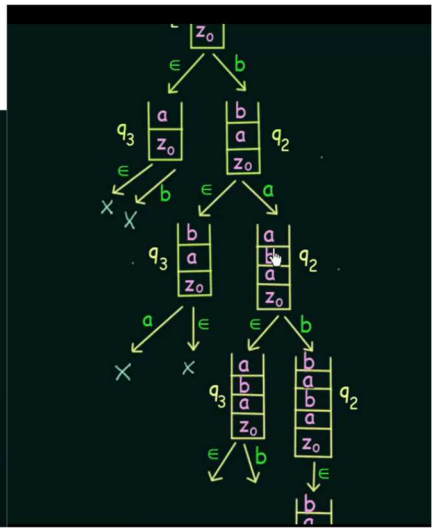
Construct a PDA that accepts Even Palindromes of the form

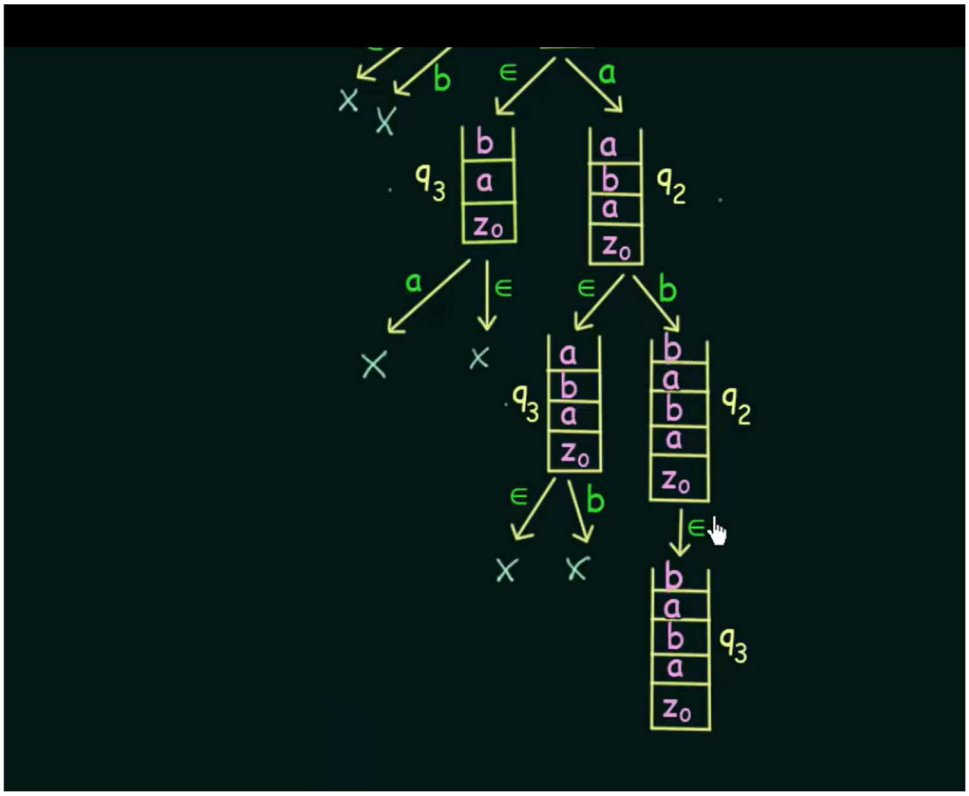
$$L = \{ ww^R \mid w = (a+b)^+ \}$$



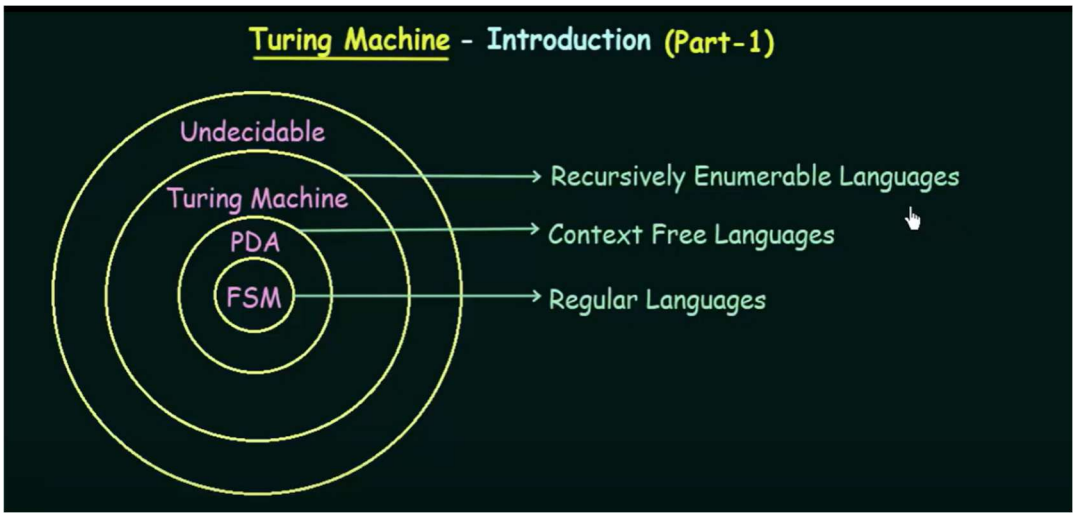
Example: ∈ a ∈ b ∈ a ∈ b ∈



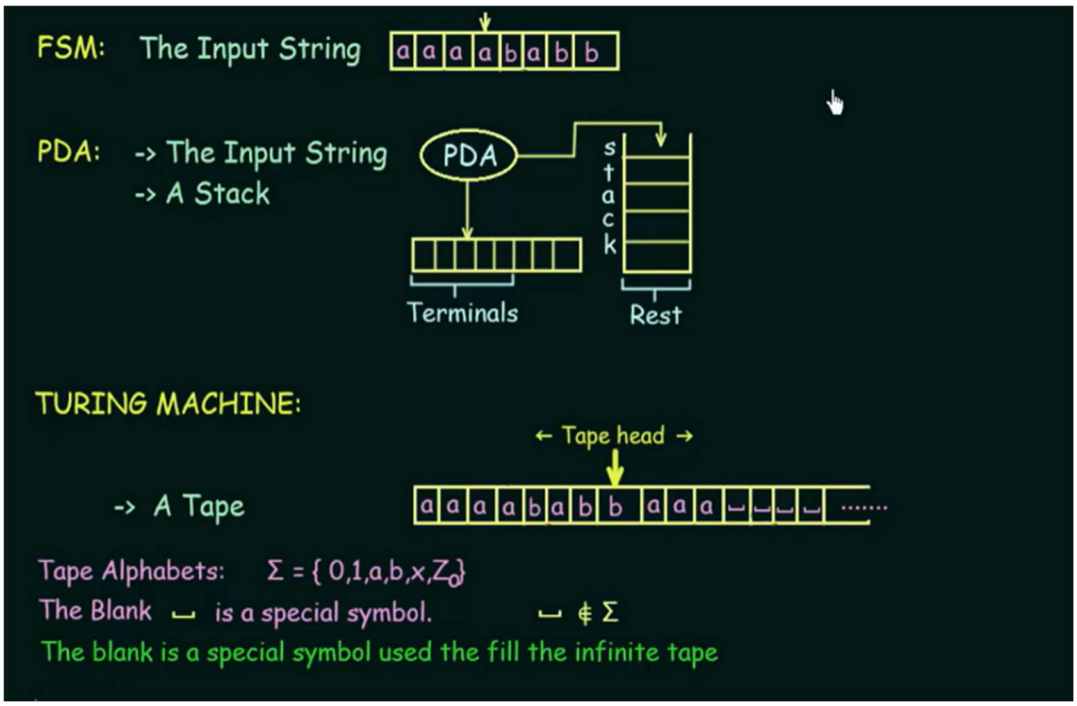




Turing-Machine



The Language accepted by the FSM are the regular languages and the language accepted by the PDA is the context free language, and the language accepted by the Turing Machine is the Recursively enumerable language.



in the FSM, the data structure that we had is just the input string and we had one control that can move only in one Direction (to the right Direction), we saw that the finite state machine has a very limited memory

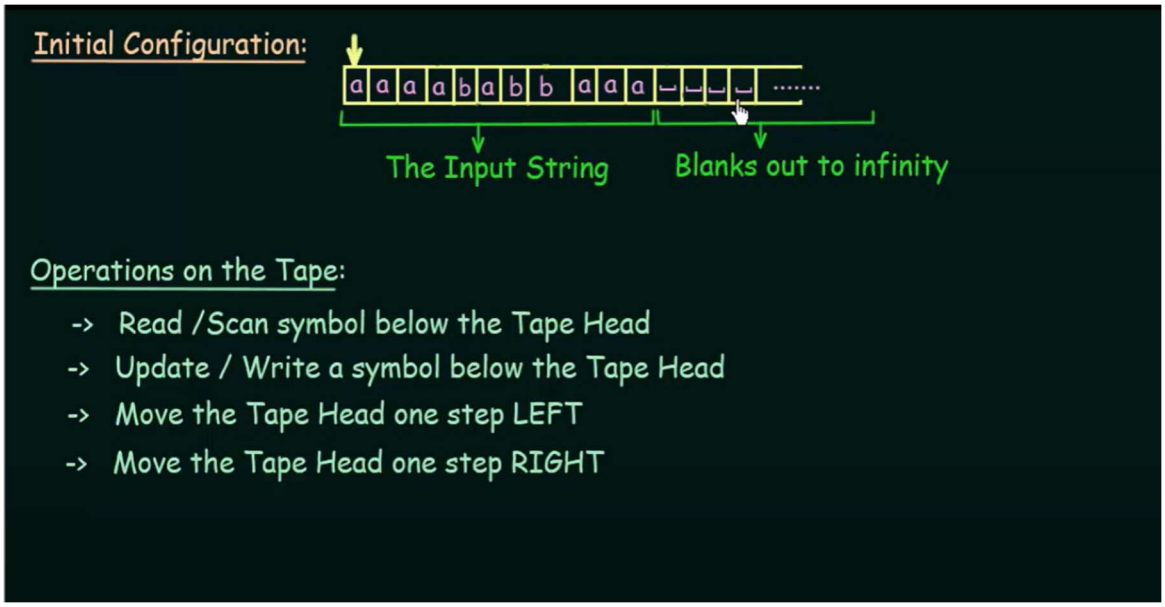
PDA we have an input string, a Stack (Difference between FSM and PDA is the presence of the Stack, because of that we saw that the PDA is more powerful than the FSM and the language accepted by this machine are a higher class of language as compared to the regular languages)

In the Turing machine, the data structure that we have is the tape.

The tape is a sequence of infinite symbols (infinite sequence of symbols) the tape head is positioned on the symbol on which the current control is present, the tape head can move one step to the right or one step to the left.

The input symbols that we have they are filled into the cells and the remaining cells instead of leaving them just blank, we fill them with a special symbol called the blank symbol.

As we know Σ is the input symbols, we call Σ as the Tape Alphabet. The blank symbol $\notin \Sigma$.

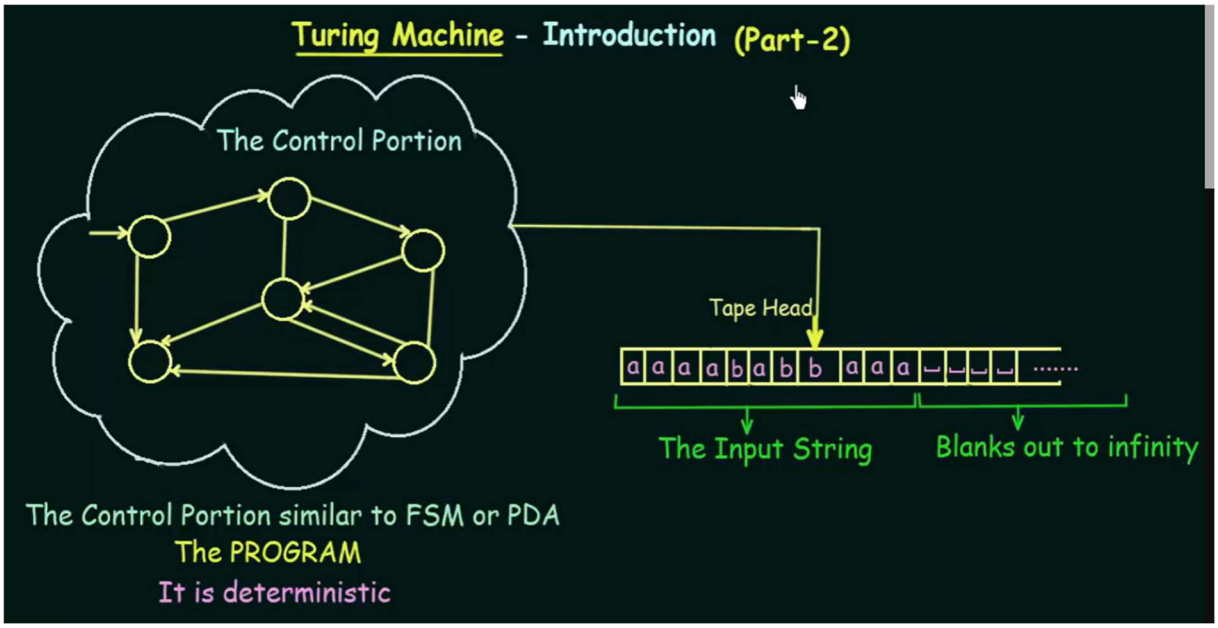


The initial configuration of the Tape: Tape head is pointing to the first element in the Tape on the left, the input string represented by the symbols a, b…

And the empty cells are represented by the blank symbol

Operations:

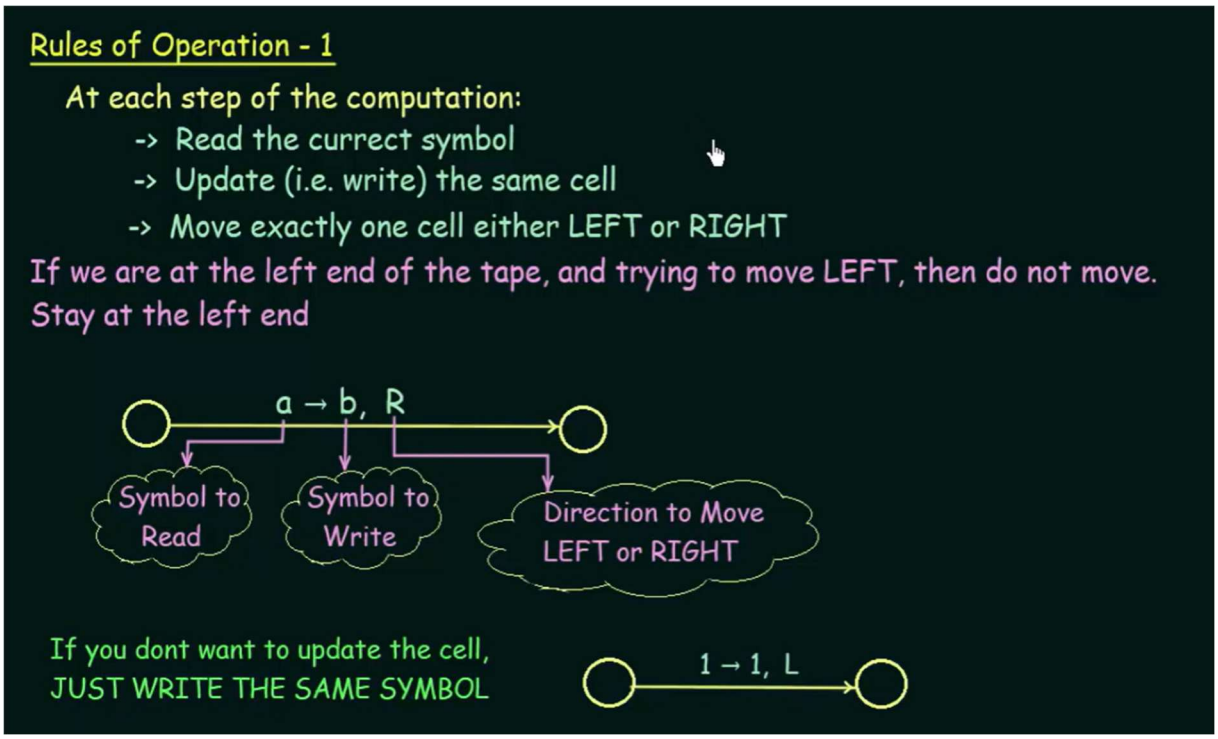
1) Read symbol below the Tape Head (tape head represents the position at which our current control Is present, you can only read / scan the symbol below the tape head)
2) Update the symbol: you should see where the tape head is positioned and the symbol below the tape head should be updated, that means the symbol below the tape head can be overwritten by another symbol.
3) 4) you can only move the tape head to the left or to the right (you can't move the tape head more than one step and not in both directions)



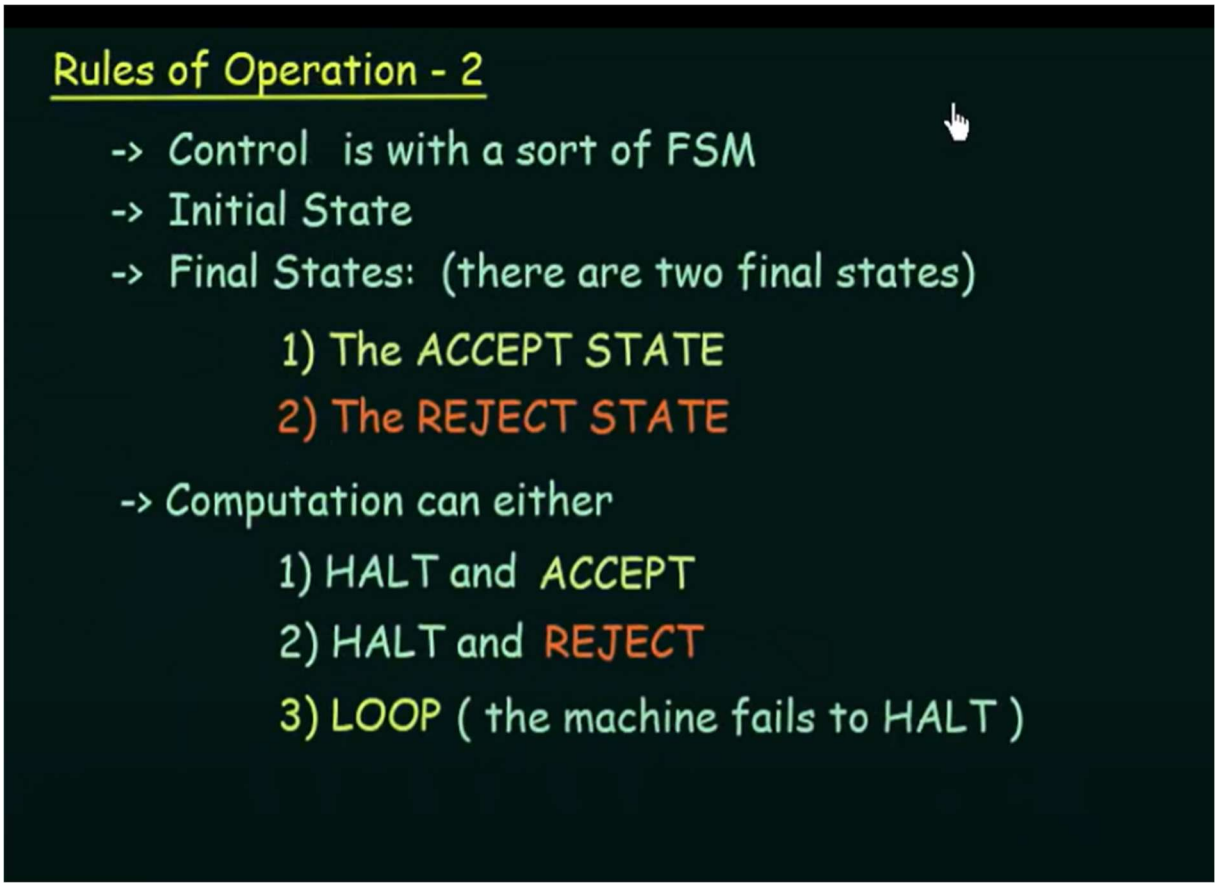
Control portion is the main controlling portion of the Turing machine which takes care of how the control must be handled in the Turing machine.

You can say that the control portion is the Program, and it is deterministic.

Non-determinism does not buy additional power in the case of the Turing machine.



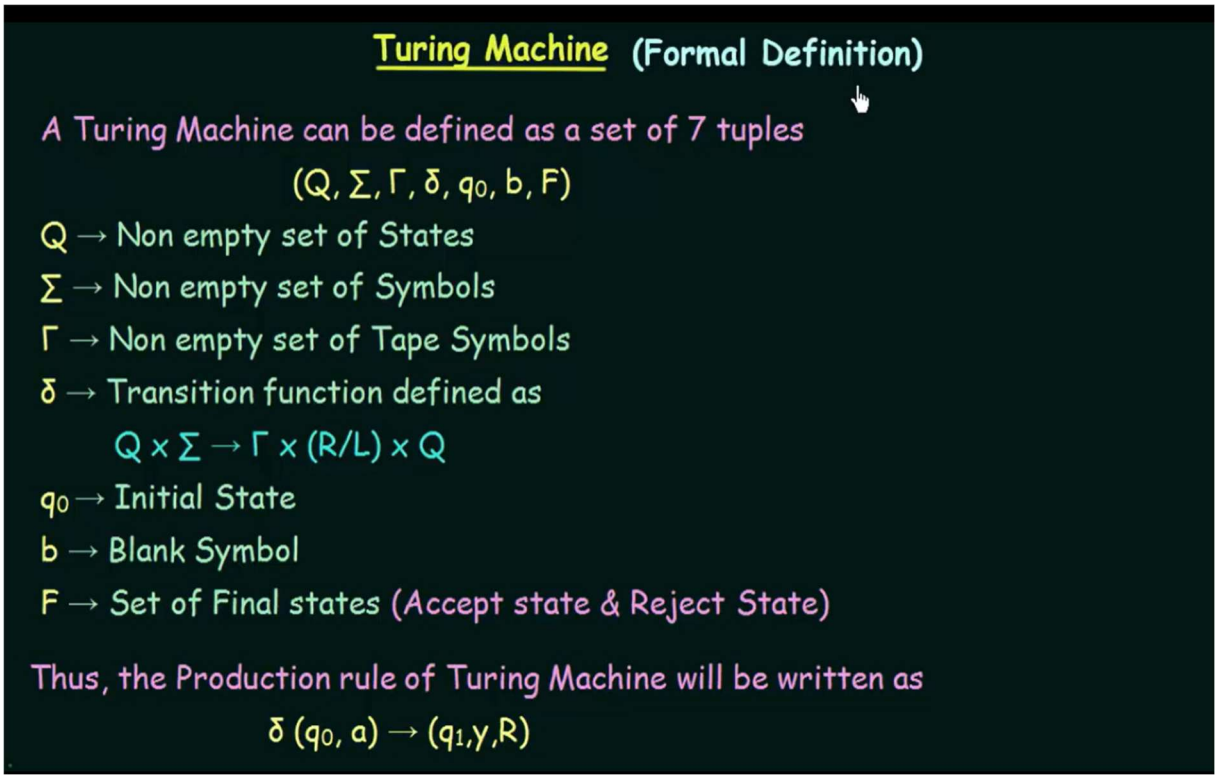
You are going to take this transition only if your tape head is on the symbol a,



Control portion is kind of FSM, initial state as usual, final state there are two final states (accept Type equation here.state and reject state)

Computation (on some input) can halt (= stop) and accept or reject, loop (= machine does not stop)

(that means on some input, the machine can halt and accept or can halt and reject), or on some input, the machine does not stop and goes into infinite loop.



Gamma➔ set of tape symbols (Tape is infinite set of symbols)

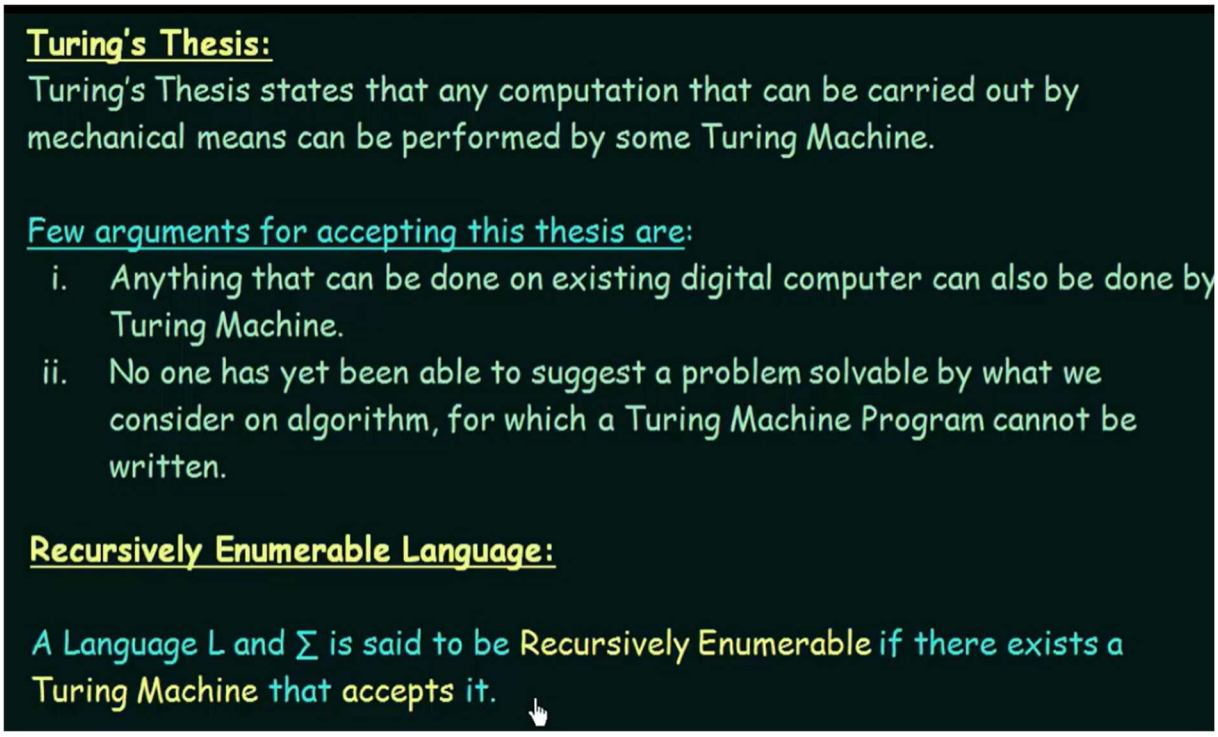
Delta the transition function: Q x ∑ ➔ Γ x (R / L) x Q

Ich bin in einem Zustand and I read an input symbol ➔ then i write the symbol in the tape and the tape head will go to the left or right and go to the next state.

B ➔ is a blank symbol used to fill the empty spaces in the tape and the blank symbol doesn't belong to the set of input symbol

F ➔ set of final state, there is an accepting state and a rejecting state

Production rule (meaning) ➔ initially we are at the starting state q0 and on getting the input symbol a ➔ I go to another state q1 and I write y in the tape I go to the right.

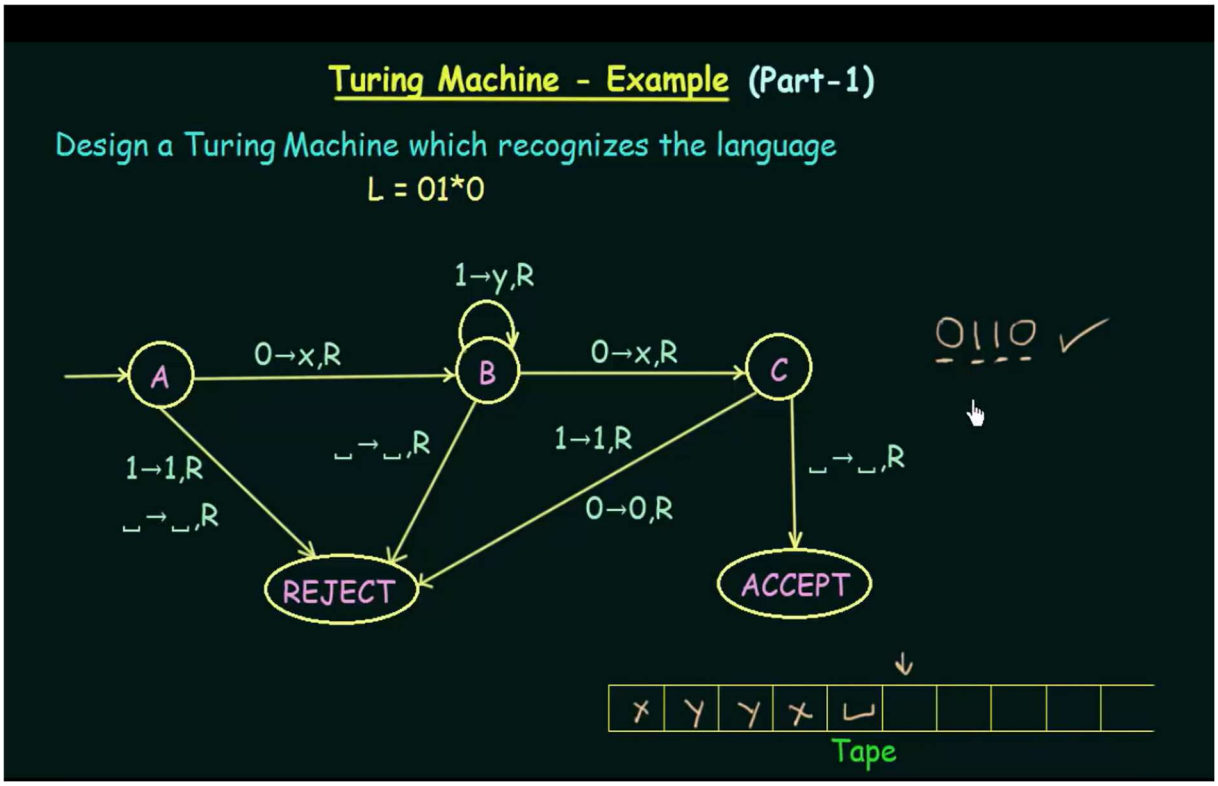


Turing thesis ➔ if there is any computation than can be mechanically performed then that computation can be performed by some Turing machine (so the Turing machine is very powerful)

There are 2 Arguments that support the Turing Thesis (2nd argument ➔ if there is a problem that you have and if you are able to write some Algorithm for that problem, then you can design some Turing machine that can solve that problem).

These 2 arguments that support the Turing thesis

The class of Language that is accepted by the Turing machine is the recursively enumerable language. (If a language is accepted by Turing machine, then that language is a recursively enumerable language, there is also Turing recognizable languages and Turing decidable languages and other languages that can't be accepted by a Turing machine)



Why do we have to write x, y in the tape?

We should write them, because we are using a Turing machine but particularly for this language x and y in the tape they don't have any meaning.

Turing machines are deterministic, means for every state there is a transition for each and every symbol that we have for each and every state.

In this example is ∑ = {0,1} and 

You should have a transition for every symbol in ∑ and also for the blank symbol for each and every state ➔ Deterministic

In this case the Turing machine is deterministic

If there is some edges are missing that means that these edges / transitions go implicitly or by default to the REJECT State.

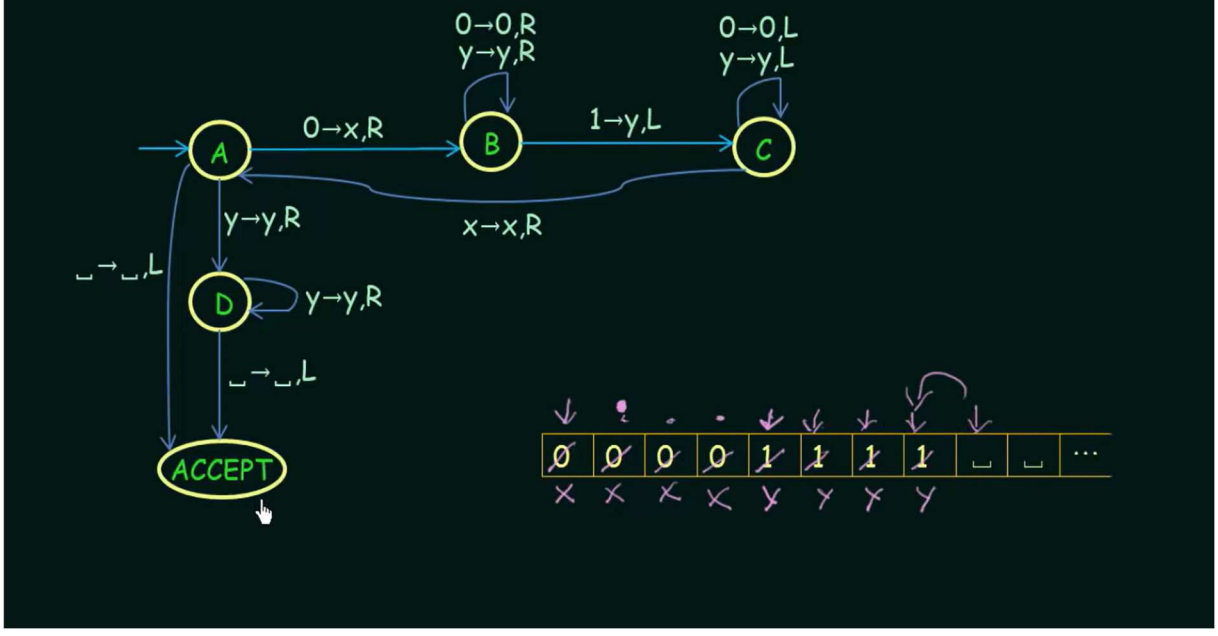
Turing Machine - Example (Part-2)

Design a Turing Machine which recognizes the language $L = 0^N 1^N$



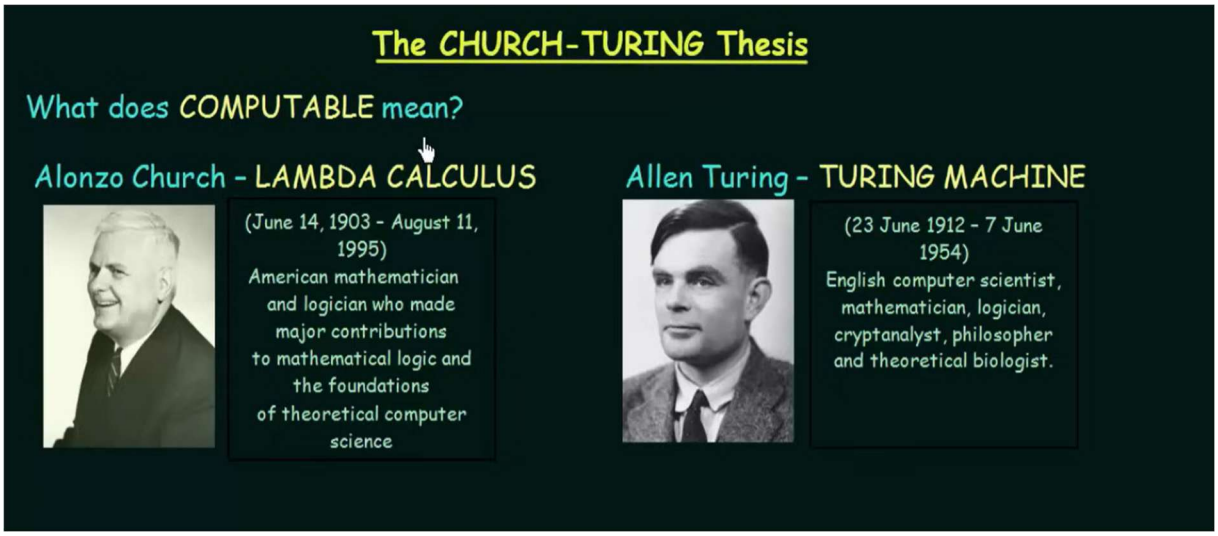
Algorithm:

- Change "0" to "x"
- Move RIGHT to First "1"
  If None: REJECT
- Change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain



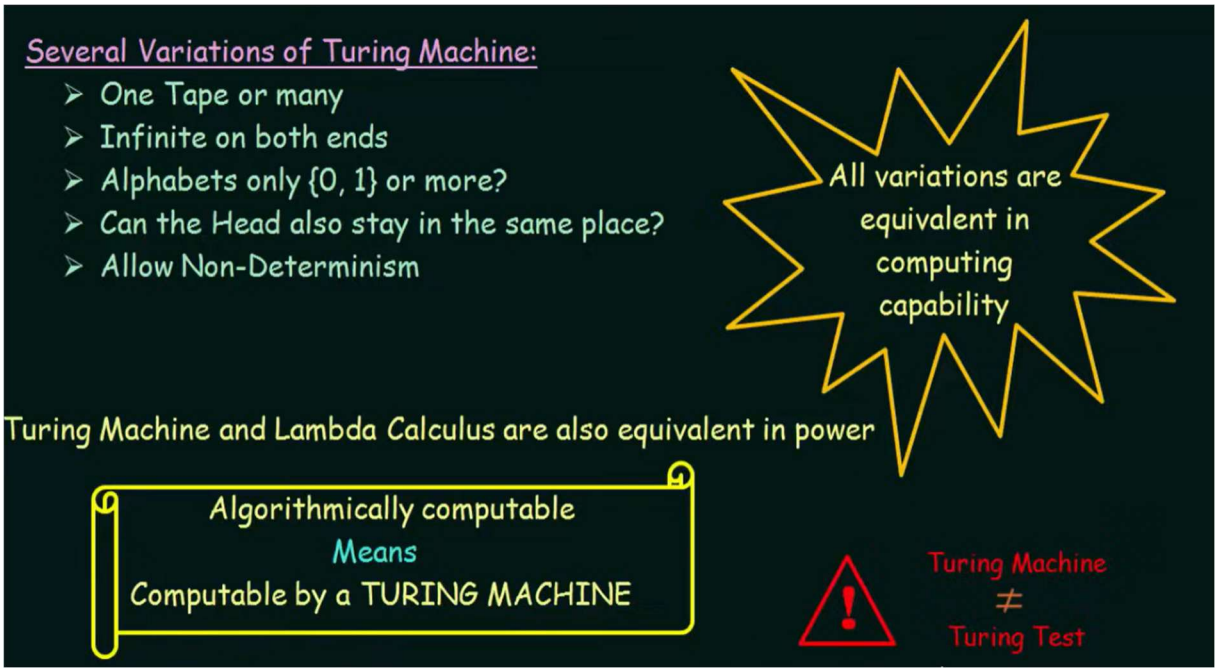
The transition 

is made to accept the empty string =””, that means if you want to the empty string , you read the blank symbol and replace the blank symbol with blank symbol itself (don't replace anything) move the tape head to the left. (but if the tape head is on the far left means don't move the tape head)



Whatever can be computed by lambda calculus, that can be called as computable (Lamda calculus is a machine , this idea came from Alonzo church)

Alan Turing came with the Turing machine, whatever can be computed using Turing machine is called computable.



Each Turing machine has only one Tape , if a turing machine has more than one tape, will the Turing machine with more than one tape perform more powerful computations as compared to the Turing machine with one tape.

We saw that the tape of the Turing machine is infinite sequence of symbols and it has from the left an end but not from the right , if both sides right al left are infinite , will the Turing machine perform more powerful computation than a Turing machine that has a tape where it is only infinite on the left hand side

What if I have more alphabets in my input symbols = ∑ , and what if tape head can stay in the same cell (as we have seen the tape head can move to the left or to the right)

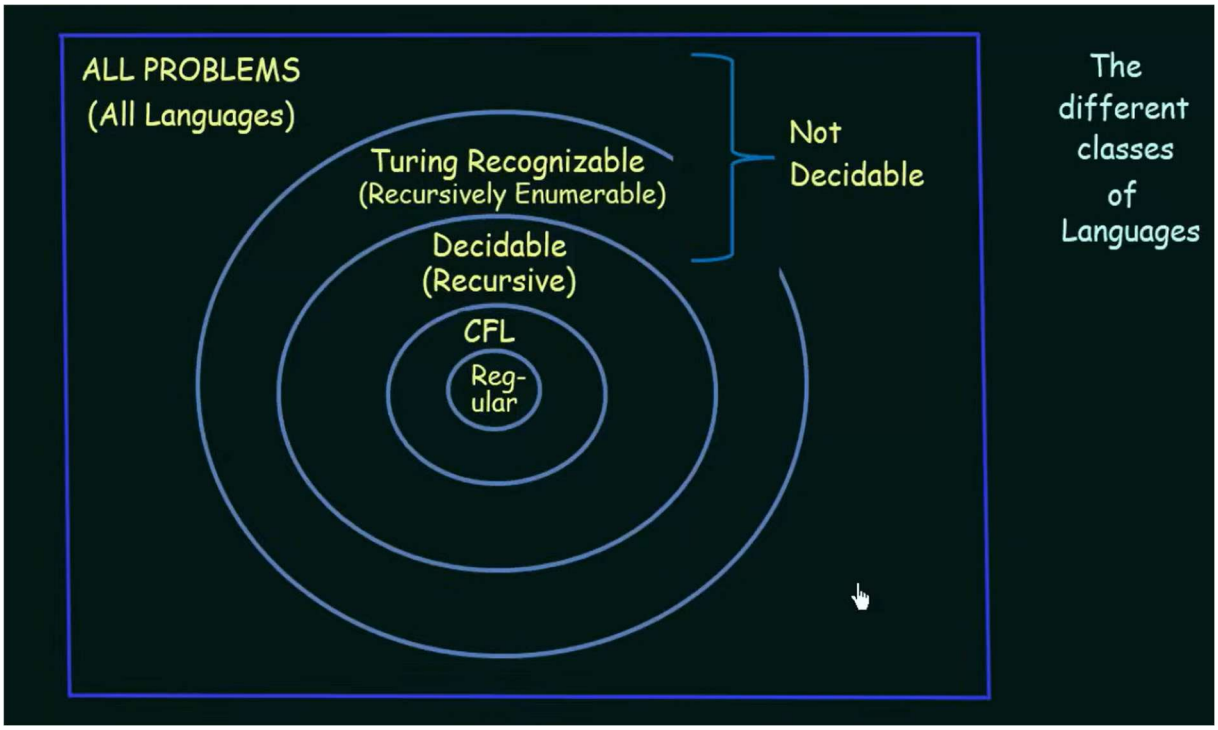
And what if the Turing machine is non-deterministic.

- ⇨ All variations are in their computation power remain the same.

Lamda calculus machine and the Turing machine are equivalent in the power and any task that can be performed using Turing machine is computable . we don't use the lambda calculus as a benchmark to standardize the computation term

If any Problem can be computed using an Algorithm means it can also be computed using Turing machine.

Turing machine ≠ Turing Test



The languages accepted by the Turing machine are the Decidable (we call it also Recursive) as well as the Recursively Enumerable Language (also known as Turing Recognizable)

Decidable Languages are Languages that can be decided using a Turing machine (that means

If you pass a string to a Turing machine, if the string belongs to the language , then the Turing machine will accept it and will halt) if the string does not belong to the language , then the Turing machine will reject the string and will halt ➔ what we say the Turing machine in case of Decidable language will always come to a halt .

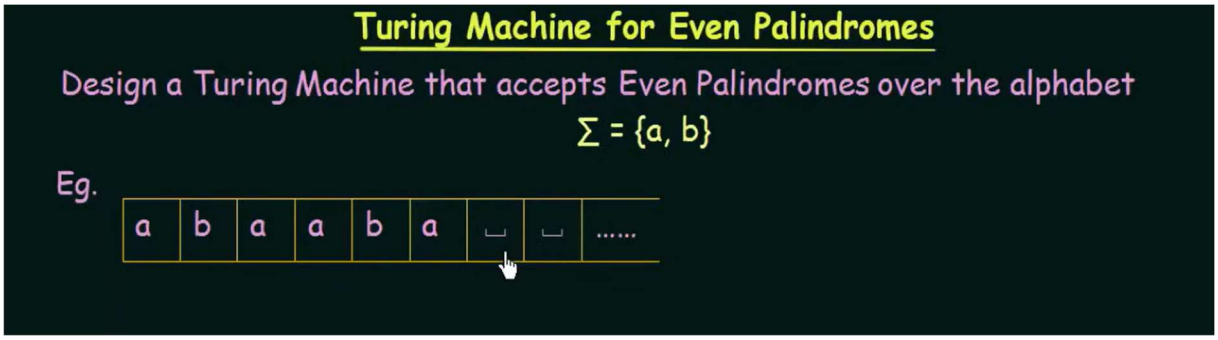
In case Recursively enumerable languages: If you pass a string that belongs to the language , then the Turing machine will accept and halt and if your pass a string that does not belong the language, then the Turing machine may not halt and will go into loop

- ⇨ Halting is not guaranteed in the Turing machine
- ⇨ In decidable language TM will always halt
- ⇨ In REL TM may or may not halt

On Top of that we have the undecidable languages ➔ what every string you pass to the Turing machine, they will never come to a halt.

- This is like a Venn diagram, the square represents all the languages

Example :



Approach:

The string : a b a a b a ➔ is an even palindrome

How are going to design this Turing machine ?

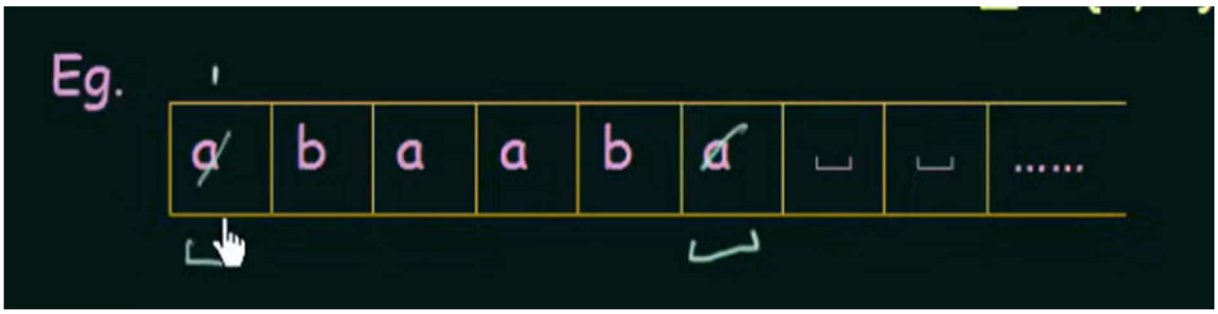
I will put the string in the tape, at each cell will a symbol come and after the last a in the string, a blank symbol will come.

The tape head will be positioned at the first a , I read a and then replace a with the blank symbol , then I will came moving to the right until I reach the end of the string , how can I know that ?

When I encounter a blank symbol on the rightmost , then move to the left and then you find the last symbol of your input string

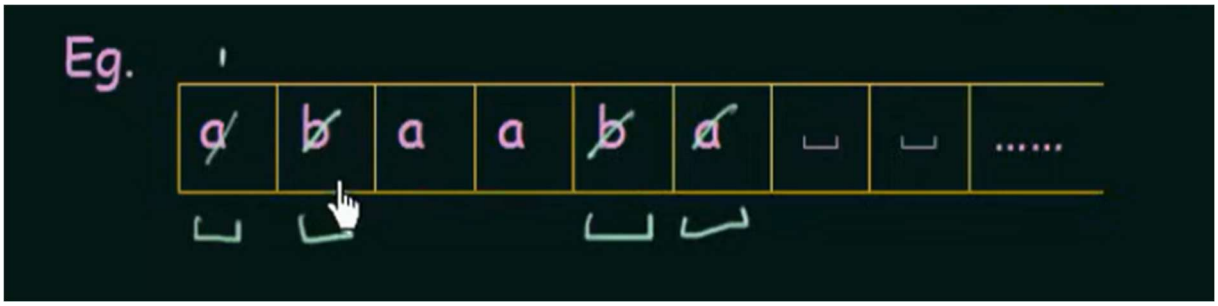
And then if you find your last symbol of your input string, check if it is equal to the first symbol , if they are equal, then replace the last symbol with a blank symbol and keep going with the algorithm, if they are not equal ➔ it isn't a palindrome.

After replacing the last symbol with a blank symbol, keep moving to the right until you reach the rightmost symbol

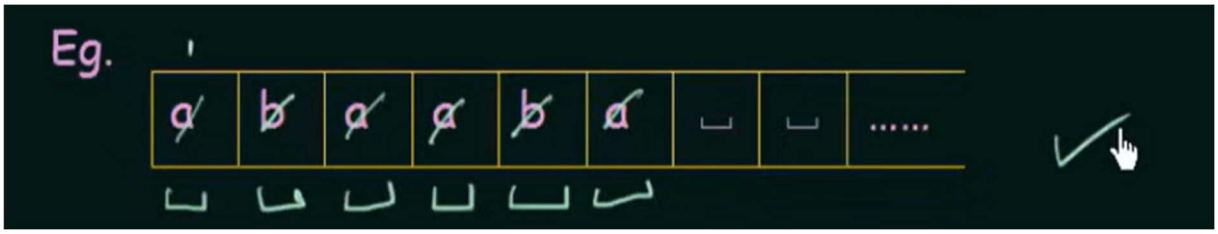


And then you will find b on the right, replace b with a blank symbol and keep going until you find the rightmost symbol ➔ if you reach the rightmost symbol, check if they are equal , if yes then replace it with a blank symbol and go the leftmost symbol

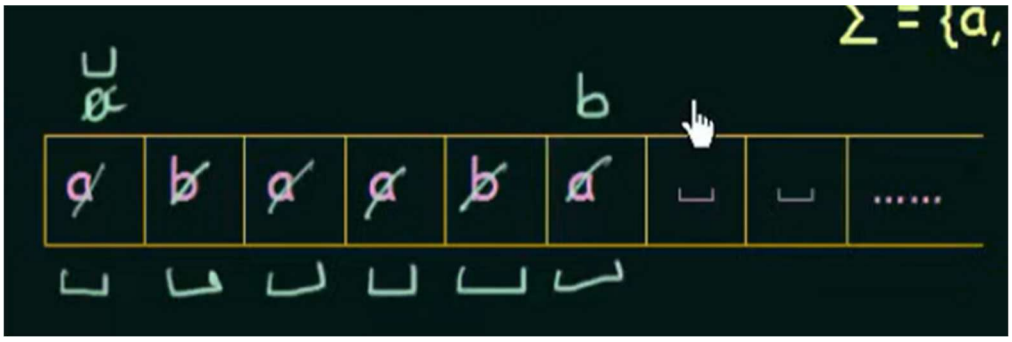
If no ➔ not a palindrome



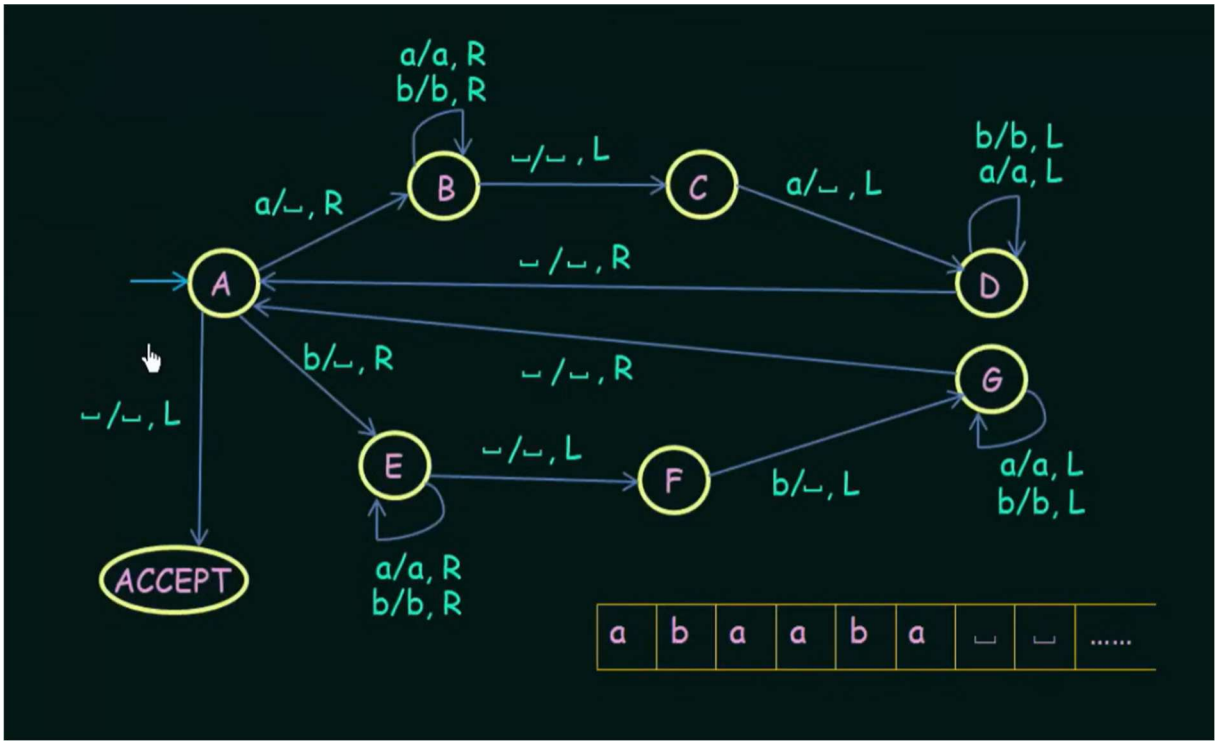
And then continue with the same procedure



If you have only blank symbols at the end ➔ your string has been accepted



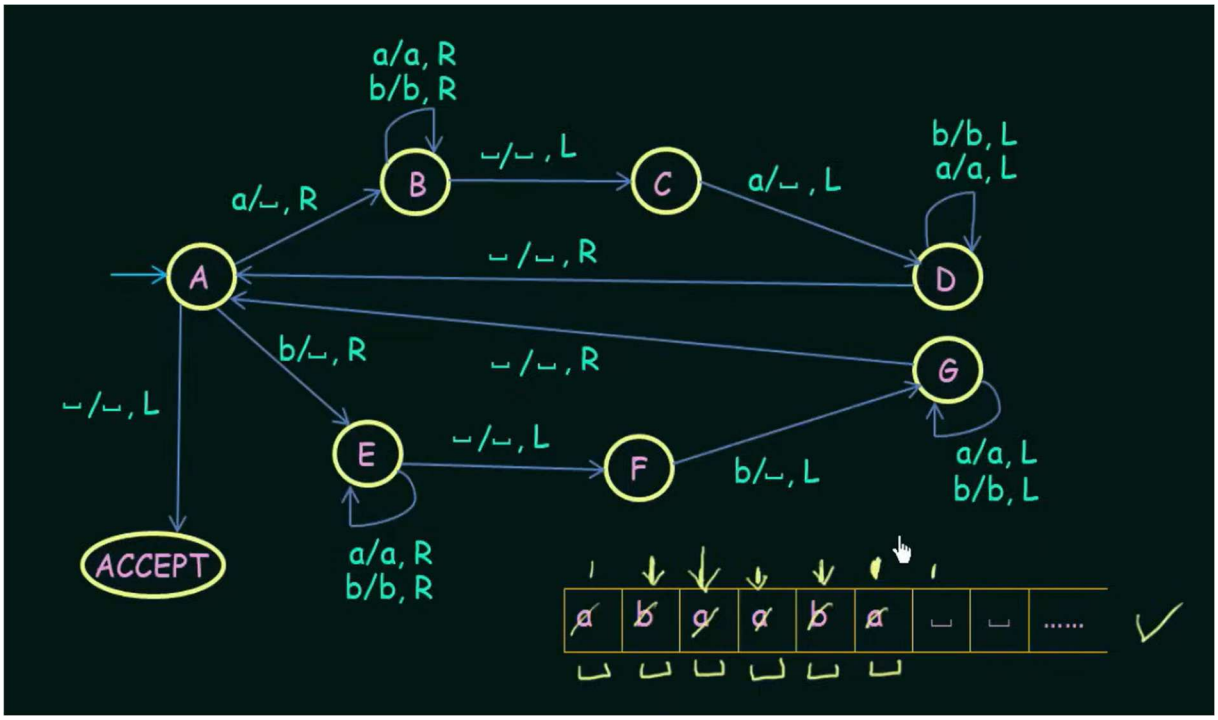
Suppose that the start symbol is a and the last symbol is b ➔ you start with a , replace a with the blank symbol and go the rightmost symbol of the string , you will see b , check if they are equal , in this case you can't replace with a blank symbol ➔ string reject , it isn't a palindrome .



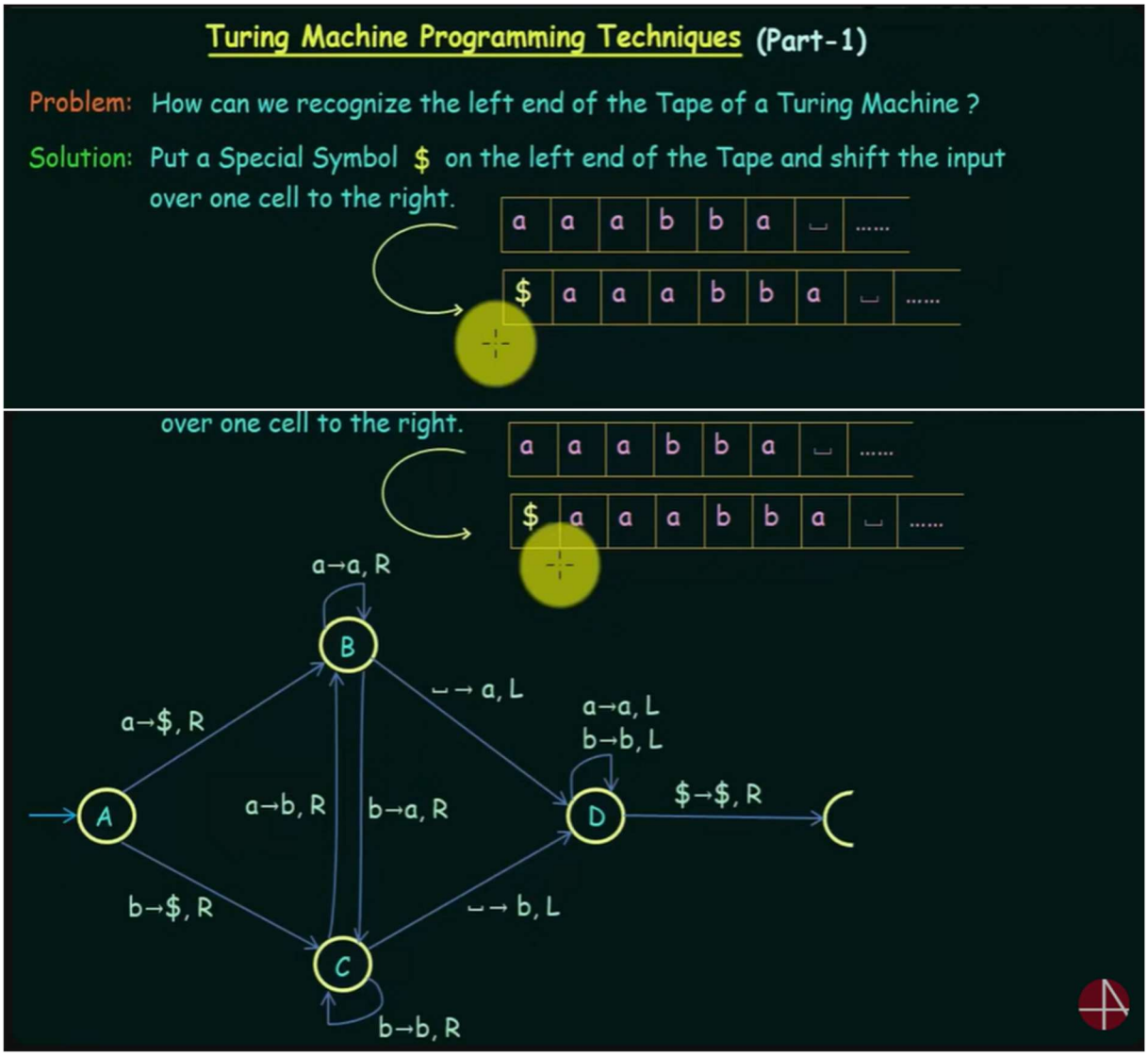
Notation:

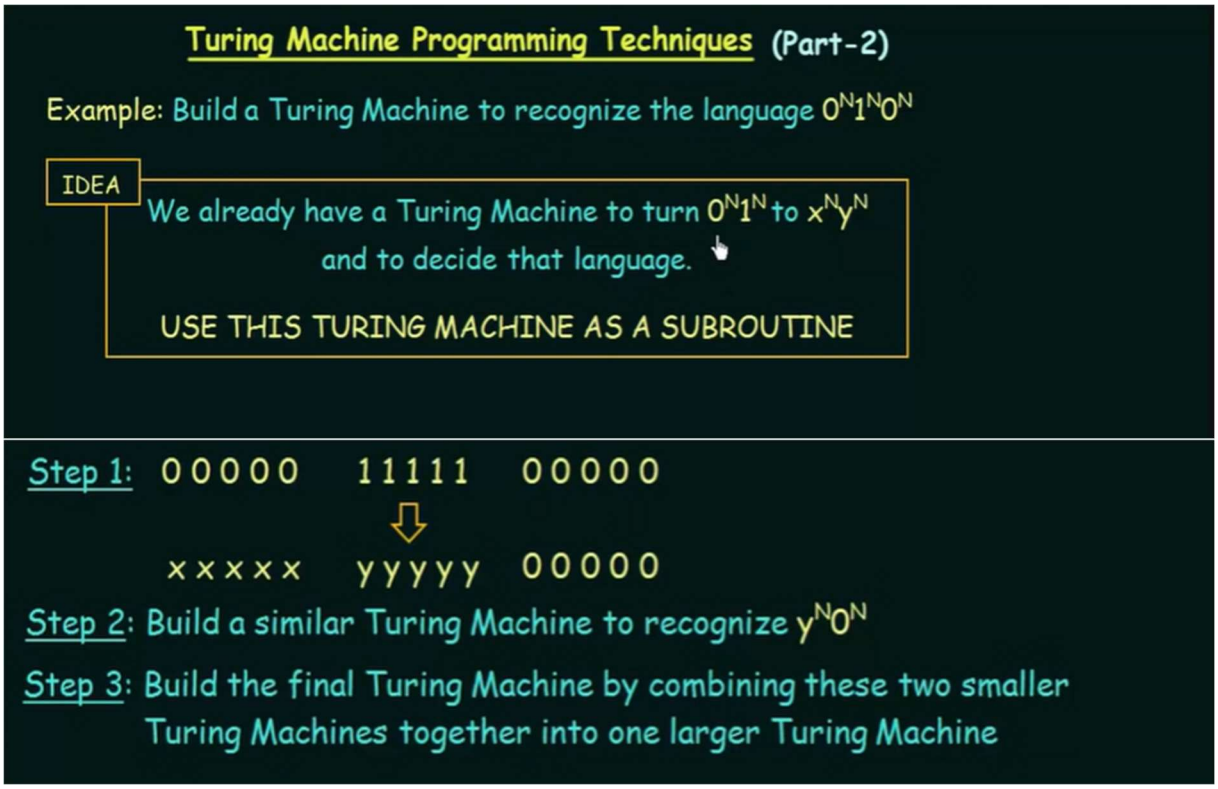
We used to have this notation a ➔ a, R ≡ a / a,R (mean the same thing , but different notations)

As we have said before, if I get an input and the state doesn't have a transition with a particular input, this transition goes by default to the Reject State, for example If you are in the state C , you get the input b , you don't have a transition with the input b, this string goes by default to the REJECT STATE.

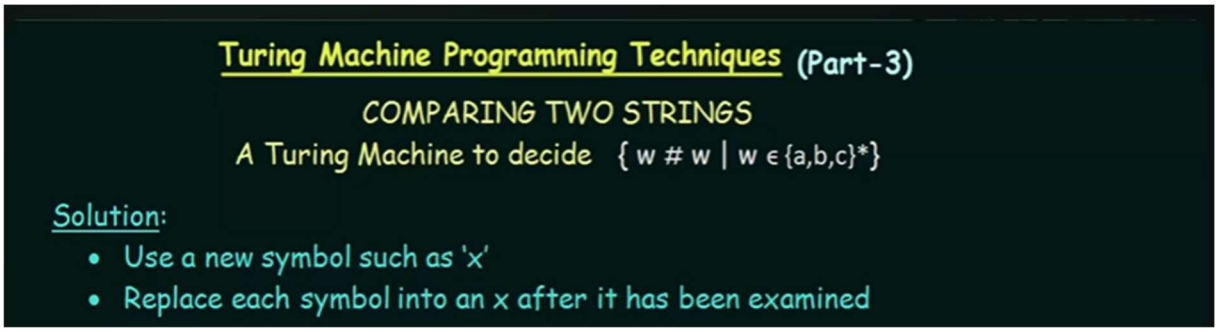


Turing Machine Programming Techniques

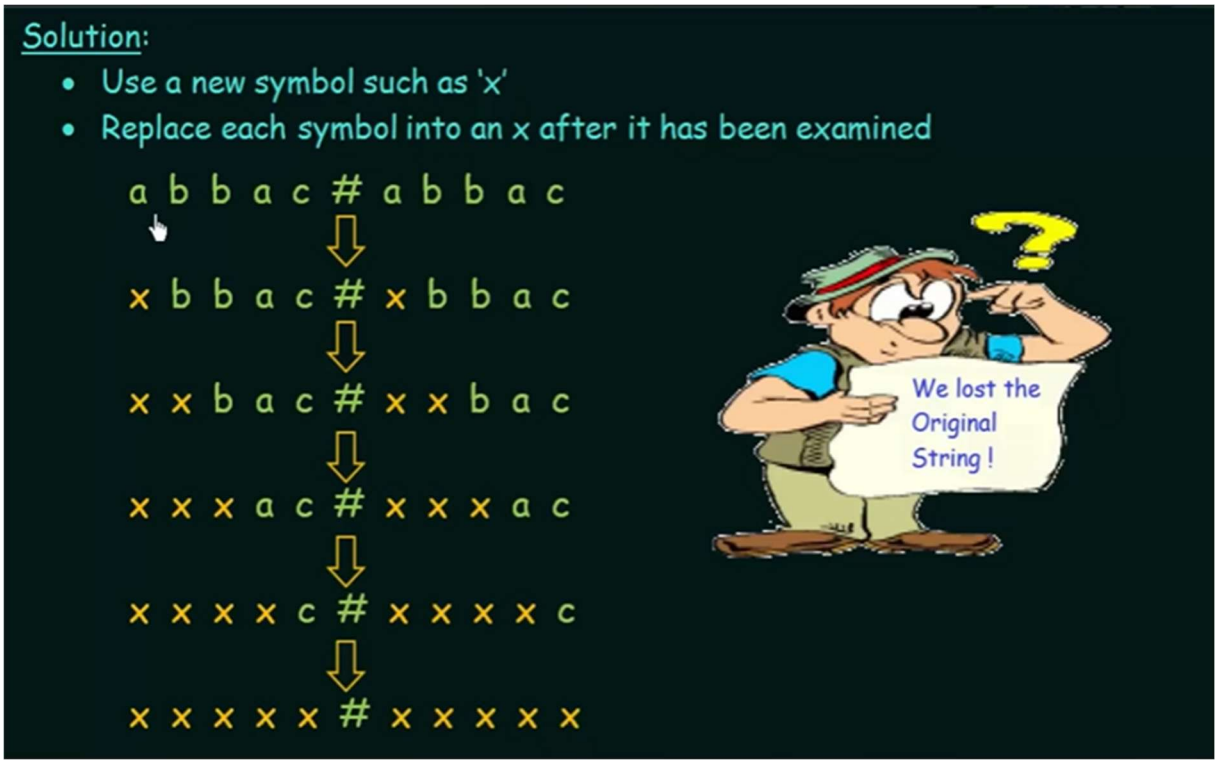




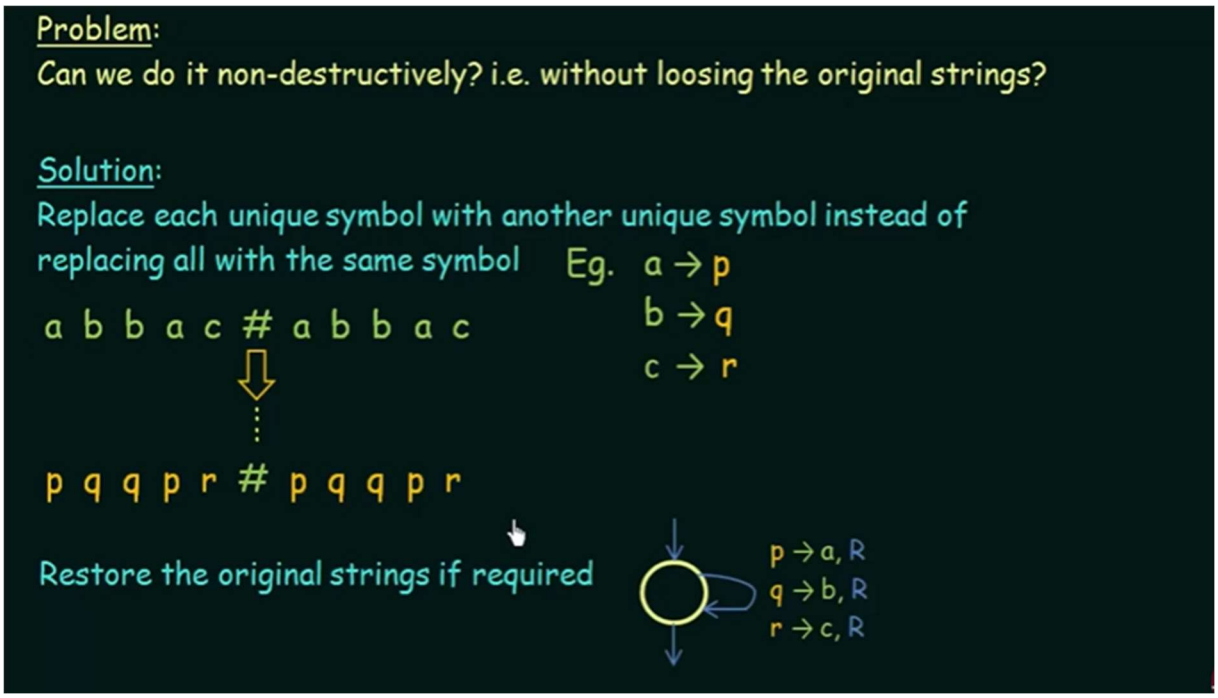
In step 1: we will consider the 1st Part 0^N 1^N and then consider the 2nd Part Y^N 0^N and combine them together.



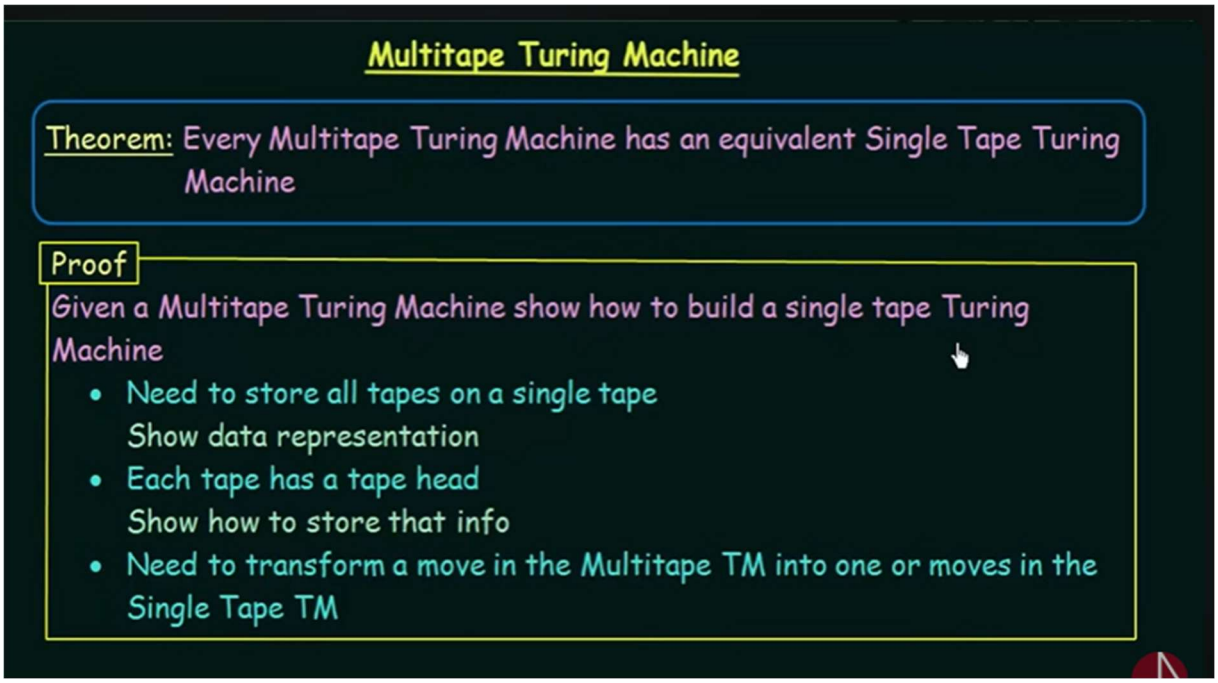
denotes the middle of the string, this Turing machine should decide if the 1st half of the string is equal to the 2nd Half of the String. If they are equal , the Turing machine should accept it if they are not equal, the Turing machine shouldn't accept it

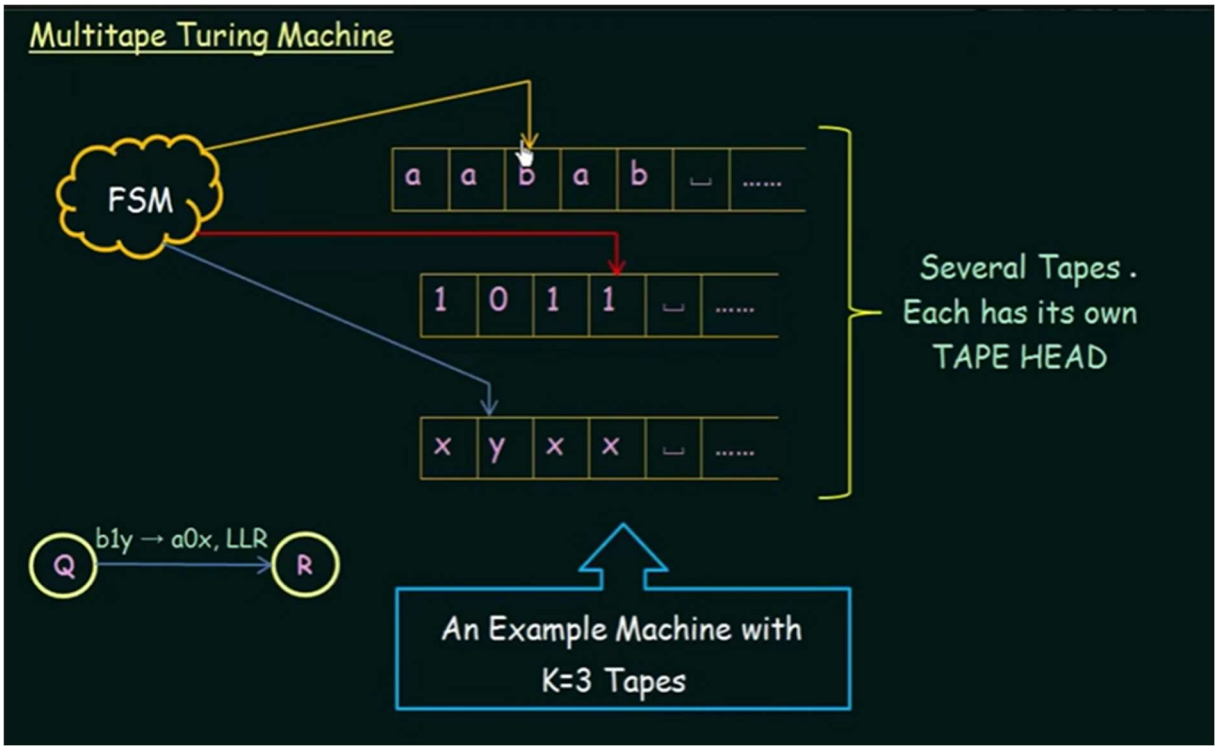


Disadvantage in this method: we've lost the original string

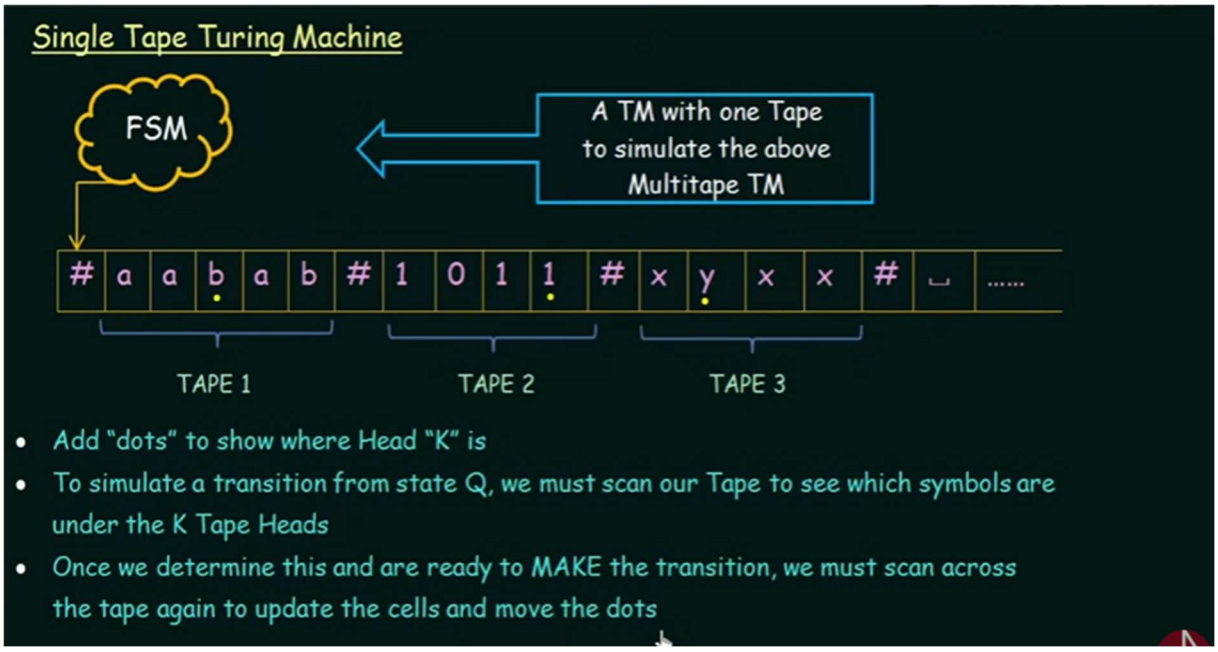


In this method, we can perform this Task and at the same time we can retain our original string without losing it. This technique is known as marking, because we're marking the symbol with another unique symbol.





We have a Turing machine with several taps, and at each tape each Tape head is pointing to another symbol, b1y that means : I read the tape head in the 1st tape head is pointing to b , the 2nd tape head is pointing to 1 , the 3rd tape head is pointing to y , so that means I read b1y and the 1st tape the b will be overwritten by a, the 2nd tape head is pointing to 0 and the 3rd tape head is pointing to x and the 1st tape head will move to the left the 2nd tape head will move to the left and the 3rd tape head will move to the right.



For the 1st step , to add dots to our tape, we use the marking technique.

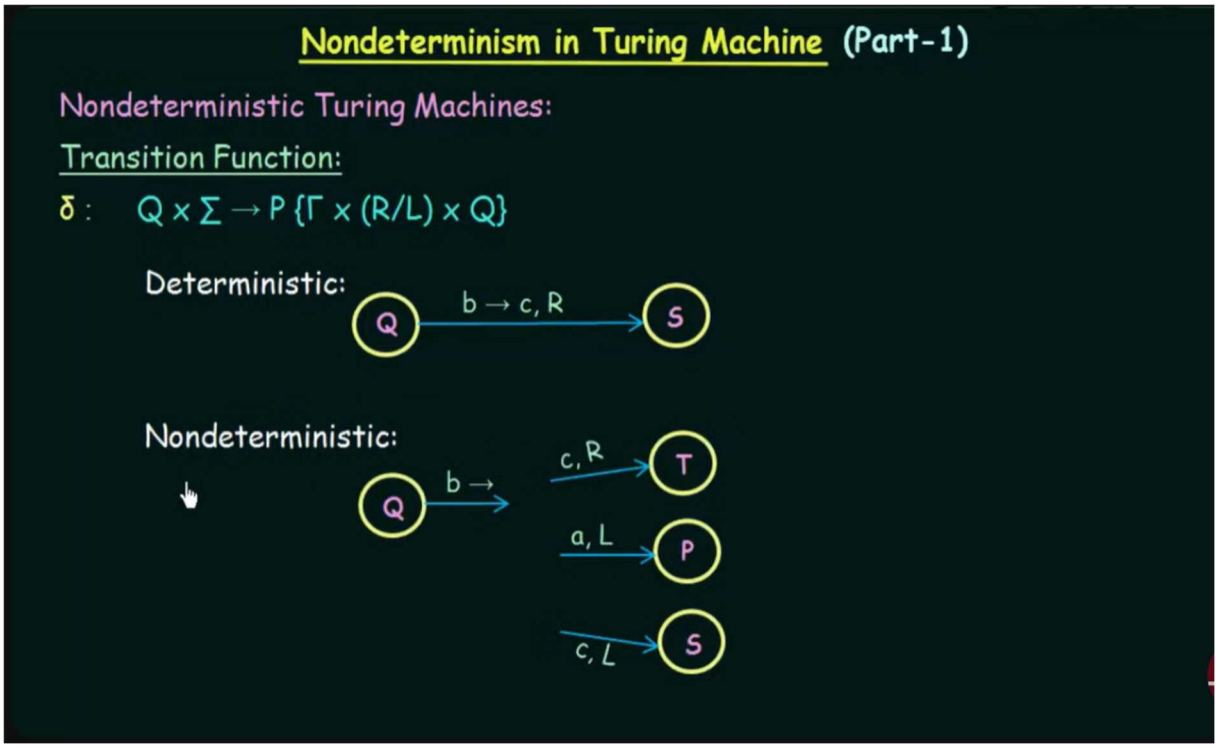
For the 2nd step , to take a transition from one state to another , we should scan the element where the tape head is pointing to, that task can be accomplished using the dots .

For the 3rd step, if we know where the dot is, we should be able to update the symbols and then move the dot to the left or to the right.

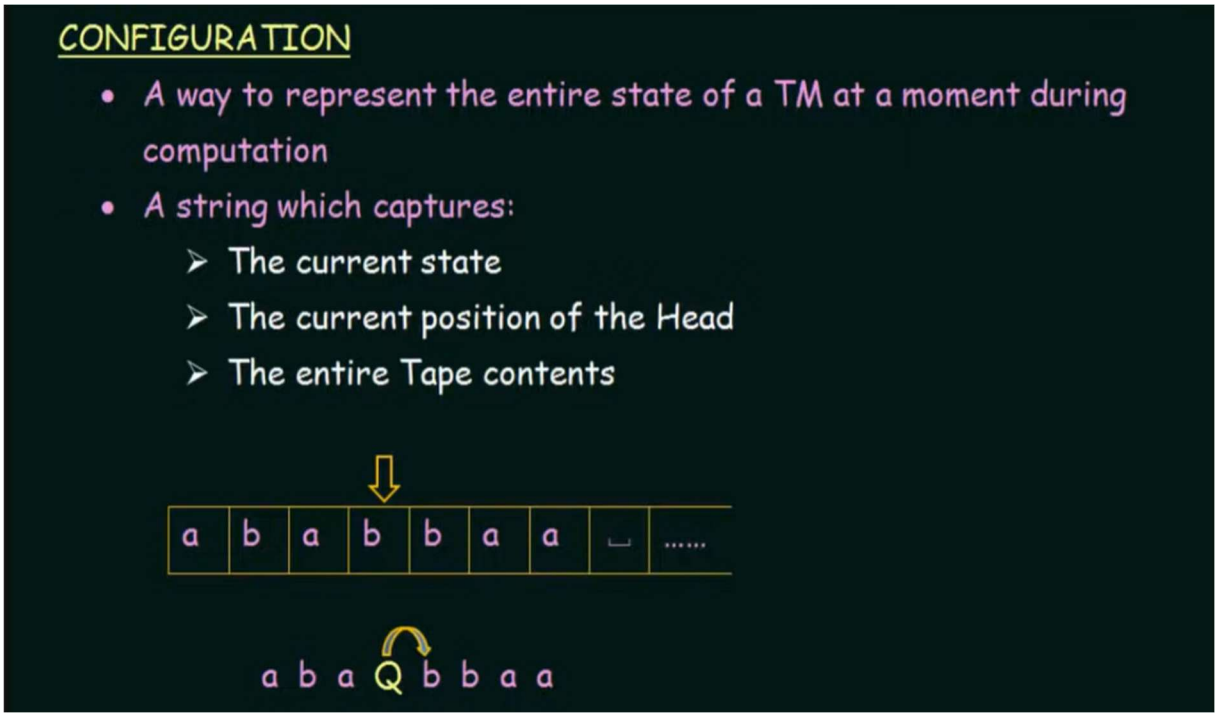
4th step: if we encounter the right end of any tape and we should move to the right , we have to shift the entire string to the right and insert the blank symbol , for example if I am in the section #1011# and the dot is pointing to the last symbol in 1011 and the dot is pointing to the 1 and I should move to the right , then I should add a blank symbol at the of the string 1011 , I can do that by shifting the part #xyxx # one step to the right .

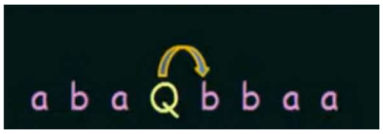
- ⇨ We have seen that we can convert the multitape Turing machine to a single tape Turing machine and they ‘re equivalent, since they ‘re equivalent, we don’t see a Turing machine using a multiple tape to design it and we know that a Turing machine with multiple tapes doesn’t provide us with extra power.

Non-Deterministic Turing Machine

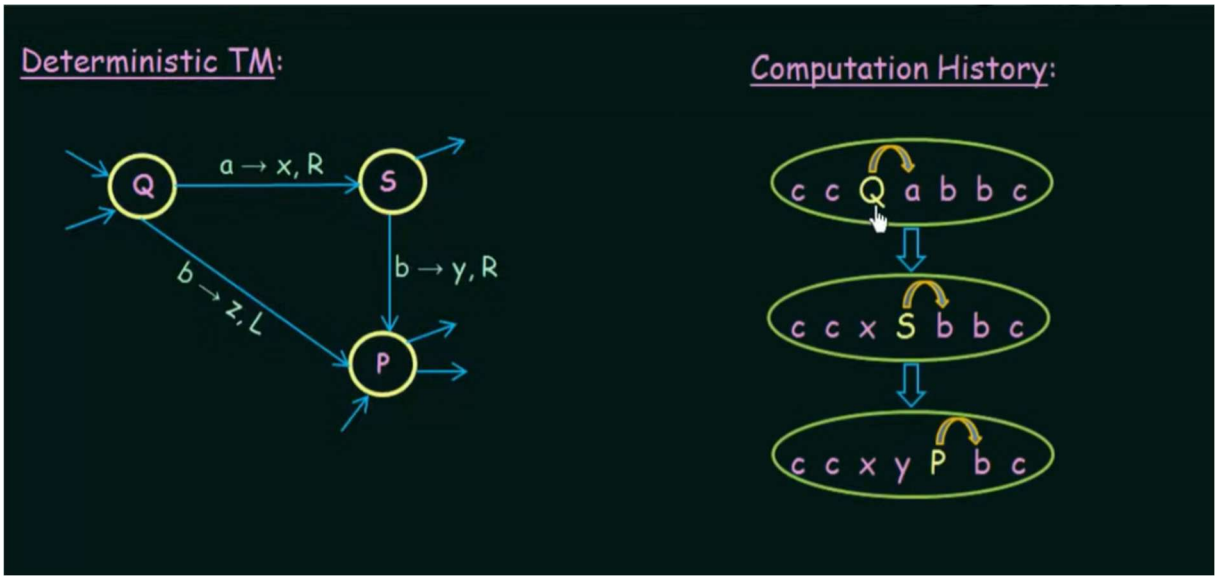


The Difference between Deterministic TM and non-Deterministic Turing Machine is in the Definition, the transition function is defined by the Q x Σ -> P(Γ x (R/L) x Q), where P is the Powerset , in the non-deterministic version of the TM, by a single input, I can go to multiple states.

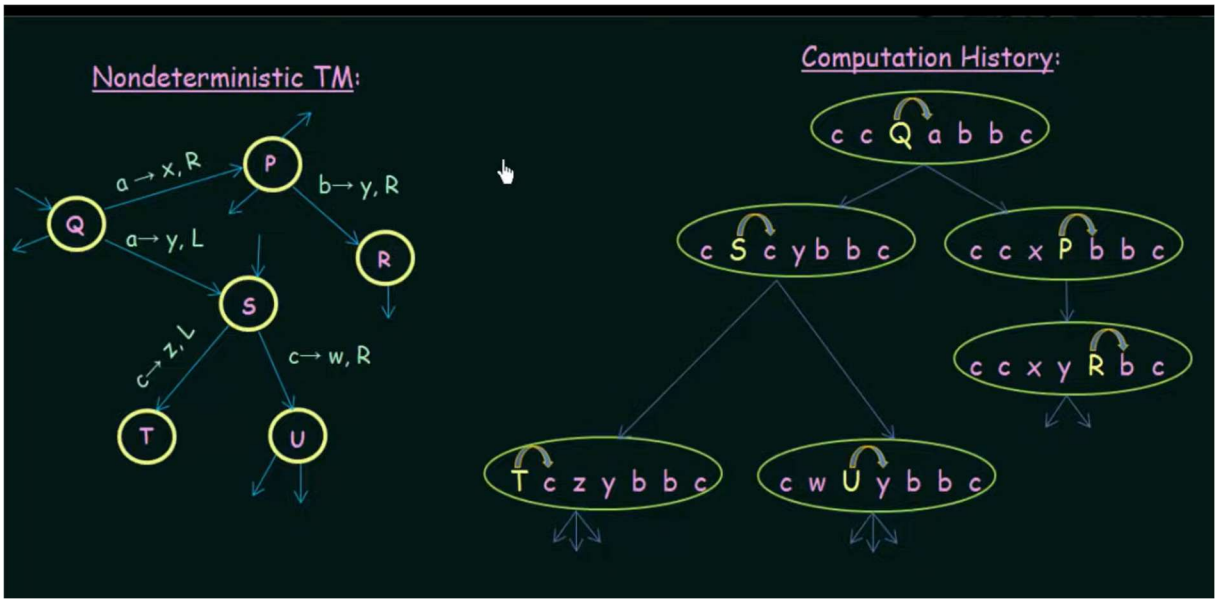




Tape head is pointing to b, b is the symbol where I look at, in my tape and I am in the state Q, that's mean I am in the state Q an I'm looking at the symbol b , The configuration contain all tape content. The Arrow says I am in the state Q and I'm looking at the symbol b that I should read .



I'm in the state Q and I read a, then I should replace a with x and move to the right and will come to the state S, if I reach the State S. then I'm looking at the symbol b, then I should replace b with y and move to the right. Then you reach the state P, and you look at the symbol b .....



Outcomes of a Nondeterministic Computation:

ACCEPT If any branch of the computation accepts, then the nondeterministic TM will Accept.

REJECT If all branches of the computation HALT and REJECT (i.e. no branches accept, but all computations HALT) then the Nondeterministic TM Rejects.

LOOP Computation continues but ACCEPT is never encountered. Some branches in the computation history are infinite.

Nondeterminism in Turing Machine (Part-2)

Theorem: Every NondeterministicTM has an equivalent Deterministic TM

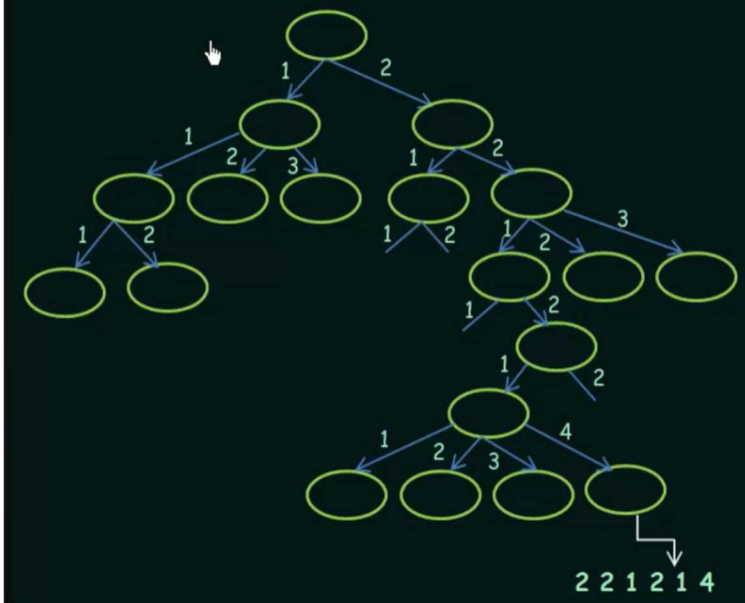
Proof:

- Given a Nondeterministic TM (N) show how to construct an equivalent Deterministic TM (D)
- If N accepts on any branch, the D will Accept
- If N halts on every branch without any ACCEPT, then D will Halt and Reject.

Approach:

- Simulate N
- Simulate all branches of computation
- Search for any way N can Accept

Computational History:



-A Path to any Node is given by a number

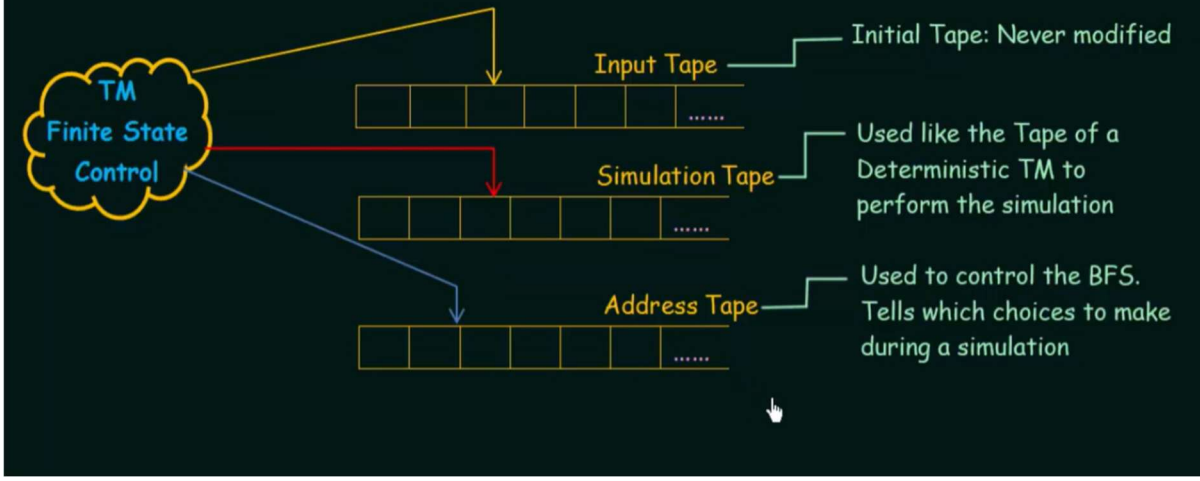
- Search the tree looking for ACCEPT

Search Order:

- DEPTH FIRST SEARCH x
- BREADTH FIRST SEARCH ✓

To examine a node:

- Perform the entire computation from scratch
- The path numbers tells which of the many nondeterministic choices to make



Algorithm:

Initially: TAPE 1 contains the Input
TAPE 2 and TAPE 3 are empty

- Copy TAPE 1 to TAPE 2
- Run the Simulation
- Use TAPE 2 as "The Tape"
- When choices occur (i.e. when Nondeterministic branch points are encountered) consult TAPE 3
- TAPE 3 contains a Path. Each number tells which choice to make
- Run the Simulation all the way down the branch as far as the address/ path goes (or the computation dies)
- Try the next branch
- Increment the address on TAPE 3
- REPEAT

If ACCEPT is ever encountered, Halt and Accept
If all branches Reject or die out, then Halt and Reject

Turing Machine as Problem Solvers

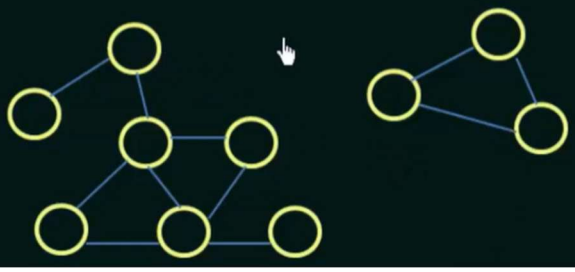
Any arbitrary Problem can be expressed as a language

-Any instance of the problem is encoded into a string

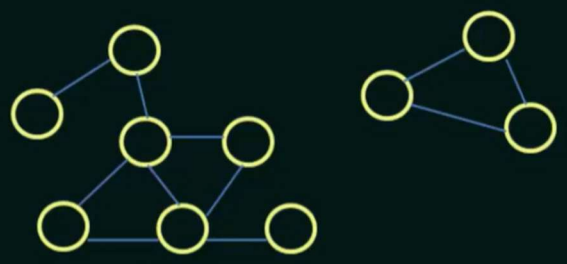
The string is in the language ⇨ The answer is YES

The string is not in the language ⇨ The answer is NO

Example: Is this undirected graph connected?



Example: Is this undirected graph connected?



We must encode the problem into a language.

A = { <G> | G is a connected graph }

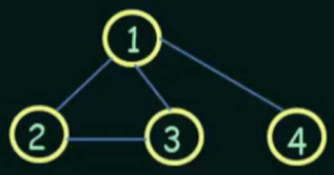
We would like to find a TM to decide this language:

ACCEPT = "YES", This is a connected graph

REJECT = "NO", This is not a connected graph / or this is not a valid Representation of a graph.

LOOP = This problem is decidable. Our TM will always halt

Representation of Graph:



<G> = (1,2,3,4) ((1,2), (2,3), (1,3), (1,4))

List of nodes Edges

Σ = { (,), ,, 1, 2, 3, 4, ……., 0 }

(	1	,	2	,	3	,	4	,	……..	)	)	␣	……

High Level Algorithm:

```
Select a Node and Mark it
REPEAT
    For each node N
        If N is unmarked and there is an edge from N to an already marked
        node
    Then
        Mark Node N
    End

Until no more nodes can be marked
    For each Node N
        If N is unmarked
        Then REJECT
    End
ACCEPT
```

Implementation Level Algorithm:

- Check that input describes a valid graph
- Check Node List
    - Scan "(" followed by digits ...
    - Check that all nodes are different i.e. no repeats
    - Check edge lists ...
        etc.

- Mark First Node
    - Place a dot under the first node in the node list
    - Scan the node list to find a node that is not marked
        etc.

Decidability and Undecidability

Recursive Language:

- A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The Turing machine will halt every time and give an answer (accepted or rejected) for each and every string input.

Recursively Enumerable Language:

- A language 'L' is said to be a recursively enumerable language if there exists a Turing machine which will accept (and therefore halt) for all the input strings which are in 'L'.
- But may or may not halt for all input strings which are not in 'L'.

Decidable Language:

A language 'L' is decidable if it is a recursive language. All decidable languages are recursive languages and vice-versa.

Partially Decidable Language:

A language 'L' is partially decidable if 'L' is a recursively enumerable language.

Undecidable Language:

- A language is undecidable if it is not decidable.
- An undecidable language may sometimes be partially decidable but not decidable.
- If a language is not even partially decidable, then there exists no Turing machine for that language

In Short :

Recursive Language	TM will always Halt
Recursively Enumerable Language	TM will halt sometimes & may not halt sometimes
Decidable Language	Recursive Language
Partially Decidable Language	Recursively Enumerable Language
UNDECIDABLE	No TM for that language

The Universaal Turing Machine

